UNIVERSIDADE FEDERAL DA GRANDE DOURADOS
FACULDADE DE CIÊNCIAS AGRÁRIAS
ENGENHARIA AGRÍCOLA

PROGRAMAÇÃO EM VISUAL BASIC APLICADA À ENGENHARIA AGRÍCOLA

Prof. Cristiano Márcio Alves de Souza

http://www.do.ufgd.edu.br/cristianosouza

Dourados – MS
2009

SUMÁRIO

Páginas

1.0 INTRODUÇÃO 1
1.1 As Origens do Visual Basic....1
1.2 Pensando Como um Programador....1
1.3 Planejando o Programa....1
1.4 Fazendo Perguntas a Você Mesmo....2
1.5 Testando, Compilando e Distribuindo o Programa....2
1.6 Testando o Programa....3
1.7 Compilando o Programa....3
1.8 Distribuindo o Programa....3
2.0 FUNDAMENTOS SOBRE PROGRAMAÇÃO 4
2.1 Sistemas de Linguagem....4
2.2 Estrutura de Programação no Visual Basic....4
2.3 A Caixa de Ferramenta....5
3.0 TRABALHANDO COM CONTROLES, MENU E OBJETO DE DIÁLOGO COMUNS 6
3.1 A Utilização Básica dos Controles: O Programa "Alô Engenheiro Agrícola!"....6
3.2 Utilizando os Objetos do Sistema de Arquivos....7
3.3 O Programa Procurar....7
3.4 Utilizando um Objeto OLE para Iniciar Aplicativos....8
3.5 Instalando Controles Personalizados....9
3.6 Trabalhando com Menu e Objeto de Diálogo Comum....10
3.7 Acrescentando Caixa de Diálogo Não Personalizada aos Programas....15
4.0 VARIÁVEIS E OPERADORES DO VISUAL BASIC 15
4.1 A Anatomia de uma Instrução de Programa em Visual Basic....15
4.2 Utilizando Variáveis para Armazenar Informações....16
4.3 Fazendo as Reservas para as Variáveis: A Instrução Dim....16
4.4 Declarando Variáveis sem Dim....16
4.5 Convenções para Nomear Variáveis....17
4.6 Utilizando uma Variável para Armazenar Entradas....17
4.7 O Que é uma Função....18
4.8 Utilizando uma Variável para Saída de Dados....18
4.9 Trabalhando com Tipos de Dados Específicos....18
4.10 Constantes: Variáveis que não se Alteram....20
4.11 Trabalhando com Operadores do Visual Basic....20
4.12 Utilizando Operadores Avançados: \, Mod, ^ e &....20
4.13 Precedência de Operador....21
4.14 Funções Matemáticas do Visual Basic....22
5.0 UTILIZANDO ESTRUTURA DE DECISÃO 22
5.1 Programação Orientada por Evento....22
5.2 Eventos Suportados pelos Objetos do Visual Basic....23
5.3 Utilizando Expressões Condicionais....23
5.4 Estruturas de Decisão *If ... Then*....24
5.5 Testando Várias Condições em um Estrutura de Decisão *If...Then*....24
5.6 Utilizando Operadores Lógicos em Expressões Condicionais....26
5.7 Estruturas de Decisão *Select Case*....27
5.8 Utilizando Operadores de Comparação com a Estrutura *Select Case*....27
6.0 UTILIZANDO LOOPS E TEMPORIZADORES 29
6.1 Escrevendo Loops *For...Next*....29
6.2 Loops *For...Next* Complexos....31
6.3 Instrução *Exit For*....32
6.4 Escrevendo Loops *Do*....32
6.5 Evitando um Loop sem Fim....33
6.6 Utilizando a Palavra-Chaves *Until* em Loops *Do*....34
7.0 TRABALHANDO COM FORMULÁRIOS, IMPRESSORAS E SUB-ROTINAS DE TRATAMENTO DE ERROS 34
7.1 Acrescentando Novos Formulários ao Programa....34
7.2 Como os Formulários São Utilizados....35
7.3 Instruções do Formulário em Código de Programa....35
7.4 Ocultando e Descarregando Formulários....36
7.5 Formulários MDI: Janelas que Possuem Relacionamento Pai-Filho....36

7.6 Enviando as Saídas do Programa para uma Impressora 37
7.7 Imprimindo um Formulário Inteiro Utilizando o Método PrintForm 38
7.8 Processando Erros Utilizando as Sub-rotinas de Tratamento de Erros 39
7.9 Quando utilizar as Sub rotinas de Tratamento de Erros 39
7.10 Definindo a Armadilha: A Instrução On Error 39
7.11 Outras Técnicas sobre Sub-rotinas de Tratamento de Erros 41
7.12 Especificando um Período de Nova Tentativa 41
8.0 ACRESCENTANDO ARQUIVOS GRÁFICOS E EFEITOS ESPECIAIS 42
8.1 Acrescentando Arquivos Gráficos Utilizando os Controles Line e Shape 42
8.2 O Controle Line 42
8.3 O Controle Shape 42
8.4 Criando Botões de Comando Gráficos 43
8.5 Detectando um Evento MouseDown 44
8.6 Permuta de Botões 44
8.7 Desenho com o Visual Basic 47
8.8 Desenho de Pontos 47
8.9 Seleção de Cores 49
8.10 Desenho de Linhas 50
8.11 Desenho de Retângulos 52
8.12 Preenchendo Figuras com Padrões 54
8.13 Desenho de Círculos 54
9.0 TRABALHANDO COM ARQUIVO TEXTO E BANCOS DE DADOS 56
9.1 Exibindo Arquivos de Texto Utilizando um Objeto Caixa de Texto 56
9.2 Abrindo um Arquivo de Texto para Entrada de Dados 57
9.3 A Instrução Open 57
9.4 Criando um Novo Arquivo de Texto em Disco 59
9.5 Gerenciando Bancos de Dados 60
9.6 Criando Aplicativos de Banco de Dados Personalizados 60
9.7 Utilizando Controles de Limite para Exibir Informação de Banco de Dados 61
9.8 Utilizando Objetos Caixa de Texto para Exibir Dados 61
9.9 Utilizando um Objeto Recordset 62
9.10 Acrescentando registros ao Banco de Dados Biblio.mdb 63
9.11 Excluindo Registros do Banco de Dados Biblio.mdb 64
9.12 Fazendo uma Cópia de Segurança de um Arquivo 65
10.0 CRIANDO UM PROGRAMA DE INSTALAÇÃO 66
10.1 Criando um Programa de Instalação 66
10.2 O Setup Wizard vs. Setup Toolkit 66
10.3 Recursos do Setup Wizard 67
10.4 Criando um Programa de Instalação 67
10.5 Recursos Avançados 68
10.6 Arquivos que Você Está Autorizado a Distribuir 68
10.7 Utilizando o Setup Wizard do Visual Basic 68
10.8 Arquivos de Dependência Explicados 69
10.9 Gerando Arquivos de Dependência com o Setup Wizard 71
10.10 Opções de Distribuição 72
10.11 Trabalhando com Modelos do Setup Wizard 74
10.12 Instalando Data Access Components 74
10.13 Instalando os componentes Remote Automation e Distributed COM 75
10.14 Descarga de Componente da Internet 77
11.0 BIBLIOGRAFIA 81
APÊNDICE A 82
APÊNDICE B 84

1.0 INTRODUÇÃO

1.1 As Origens do Visual Basic

A linguagem Basic foi criada por John G. Kemeny e Thomas E. Kurtz em 1963 no Dartmouth College. Ela rapidamente tornou-se popular como uma linguagem de ensino nas universidades e escolas, e foi adaptada para o uso em computadores pessoais por Bill Gates em meados dos anos 70. Desde aquele tempo, foram feitas várias versões atualizadas do Basic criado para computadores pessoais, incluindo o Microsoft Quick Basic e MS-DOS Qbasic. Embora o ambiente de desenvolvimento totalmente gráfico e a linguagem de programação do Visual Basic constituem um grande contraste com os primeiros interpretadores do Basic, a elegância e a simplicidade do Basic original permanece em grande parte na linguagem. O poder do Visual Basic e a sua facilidade de uso são as principais razões dele ter sido escolhido como a linguagem para os aplicativos do Windows, como, por exemplo, o Excel.

1.2 Pensando Como um Programador

Se você nunca escreveu um programa, deve estar curioso sobre o que exatamente é um programa e como criar um em Visual Basic. Esta seção fornece uma visão geral do processo de programação.

Um programa em Visual Basic é um aplicativo para Windows

Um *programa* é um conjunto de instruções que coletivamente faz com que o computador desempenhe uma tarefa útil, tal como processar um arquivo gráfico eletrônico ou gerenciar arquivos em uma rede. Ele pode ser um pequeno utilitário, tal como um programa que calcula a hipoteca de uma casa, ou pode ser um aplicativo sofisticado, tal como o Microsoft Excel. Um programa em Visual Basic é um aplicativo especificamente para Windows, utilizando o Sistema de Programação do Visual Basic. Para escrever um programa em Visual Basic, você determina o que quer que o programa faça, projeta a tela - ou a interface do usuário - para o programa e, então, constrói o programa, utilizando o Sistema de Programação do Visual Basic. Vamos examinar cada um destes passos em detalhe.

1.3 Planejando o Programa

O primeiro passo em programação é determinar exatamente o que você quer que o seu programa cumpra. Isso parece simples (não é óbvio?), porém, sem o estabelecimento de uma missão, mesmo o melhor programador pode ter problemas ao construir um aplicativo que o deixe satisfeito. Planejar um programa é um pouco parecido com planejar um churrasco. Para um churrasco transcorrer tranqüilamente, você precisa prepará-lo com antecedência. Precisa organizar o menu, convidar os seus amigos, comprar a comida e (muito provavelmente) limpar a casa. Embora um churrasco possa ser um divertimento, quando os amigos simplesmente aparecem trazendo os ingredientes, os programas em geral não são melhores se forem construídos com a mentalidade de "sopa de letrinhas".

O primeiro passo em programação é planejar

Quando você estiver aprendendo a programar, você deverá gastar algum tempo refletindo sobre o problema de programação que estiver tentando resolver, bem antes de se sentar em frente ao seu computador. O planejamento antecipado poupa o seu tempo de desenvolvimento ao longo do caminho, e você provavelmente ficará muito mais contente com o resultado. Uma parte do planejamento poderá ser a criação de uma lista ordenada de passos de programação chamada *algoritmo.* Algoritmo é o método de resolução de um determinado problema, onde estão dispostos todos os passos e regras fundamentais para a obtenção dos resultados. O algoritmo para um churrasco pode ser como a ilustração a seguir.

	Missão: Churrasco no próximo Sábado
●	
	Planejando: - *Decidir o menu (tipos de carnes)*
	- Enviar os convites
	- Limpar a casa
	- Prepara a churrasqueira
	Dia do Churrasco: - Comprar as coisas
	- Acender a churrasqueira
	- Preparar a comida
	- Futebol e baralho
●	
	Depois: - Limpar a bagunça

Figura 1.3 – Algoritmo para um churrasco

A interface do usuário compreende todas as telas e imagens utilizadas no Programa

Se estiver familiarizado com os aplicativos baseados em Windows, você reconhecerá a maioria dos elementos de interface do usuário; os *Menus* contém muitos dos *comandos* disponíveis no aplicativo baseado em Windows; *as caixas de diálogo, os botões e o ponteiro do mouse* ajudam o usuário a fornecer informações ao programa; e as *janelas e barras de rolagem* permitem que o usuário visualize e navegue pelas informações exibidas na tela. Para maximizar o funcionalidade do seu programa em Visual Basic é importante que você utilize os elementos de interface do usuário do Windows de maneira padronizada e previsível. A Microsoft publicou um conjunto de diretrizes de projeto para os aplicativos baseados em Windows e você aprenderá sobre as diretrizes de interface mais importantes durante a leitura.

1.4 Fazendo Perguntas a Você Mesmo

Quando planejar os projetos de programação em Visual Basic, você poderá achar útil criar um esboço da interface do usuário que gostaria de apresentar aos seus usuários. Ao fazê-lo, isso ira ajudá-lo tanto a projetar a interface, quanto a pensar sobre como o seu programa deve funcionar. Tal procedimento poderá ser útil para fazer a si mesmo as seguintes questões sobre o seu programa:

- Qual é o objetivo, ou a missão, do programa que eu estou escrevendo?
- Quem utilizará o programa?
- Como o programa irá se parecer quando iniciar a execução?
- Quais as informações que o usuário inserirá no programa?
- Como o programa processará a entrada de dados?
- Quais informações (saída) o programa produzirá?
- Como os resultados serão apresentados ao usuário?

1.5 Testando, Compilando e Distribuindo o Programa

Após criar uma versão de trabalho do seu programa, você precisa testá-lo cuidadosamente, compilá-lo em um programa executável (isto é, um programa Windows dedicado) e distribuí-lo para os seus usuários. Se você decidir revisar o programa mais tarde, repita o processo, começando com o planejamento e uma análise de quaisquer novos objetivos. Esses passos completam o ciclo de vida de desenvolvimento do software, o qual é ilustrado no diagrama a seguir:

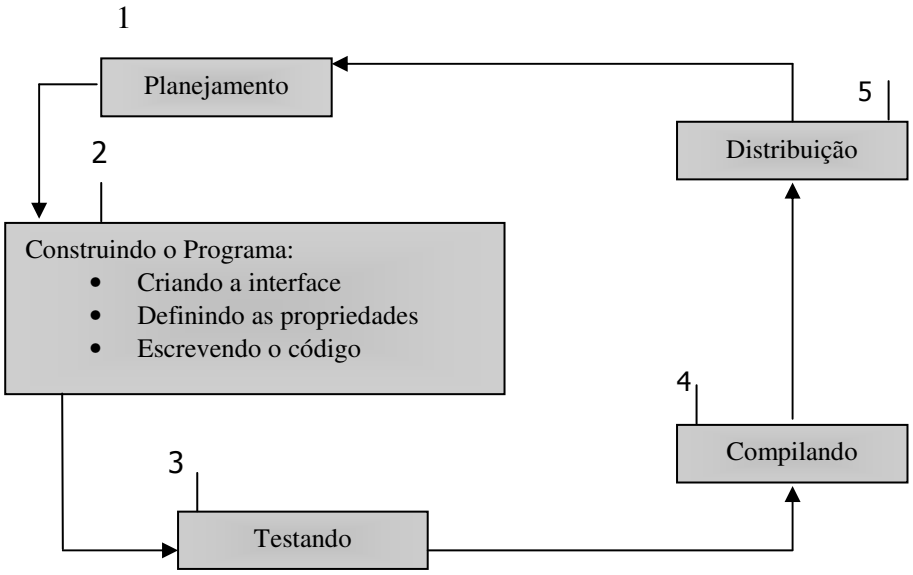


Figura 1.5 – Diagrama de passos que completam o ciclo de desenvolvimento de um software

1.6 Testando o Programa

Testar um programa envolve verificar uma variedade de condições operacionais reais para determinar se ele está funcionando corretamente. O problema que interrompe a execução do programa, ou que não produz o resultado esperado, é chamado um defeito do software, ou bug. Se você utiliza softwares há um bom tempo, provavelmente já se deparou com algumas falhas provocadas por um software defeituoso. Agora que *você* está se tornando um programador, e o seu trabalho é eliminar esses problemas *antes* que eles cheguem até o usuário final, felizmente para todos nós. O Visual Basic fornece algumas ferramentas excelentes de depuração para pegar esses bugs. Você aprenderá a localizar e corrigi-los.

Um defeito no software é chamado bug (que significa inseto *em inglês)*

1.7 Compilando o Programa

Quando terminar de criar e testar o seu programa, você pode compilá-lo em um arquivo executável (.exe) que rodará em ambiente Windows 9x ou Windows NT. Criar um arquivo executável em Visual Basic é tão simples quanto dar um clique no comando Make EXE File no menu File. Se você planeja executar o seu novo aplicativo do Visual Basic somente no seu próprio sistema, criar o arquivo executável será o seu passo final para o programa. Simplesmente de um clique duplo no arquivo .exe do programa no Windows Explorer quando você quiser executar um novo programa, ou crie um ícone de atalho para o arquivo na área de trabalho do Windows.

Os arquivos executáveis possuem a extensão de nome de arquivo .exe

1.8 Distribuindo o Programa

Se quiser compartilhar o seu novo programa com amigos ou colegas, ou se quiser vender o seu programa, você precisará colocar o programa e alguns arquivos de suporte necessários em um disco (ou na rede), onde os seus usuários ou clientes possam ter acesso a eles. Todos os programas em Visual Basic requerem um ou mais arquivos de biblioteca de ligações dinâmicas (DLL), de maneira que possam ser executados. Portanto, você precisará copiar os arquivos necessários do seu disco rígido para um disco de distribuição, de forma que os seus usuários tenham acesso a eles. Os arquivos .DLL necessários dependem do sistema operacional que os seus usuários utilizam e das características do Visual Basic usadas pelo seu programa. O Capítulo 10 desta apostila documenta os arquivos de que você precisará para distribuir os discos e discute como você pode escrever um programa de configuração personalizado para instalar o seu

software, caso você queira. Por hora, tenha a certeza de que, quando programar em Visual Basic, você e os seus usuários estarão imediatamente aptos a desfrutar dos benefícios do novo software.

Os programas em Visual Basic requerem arquivos de biblioteca de ligações dinâmicas (DLL) para serem executados

2.0 FUNDAMENTOS SOBRE PROGRAMAÇÃO

A solução de um problema por meio de programação consiste de 2 passos:

1º Montagem de uma sequência de operações que, quando executadas, produzem um resultado, que é a solução do problema (algoritmo);

2º Transformação da sequência acima descrita em uma linguagem executável por um computador (linguagem de programação).

Então, programação é a criação de um algoritmo (pseudolinguagem) para ser executado por um computador através de uma linguagem de programação.

2.1 Sistemas de Linguagem

Uma vez que muitas linguagens não podem ser executadas diretamente pelo microcomputador, elas são submetidas a processos adicionais. A natureza desses processos depende do tipo de sistema de linguagem usado, ou seja, do modo pela qual a linguagem é implementada.

- **Assemblers**

Um *programa-fonte* em linguagem Assembly é diretamente convertido para linguagem de máquina através de um programa chamado *assembler* (montador). O resultado, chamado *programa-objeto*, é uma série de valores binários, que será executada diretamente pelo computador. É um programa muito compacto e eficiente.

- **Compiladores**

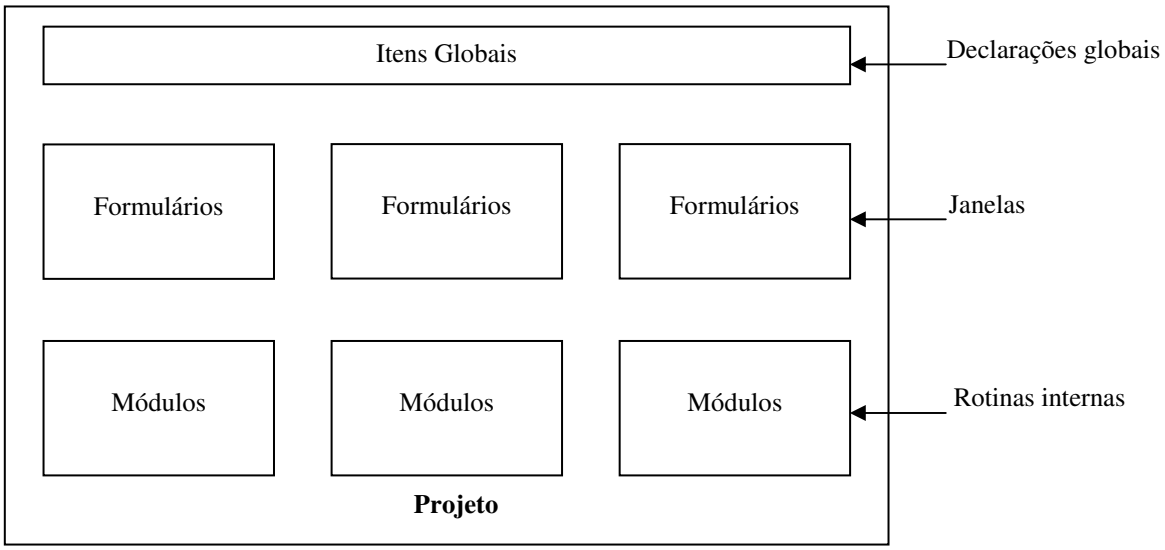
Um compilador é um programa para traduzir um programa-fonte de uma linguagem de alto nível (BASIC, Pascal, FORTRAN, C) para um programa-objeto executável. É um programa mais volumoso e menos eficiente do que aquele produzido por um assembler, porém, o programa-fonte poderá ser desenvolvido muito mais rapidamente e sua manutenção será mais fácil.

- **Interpretadores**

São programas que processam programas-fontes na memória do computador, ou seja, o objetivo de um interpretador não será produzir programas-objeto permanentes em discos. Simplificando, um interpretador lê um programa-fonte, linha por linha, e as executa na ordem em que ocorrem. E para tal, o interpretador converte o programa-fonte em programa-objeto na memória. Utilizando um interpretador, o usuário pode, facilmente, fazer modificações, correções e interrupções no seu programa-fonte e, então, recomeçar a sua execução sem ter que passar pela fase de compilação. Das versões 5 em diante, o Visual Basic foi dotado de um compilador e continua com o antigo interpretador, isso facilita muito o trabalho do programador.

2.2 Estrutura de Programação no Visual Basic

O Visual Basic é estruturado da forma que é mostrado a seguir:



O Visual Basic organiza suas tarefas por *projetos*. Uma aplicação pode ter uma série de formulários associada a ela, isto é, múltiplas janelas, e reunir tudo em somente um único projeto facilmente gerenciado. O Visual Basic permite que você trabalhe em vários projetos abertos ao mesmo tempo.

Formulário é uma janela onde projetamos nosso programa no Visual Basic. As aplicações normalmente possuem pelo menos um formulário (embora tecnicamente isso não seja necessário).

Um módulo do Visual Basic é composto de um código, mas esse é o código que não é associado diretamente a um formulário em particular. As rotinas em um módulos podem ser alcançadas de qualquer lugar na aplicação, isso para que não haja a necessidade de ter todo código ligado a algum formulário. Exatamente por isso o Visual Basic introduziu a idéia de módulo, que foi projetado apenas para conter código de instrução.

Outro ponto importante no Visual Basic é a idéia de *propriedades*, que é a característica de cada objeto. As propriedades já trazem consigo rotinas que podem ser mudadas antes ou durante o tempo de execução do programa. Nessa linha, o Visual Basic possui também os denominados *controles*, que são as lista de texto (listbox), os labels, etc. Formulários e controles são denominados de objetos. O Visual Basic conta também com os denominados *Grupos*, que é um local onde são armazenados os projetos.

2.3 A Caixa de Ferramenta

Como isso é Visual Basic, vamos desenhar os controles que queremos diretamente no formulário que estamos projetando. E essa tarefa é muito fácil necessitamos apenas dar um clique sobre o controle desejado (para que ele se ative como o apontador, como mostra a Figura 2.3), e pronto é só colocá-lo no formulário.

Características de alguns Controles

Para	Faça	Botão
Criar uma caixa de texto	Dê um clique no controle de texto e desenhe a caixa	Quadro de texto
Criar um botão de comando	Dê um clique no controle CommanButton e desenhe o botão	Botão de comando
Criar uma caixa de listagem de drives	Dê um clique no controle DriveListBox e desenhe a caixa	Quadro de listagem de Unidade

Os procedimentos descritos acima refere-se a todos os controles contidos na caixa de ferramenta do Visual Basic.

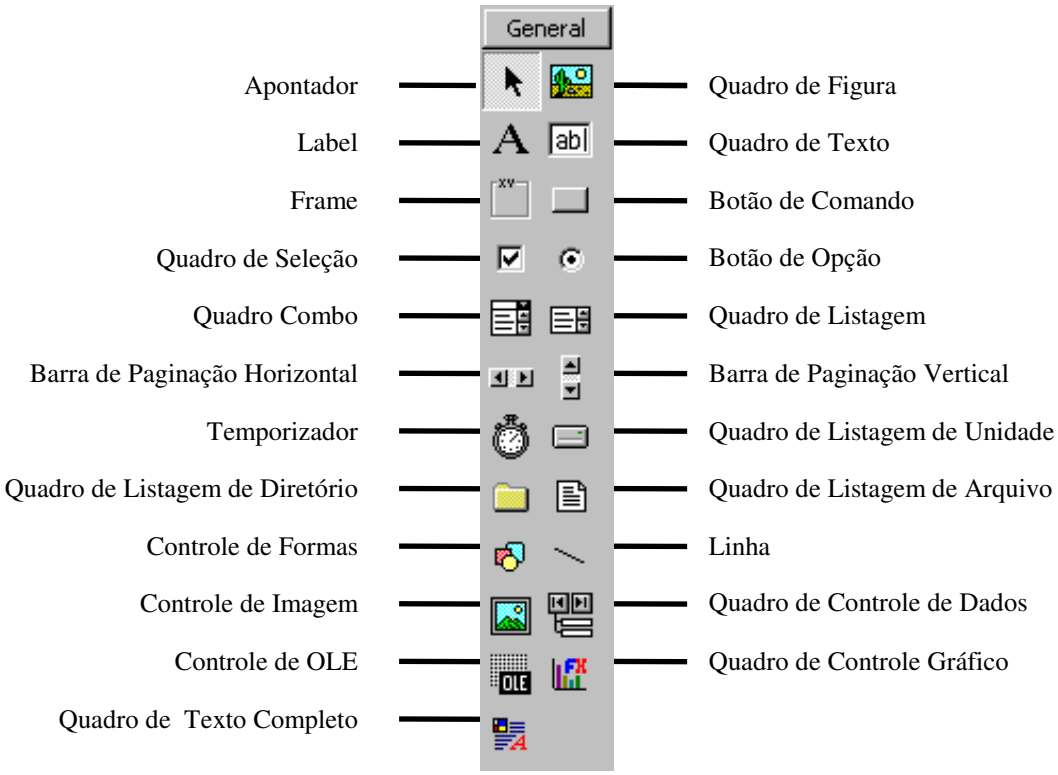


Figura 2.3 – A caixa de ferramenta do Visual Basic

3.0 TRABALHANDO COM CONTROLES, MENU E OBJETO DE DIÁLOGO COMUNS

Controle é uma ferramenta que você utiliza para criar objetos em um formulário em Visual Basic. Você seleciona os controles da caixa de ferramenta e utiliza-os para desenhar objetos no formulário, usando o mouse. Você utiliza a maioria dos controles para criar os elementos de interface do usuário, tais como os botões de comando, as caixas de imagem e a caixas de listagem. Pode-se também, inserir diretamente um controle no formulário dando um duplo clique na controle desejado, e o mesmo aparecerá no centro do formulário.

3.1 A Utilização Básica dos Controles: O Programa "Alô Engenheiro Agrícola!"

A maior tradição em livros de introdução a programação é o programa "Alô Mundo!". Aqui usaremos outra saudação, "Alô Engenheiro Agrícola!", que é o nome dado a um pequeno programa que demonstra como o utilitário mais simples pode ser construído e executado em uma determinada linguagem de programação. Nos tempos da programação baseada em caracteres, o Alô Engenheiro Agrícola! sempre foi um programa de duas ou três linhas digitadas em um editor de programa e montado por um compilador dedicado. Com o advento das ferramentas de programação gráficas, contudo, o programa Alô Engenheiro Agrícola! típico tornou-se um programa complexo contendo dúzias de linhas e exigindo diversas ferramentas de programação na sua construção. Felizmente, criar o programa Alô Engenheiro Agrícola! ainda é bem simples com o Visual Basic. Você pode construir uma interface do usuário completa, criando dois objetos, configurando duas propriedades e inserindo uma linha de código. Tente você mesmo, ao final do programa pronto e executado você terá a seguinte tela.

Figura 3.1 – Janela do programa "Alô Engenheiro Agrícola!"

A linha de comando fica assim:

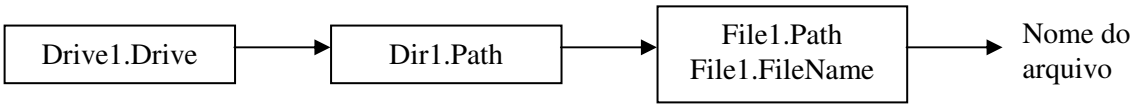
```
Text1.Text = "Alô Engenheiro Agrícola!!"
```

Isto, dentro do botão de comando no evento Command1_Click().

3.2 Utilizando os Objetos do Sistema de Arquivos

O Visual Basic fornece três objetos muito úteis para conseguir acesso ao sistema de arquivos. Eles são: as *caixas de listagens de unidade,* que deixam você pesquisar as unidades (drives) válidos em um sistema, as *caixas de listagem de pasta,* que deixam você navegar pelas pastas de uma unidade específica; e as *caixas de listagem de arquivos,* que deixam você selecionar um arquivo específico em uma pasta. Nos exercícios seguintes, você utilizará os três objetos do sistema de arquivos para construir um programa chamado Procurar que localiza e exibe arquivos gráficos no seu sistema.

Para realmente lermos o nome do arquivo que o usuário deseja carregar, teremos que verificar a propriedade FileName do controle File1, ou seja, File1.FileName. O sistema de leitura é disposto da seguinte maneira:



Você também pode utilizar um objeto do tipo imagem neste programa. Um objeto do tipo imagem pode exibir variados tipos de formatos gráficos: como bitmaps (arquivos .bmp), metafiles do Windows (arquivos .wmf; que são arquivos gráficos que você pode dimensionar) e ícones (arquivos .ico,), etc.

3.3 O Programa Procurar

O programa Procurar é um aplicativo para procurar e carregar um arquivo figura, apresentado essa figura ao usuário. Esse aplicativo facilita quando se deseja saber que figura esta vinculado a um arquivo desconhecido ou esquecido. O programa Procurar utiliza os três objetos do sistema de arquivos, um objeto do tipo imagem e várias linhas de código de programa para criar um programa de pesquisa de arquivo gráfico. Quando você tiver terminado de criar o Procurar, poderá utilizá-lo no seu trabalho diário para analisar o arquivo gráfico de qualquer disco flexível, disco rígido, unidade de rede ou CD-ROM. Os objetos do sistema de arquivos suportam todos os tipos de unidades.

O programa depois de pronto e executado fica da forma que se encontra na Figura 3.3, note que ele possuí três label, uma caixa de listagem de unidade, de diretório e de arquivo, e um controle Image, com sua propriedade "Borderstyle" = 1 – Fixed Single. As linhas de comando ficam da forma seguinte:

```
Private Sub Dir1_Change()
  File1.Path = Dir1.Path
End Sub
```

```
Private Sub File1_Click()
  FileName = File1.Path + "\" + File1.filename
  Image1.Picture = LoadPicture(FileName)
End Sub

Private Sub Drive1_Change()
  Dir1.Path = Drive1.Drive
End Sub
```

Apesar de ser muito fácil carregar um arquivo por esse método, isso pode não ser suficiente, se estivermos no diretório principal de uma unidade, como em d:\, então Dir1.Path seria "d:\", se o arquivo ativo fosse Novel.bmp, então FileName – igual a Dir1.Path + "\" + FileName - seria "d:\\Novel.bmp". Em outras palavras, teríamos uma contrabarra a mais. Para evitar isso, devemos primeiro verificar o último caráter de Dir1.Path. Se já for uma contrabarra, não podemos colocar outra. Dessa forma devemos incluir uma rotina de decisão (veremos mais estruturas de decisão mais a frente), como segue:

```
If (Right$(Dir1.Path,1) = "\" ) Then
        FileName = Dir1.Path + File1.FileName
Else
        FileName = Dir1.Path + "\" + File1.FileName
End If
```

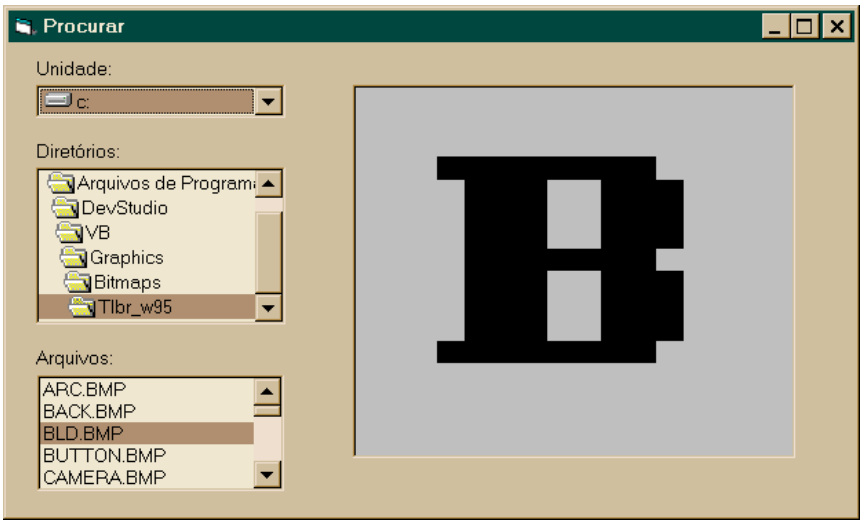


Figura 3.3 – Janela do programa Procurar, utilizando controles de sistema de arquivo

As propriedades Drive, Path e FileName estão disponíveis apenas durante a execução do programa. (Elas contêm os valores que são atribuídos quando as caixas de listagem do sistema de arquivos são utilizadas). Você não pode configurá-las utilizando a janela Properties

3.4 Utilizando um Objeto OLE para Iniciar Aplicativos

Uma das características mais excitantes do Visual Basic é a capacidade de trabalhar intimamente com outros aplicativos para Windows. Utilizando um objeto OLE, você pode iniciar aplicativos do seu programa, enquanto estiver sendo executado, e processar vários tipos de informação. Você também pode utilizar um objeto OLE para executar componentes individuais de outros aplicativos (tal como o verificador ortográfico do Microsoft Word), utilizando uma tecnologia especial *chamada Automação OLE.* No próximo exercício, você conhecerá o funcionamento de um objeto OLE e como ele pode ser utilizado (sem código de programa) para criar um aplicativo chamado Estimador de Custos que inicia os documentos do Microsoft Word, Microsoft Excel e Paint, de maneira que o usuário possa inserir as informações de cálculo, custeio e desenhos do local para um projeto de construção.

Controle OLE, para ativá-lo você deve selecioná-lo e desenhá-lo no formulário desejado

Um objeto OLE permite que você inicie os aplicativos para Windows de dentro de um aplicativo do Visual Basic

Esse programa mostra como pode ser feito um aplicativo para estimar os custos de uma pequena empresa ou de uma casa. Para executar o programa Estimador de Custos, você precisará de uma cópia dos programas Word, Excel e Paint no seu disco rígido. (O Paint acompanha o Windows 9x). Quando você criar pela primeira vez um objeto OLE, será exibida uma caixa de dialogo Inserir Objeto (Veja a Figura 3.4.1) que lista os objetos aplicativos disponíveis que podem ser utilizados no seu programa. Se você não tem os programas Word, Excel ou Paint no seu disco rígido, a caixa de dialogo Inserir Objeto não ira lista-los, mas sinta-se a vontade para escolher outro objeto no lugar. O propósito do exercício é praticar utilizando aplicativos que você gostaria de iniciar a partir do programa Visual Basic. Quando você executar um utilitário, ele ira se parecer com o da Figura 3.4.2.

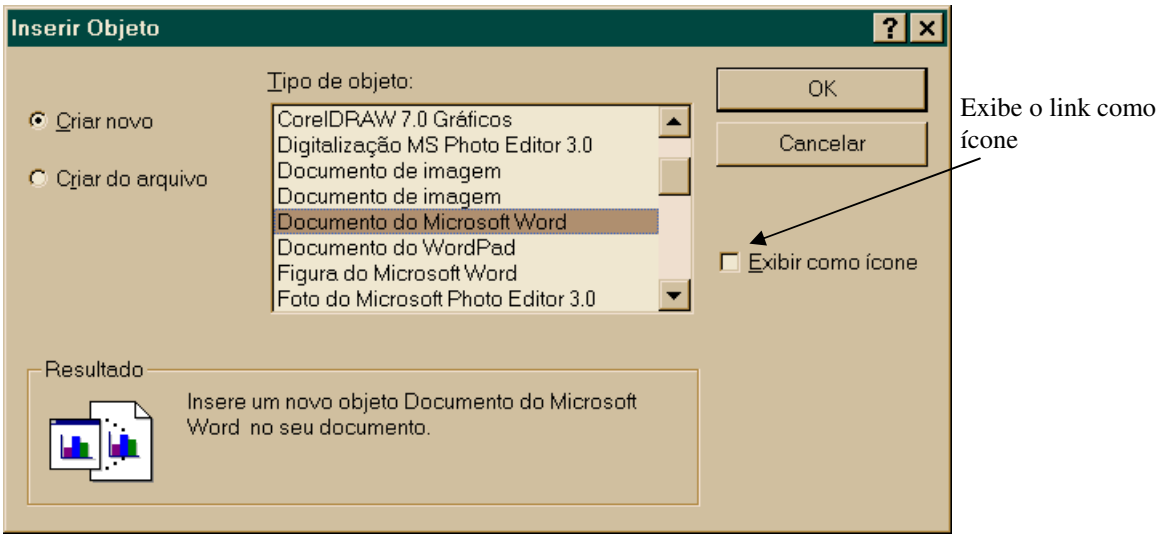


Figura 3.4.1 – Janela para inserção de um objeto OLE

Figura 3.4.2 – Janela do programa Estimador de Custos, com objeto OLE do Word, Excel e Paint

Para utilizar os programas do Word, Excel e Paint é só dar um duplo clique no objeto desejado, e o objeto OLE abrirá o programa para você.

3.5 Instalando Controles Personalizados

Você pode estender a funcionalidade do Visual Basic instalando os controles personalizados que acompanham o Visual Basic, ou os controles personalizados de outros fabricantes. Os controles

personalizados são instalados na caixa de ferramentas do projeto, utilizando o comando Custom Controls no menu Tools. Os novos controles personalizados, que aproveitam a tecnologia OLE de 32 bits, possuem a extensão de nome de arquivo .ocx e podem ser utilizados para acrescentar novas características aos programas do Windows 9x e do Windows NT. Os controles personalizados antigos, que foram projetados para as versões de 16 bits do Visual Basic e do Windows 3.1, possuem a extensão de nome de arquivo .vbx. Você poderá sempre utilizar os novos controles .ocx, se tiver esta opção. Em muitos casos, o Visual Basic substituíra automaticamente os controles antigos pelos novos quando forem instalados os novos controles.

Para instalar um Controle personalizado você deve clicar com o botão direito do mouse na caixa de ferramenta, esse procedimento fará aparecer uma janela (Figura 3.5.1) contendo a opção "Components", de um clique nessa opção e aparecerá a janela para você escolher um Controle ou um Objeto (Figura 3.5.2).

Figura 3.5.1 – Janela de opções secundárias da caixa de ferramenta

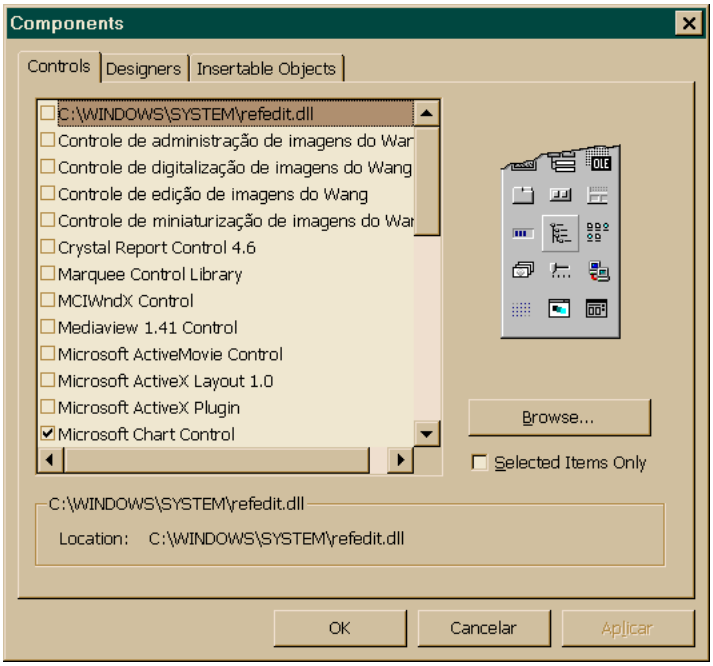


Figura 3.5.2 – Janela de inserção de Components do Visual Basic

3.6 Trabalhando com Menu e Objeto de Diálogo Comum

O editor de menu é uma ferramenta gráfica que gerencia os menus nos seus programas. Ele permite que você acrescente novos menus, modifique e reordene os menus existentes e exclua os menus não mais desejados. Também permite que você acrescente efeitos especiais aos seus menus, tais como teclas de acesso, marcas de verificação e atalhos de teclado.

Por convenção, os títulos de menu e comandos de menu em um aplicativo para o Microsoft Windows possuem a letra inicial maiúscula, vale lembrar que caso haja um menu "Ajuda" no seu formulário ele deve ser o último.

Os menus e comandos devem ser fáceis de serem utilizados e, na medida do possível, devem ter muito em comum com os menus de outros aplicativos. Ao criar um menu, utilize as seguintes diretrizes:

- Utilize legendas curtas e específicas, com uma ou duas palavras no máximo;
- Atribua cada item de menu no programa a uma única tecla de acesso. Utilize a primeira letra do item, se possível;

- Se um comando for utilizado como uma alternativa ativado/desativado, coloque uma marca de verificação no item quando ele estiver ativo;
- Coloque reticências (...) após o comando de menu que solicite ao usuário para inserir mais informações depois que o comando for executado;
- Utilize as convenções para nomear menus, tais como o prefixo *mnu* e o sufixo *Item*, quando atribuir os nomes de menu

Abra o Menu Editor, utilizando o *menu tools* e clique em Menu Editor. Você pode também teclar - Ctrl + E - ou utilizar a barra de ferramenta do VB. Vamos agora criar nosso primeiro menu, ative o editor de menus e aparecerá a janela seguinte:

Figura 3.6.1 – Janela de projeto de menu do Visual Basic

As rotinas criadas ficaram dispostas da seguinte forma:

```
Private Sub mnuDataItem_Click()
        Label1.Caption = Date
End Sub

Private Sub mnuHoraItem_Click()
        Label1.Caption = Time
End Sub

Private Sub mnuEncerrar_Click()
        End
End Sub
```

Onde os nomes das rotinas estão em formato itálico. As rotinas acima, juntamente com dois labels, chega-se ao programa para exibição da hora ou a data. A Figura 3.6.2 mostra como fica o programa depois de executado. Tente você mesmo construir o programa.

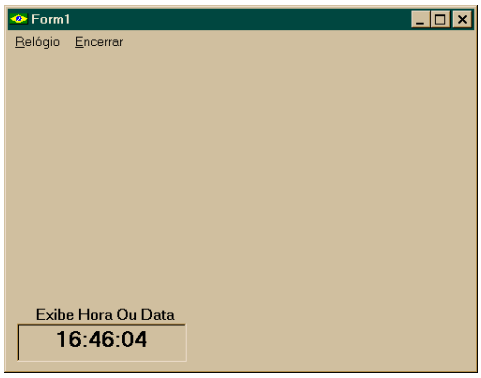


Figura 3.6.2 – Janela de exibição de horas e data com menu

Um objeto de diálogo comum permite exibir cinco caixa de diálogo padrões nos seus programas. Cada caixa de diálogo comum pode ser exibida a partir de um único objeto de diálogo. Você exibe uma caixa de diálogo comum utilizando o seu método de objeto de diálogo comum correspondente, e pode controlar os conteúdos de uma caixa de diálogo comum configurando suas propriedades associadas.

As seis caixa de diálogo comum disponíveis pelo Windows são as seguintes:

Caixa de diálogo	Finalidade	Método
Abrir	Utiliza o unidade, a pasta e o nome de arquivo do arquivo existente	ShowOpen
Salvar Como	Utiliza o unidade, a pasta e o nome de arquivo do novo arquivo	ShowSave
Imprimir	Permite que o usuário defina as opções de impressão	ShowPrinter
Fonte	Permite que o usuário escolha uma fonte e um estilo novo	ShowFont
Cor	Permite que o usuário escolha uma cor da paleta	ShowColor
Ajuda	Permite ativar arquivos de ajuda	ShowHelp

Utilizando o formulário mostrado na Figura 3.6.2, vamos inserir um novo menu com o nome de "Arquivo" contendo as opções de "Abrir", "Salvar" e "Imprimir". A Figura 3.6.3 e 3.6.4 mostram como ficam as janelas após adição do menu. Após finalizada essa passagem, deve-se inserir no formulário um Controle de Diálogo Comum (CommonDialog) para que os objetos que desejamos exibir sejam carregados. Se o controle CommandDialog não estiver disponível na caixa de ferramenta, insirá-o como mostrado na seção 3.5.

As rotinas de cada opção do menu ficam da seguinte forma:

```
Private Sub mnuImprimir_Click()
Dim BeginPage, EndPage, NumCopies, i
        ' configura cancelar para verdade
        CommonDialog1.CancelError = True
        On Error GoTo ErrHandler
        ' Mostra o quadro de dialogo na tela
        CommonDialog1.ShowPrinter
        ' O usuario seleciona as opcoes desejadas
        BeginPage = CommonDialog1.FromPage
        EndPage = CommonDialog1.ToPage
        NumCopies = CommonDialog1.Copies
        Printer.FontName = "courier new"
        Printer.FontSize = 10
        'Você decide o como imprimir, se retrato ou paisagem
        ' Printer.Orientation = vbPRORPortrait
         Printer.Orientation = vbPRORLandscape
For i = 1 To NumCopies
         Printer.Print text1.txtArquivo
        '  Imprimir resultados
        Printer.EndDoc
```

```
Next i
Exit Sub
ErrHandler:
   ' o usuario pressionou cencelar
   Exit Sub
End Sub

Private Sub mnuAbrir_Click()
Dim Abrir As String
        CommonDialog1.CancelError = True
        On Error GoTo ErrHandler
        CommonDialog1.Filter = "Arquivo Figura (*.bmp)|*.bmp"
        CommonDialog1.ShowOpen
        Abrir = CommonDialog1.filename
        Rem Digite aqui a linha de comando
        ErrHandler:
        ' o usuario pressionou cencelar
Exit Sub
End Sub

Private Sub mnuSalvar_Click()
        CommonDialog1.CancelError = True
        On Error GoTo ErrHandler
        CommonDialog1.Filter = "Todos Arquivos (*.*)|*.*"
        CommonDialog1.ShowSave
        Abrir = CommonDialog1.filename
        Rem Digite aqui a linha de comando
ErrHandler:
        ' O usuario pressionou cencelar
Exit Sub
End Sub
```

Para atribuir uma tecla de atalho ao menu, selecione a opção do menu que você deseja atribuir uma tecla de atalho, em seguida clique na caixa de listagem drop-down Shortcut, role para baixo na caixa de listagem e escolha a tecla de atalho desejada. Não se consegue atribuir uma tecla de atalho a um menu, para isso utilize teclas de acesso. Você pode também utilizar teclas de acesso nas opções dos menus.

Figura 3.6.3 – Janela de projeto de menu do Visual Basic com o novo menu e teclas de atalho

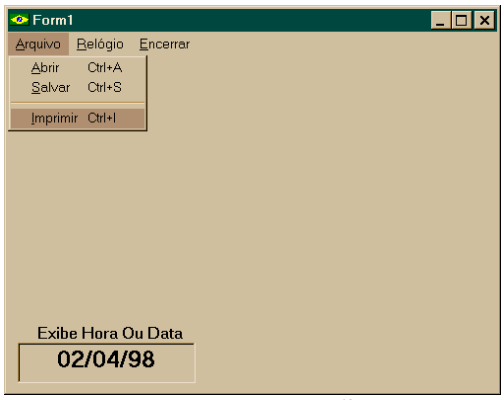


Figura 3.6.4 – Janela com o novo menu, e a opções com suas respectivas teclas de atalho e de acesso

Até agora, nossa discussão sobre os menus se deteve na barra de menus que aparece ao longo da parte superior do formulário. O VB também admite *menus instantâneos* nos seus programas. Um menu instantâneo é um pequeno menu que surge em algum lugar do seu formulário, em resposta a um evento do programa.

Quando um menu instantâneo é invocado, o menu aparece na tela na posição atual do mouse. Então, o usuário efetua uma seleção a partir do menu. Depois que a seleção é processada, o menu desaparece da tela.

Crie um menu instantâneo da mesma forma que você criou o menu principal para seu programa. Entretanto, nesse caso há um passo extra. O menu instantâneo tem de estar oculto, assim, ele não aparecerá na barra de menus. Para conseguir isso, defina a propriedade *Visible* do item de menu do nível superior como *False*.

Em geral, você ocultará o item de menu que será usado como menu instantâneo, mas é possível utilizar qualquer dos itens do nível superior de uma barra de menus como menu instantâneo

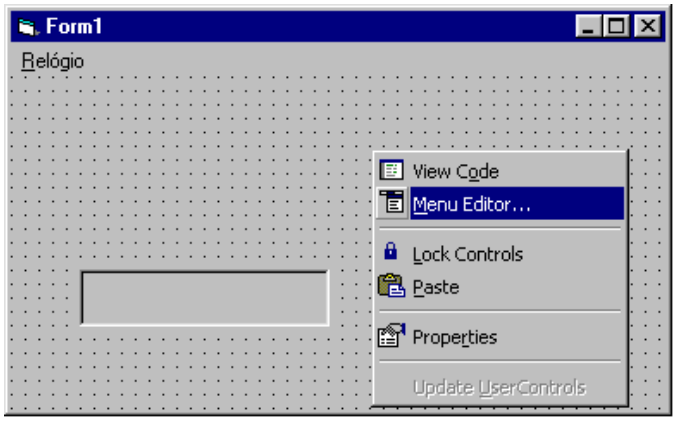


Figura 3.6.5 – Demonstração do menu instantâneo do VB

Para fazer o menu instantâneo aparecer na sua tela, você tem de invocá-lo usando o método *PopUpMenu*. Isso é feito pela especificação do nome do formulário no qual o menu será exibido, do método *PopUpMenu* e do nome do menu a ser apresentado.

Embora você possa usar esse método a partir de qualquer ponto no seu código, os menus instantâneos são usados com menor freqüência em resposta a um clique do mouse, geralmente com o botão direito. O código a seguir mostra de que maneira o menu instantâneo pode ser chamado.

```
Private Sub Form_MouseDown(Button As Integer, Shift As Integer, X As Single, Y As Single)
If Button = 2 Then
    frmMain.PopupMenu NomeMenu
End If
End Sub
```

Nesse segmento de código, o evento *MouseDown* é usado paea executar uma ação sempre que um botão do mouse for pressionado. O parâmetro *Button* é que informa qual botão do mouse foi pressionado, *frmMain* é o nome do formulário e *NomeMenu* o nome dado ao menu instantâneo.

Você pode criar vários menus instantâneos e fazer com que eles sejam exibidos em respostas a diferentes eventos de botões do mouse ou a áreas distintas da tela

3.7 Acrescentando Caixa de Diálogo Não Personalizada aos Programas

E se você precisar acrescentar uma caixa de diálogo ao seu programa que não seja um dos seis tipos de caixa de diálogo comum? Nenhum problema – mas você precisará fazer um trabalho de design extra. O programa Visual Basic pode utilizar mais de um formulário para receber e exibir informações. Para criar caixas de diálogo não padronizada, você precisará acrescentar novos formulários ao seu programa, acrescentar objetos de entrada e saída, e processar os cliques de caixa de diálogo no seu código de programa. Você aprenderá mais a frente a utilizar duas caixas especialmente projetadas para receber entrada de texto e exibir texto de saída.

4.0 VARIÁVEIS E OPERADORES DO VISUAL BASIC

4.1 A Anatomia de uma Instrução de Programa em Visual Basic

Como você aprendeu anteriormente, uma linha de código em um programa em Visual Basic é chamada instrução de programa. Uma *instrução de programa* é qualquer combinação de palavras-chave, propriedades, funções, operadores e símbolos do Visual Basic que cria coletivamente uma instrução válida reconhecida pelo compilador do Visual Basic. Uma instrução de programa completa pode ser uma palavra chave simples como:

```
Beep
```

que reproduz um som pelo alto-falante do seu computador, ou pode ser uma combinação de elementos, como a instrução a seguir, que atribui a hora atual do sistema para a propriedade Caption de um rótulo (Label):

```
Label1.Caption = Time
```

Nome do objeto	Nome da propriedade	Função do Visual Basic

Uma instrução de programa é uma instrução válida para o compilador do Visual Basic

As regras da construção que devem ser utilizadas quando você construir uma instrução de programação são chamadas de *sintaxe* da instrução. O Visual Basic compartilha muitas das suas regras de sintaxe com as antigas versões da linguagem de programação Basic e com outros compiladores de linguagem. O truque para escrever boas instruções de programa e aprender a sintaxe dos elementos mais úteis da linguagem e, então, utilizar esses elementos corretamente para processar os dados no seu programa. Felizmente, o Visual Basic fez a maior parte do trabalho duro, assim, o tempo que você gastara escrevendo o código do programa será relativamente curto e os resultados podem ser utilizados novamente em programas futuros.

Nas próximas lições, serão apresentadas as palavras-chave e as instruções de programa mais importantes do Visual Basic. Você descobrirá que elas complementarão muito bem as habilidades de programação que você já aprendeu e irão ajudá-lo a escrever programas poderosos no futuro. Os

primeiros tópicos, tipos de variáveis e de dados, são características importantes de quase todos os programas.

4.2 Utilizando Variáveis para Armazenar Informações

Uma *variável* é um local temporário de armazenagem de dados no seu programa. Você pode utilizar uma ou várias variáveis no seu código e elas podem conter palavras, números, datas ou propriedades. As variáveis são úteis porque permitem que você atribua um nome curto e fácil de lembrar a parte dos dados com a qual você planeja trabalhar. As variáveis podem guardar informações inseridas pelo usuário durante o tempo de execução, o resultado de um cálculo específico ou uma parte dos dados que você quer exibir no seu formulário. Em resumo, as variáveis são ferramentas simples que podem ser utilizadas para manter quase todos os tipos de informação.

Utilizar variáveis em um programa em Visual Basic é quase como conseguir uma mesa em um restaurante da moda. Você pode começar usando uma variável por vez no seu código de programa, mas o "gerente" ficaria mais contente se a reserva fosse feita com antecedência. Cobrirei o processo de fazer reservas, ou de *declarar,* uma variável nas próximas duas seções.

4.3 Fazendo as Reservas para as Variáveis: A Instrução Dim

Para decifrar explicitamente uma variável, antes de você utilizá-la (geralmente no começo de uma procedure de evento), digite o nome da variável após a instrução Dim. Essa ação reserva espaço na memória para a variável quando o programa for executado, e permite que o Visual Basic saiba que tipo de dado deverá ser esperado depois. Por exemplo, a instrução seguinte cria espaço para uma variável chamada LastName no programa:

```
Dim LastName
```

Por padrão, o Visual Basic reserva espaço para um tipo de variável.

Dim reserva espaço para uma variável. Você armazena dados em uma variável utilizando o operador de atribuição (=)

4.4 Declarando Variáveis sem Dim

Você também pode declarar uma variável sem a instrução Dim. Esse processo é chamado *declaração implícita.* Para declarar uma variável dessa maneira, você simplesmente utiliza a própria variável, desconsiderando completamente a instrução Dim:

```
LastName = "Charles"
```

A declaração implícita possui uma vantagem, pois você não gasta tempo digitando a instrução Dim. Porém, o "gerente" sempre desencoraja essa atitude, pois a declaração implícita não força você a organizar e listar suas variáveis com antecedência, e impede que o Visual Basic exiba uma mensagem de erro, se você errar na digitação do nome da variável mais tarde. (veja nota a seguir). Nesta apostila, farei a declaração das variáveis utilizando ambas as técnicas.

Se você decidir sempre declarar as suas variáveis utilizando a instrução Dim, poderá colocar a instrução Option Explicit no inicio da procedure de evento Form_Load do seu programa

Por exemplo:

```
Private Sub Form_Load ( )
Option Explicit
.
.
.
End Sub
```

Quando a instrução Option Explicit é utilizada, o Visual Basic gera uma mensagem de erro toda vez que encontrar uma variável que não tenha sido explicitamente declarada no código (o mesmo acontece quando ocorrer um erro de ortografia no nome da variável). Se você esta preocupado com os erros de ortografia, esta instrução pode ajudá-lo a rastreá-los

4.5 Convenções para Nomear Variáveis

Nomear variáveis pode ser um pouco difícil, pois você precisa utilizar nomes curtos, porém intuitivos e fáceis de lembrar. Para evitar confusões, utilize as seguintes convenções quando for nomear variáveis:

- Comece cada nome de variável com uma letra. Isto é uma exigência do Visual Basic. Os nomes de variável não deve ultrapassar 256 caracteres e não podem conter pontos finais;
- Escolha nomes descritivos, combinando uma ou mais palavras no nome da variável, se fizer sentido. Por exemplo, o nome da variável TaxaImpostoVenda é muito mais claro que Imposto ou Taxa;
- Utilize uma combinação de caracteres com letras maiúscula e minúscula, e números, se quiser. Uma convenção aceita é escrever com maiúsculas a primeira letra de cada palavra em uma variável. Por exemplo, DataDeNascimento;
- Não utilize as palavras chaves, os objetos ou propriedades do Visual Basic como nome de variáveis.

4.6 Utilizando uma Variável para Armazenar Entradas

Uma utilização excelente para uma variável é guardar as informações de entrada de dados do usuário. Você pode sempre utilizar um objeto (como uma caixa de listagem de arquivo ou uma caixa de texto) para recuperar essa informações, porém, algumas vezes, você desejará lidar diretamente com o usuário e salvar a entrada de dados em uma variável, e não em uma propriedade. Uma maneira de fazer isso é utilizar a função InputBox para exibir uma caixa de diálogo na tela e, então, armazenar o texto que o usuário digitar em uma variável. Você trabalhará com essa técnica no próximo exemplo.

Você pode obter uma entrada de dados de um usuário utilizando efetivamente a função InputBox e uma variável

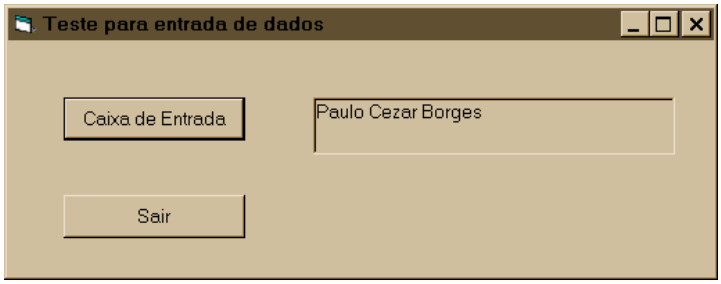


Figura 4.6.1 – Janela do programa Teste Para Entrada de Dados

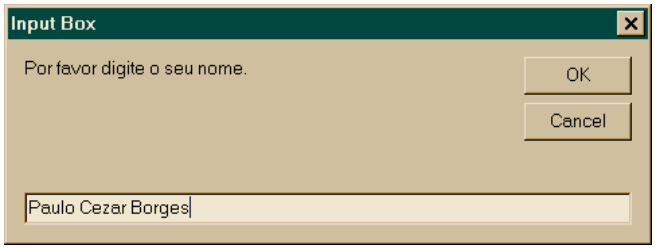
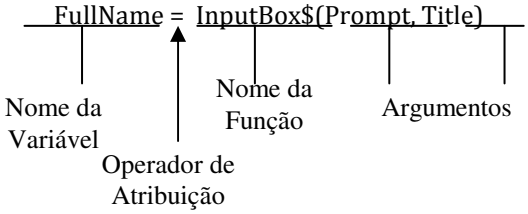


Figura 4.6.2 – Janela padrão InputBox do Windows

```
Private Sub Command1_Click()
        Dim Prompt, FullName
        Prompt = "Por favor digite o seu nome."
        FullName = InputBox$(Prompt)
        Label1.Caption = FullName
End Sub
```

4.7 O Que é uma Função

InputBox é uma palavra-chave especial do Visual Basic conhecida como função. Função é uma instrução que desempenha um trabalho importante (tal como solicitar uma informação do usuário ou calcular uma equação) e, então, retorna um resultado ao programa. O valor retornado pela função pode ser atribuído a uma variável, como no programa Teste Para Entrada de Dados, ou pode ser atribuída a uma propriedade ou a outra instrução ou função. As funções do Visual Basic em geral utilizam um ou mais argumentos para definir suas atividades. Por exemplo, a função InputBox que você acabou de executar utilizou a variável Prompt para exibir as instruções da caixa de diálogo ao usuário. Quando uma função utiliza um ou mais argumentos, os argumentos são separados por vírgulas e todo o grupo de argumentos fica entre parêntese. A instrução a seguir apresenta uma chamada de função que dois argumentos.



4.8 Utilizando uma Variável para Saída de Dados

Você pode exibir o conteúdo de uma variável atribuindo a variável a uma propriedade (tal coma a propriedade Caption do objeto rótulo), ou passando-a como um argumento para a função da caixa de dialogo. Uma função de caixa de dialogo útil para exibir as saídas de dados é a função MsgBox. Tal como a InputBox, ela utiliza um ou mais argumentos como entrada de dados e os resultados da função podem ser atribuídos a uma variável. A sintaxe da função MsgBox é:

BotaoClicado = MsgBox(Mensagem, NumeroDoBotao, Titulo)

em que: Mensagem é o texto a ser exibido na tela, *NumeroDoBotao* é o número do estilo de botão e *Titulo* é o texto exibido na barra de título da caixa de mensagem. A variável *BotaoClicado* é atribuída ao resultado retornado pela função, a qual indica em qual botão foi dado um clique na caixa de diálogo. Se você simplesmente exibir uma mensagem na MsgBox, o operador de atribuição (=), a variável *BotaoClicado* e o argumento *NumeroDoBotao* serão opcionais. Você não irá utilizá-los no exercício seguinte, para mais informação sobre eles (inclusive sobre os diferentes botões que você pode incluir na MsgBox), procure *MsgBox* na Ajuda on-line do Visual Basic.

A função MsgBox utiliza strings de texto para exibir as saídas de dados em uma caixa de dialogo. Ela suporta um número de argumentos opcionais

Agora acrescente uma função MsgBox ao programa Teste Para Entrada de Dados para exibir o nome que o usuário inserirá na caixa de dialogo InputBox.

4.9 Trabalhando com Tipos de Dados Específicos

Na maioria dos casos, o tipo de dado variante é o único tipo de dado de que você precisa. As variáveis do tipo *variant* podem armazenar todos os tipos de dados fundamentais (predefinidos) do Visual Basic e pode alternar os formatos automaticamente. As variantes são também fáceis de usar e você não precisará pensar muito sobre o eventual tamanho da variável quando a declarar. Se você quiser criar um código especialmente rápido e conciso, contudo, poderá utilizar tipos de dados mais específicos quando for apropriado. Por exemplo, se uma variável sempre contiver valores inteiros pequenos (números sem casa decimal), você poderá guardar espaço na memória quando executar o programa, declarando a variável como um número inteiro e não como *variant.* Uma variável de número inteiro também acelera as operações aritméticas, assim, você também ganha uma pequena vantagem em velocidade quando

utilizá-la. O quadro a seguir lista os tipos de dados fundamentais no Visual Basic. No próximo exercício, você verá como funcionam vários desses tipos de dados.

Se uma variável sempre contiver um tipo de dado específico, você pode melhorar o desempenho do seu programa declarando a variável como daquele tipo

Tipo de Dado	Tamanho	Faixa	Exemplo
Integer (Inteiro)	2 bytes	De -32.768 até 32.767	Dim Birds% Birds% = 37
Long integer (Inteiro Longo)	4 bytes	De -2.147.483.648 até 2.147.483.647	Dim Loan& Loan& = 350.000
Single precision floating point (Ponto flutuante de precisão simples)	4 bytes	De -3,042823E38 até 3,402823E38	Dim Price! Price! = 899,99
Double precision floating point (Ponto flutuante de precisão dupla)	8 bytes	De -1,79769313486232D308 até 1,79769313486232D308	Dim Pie# Pie# = 3,1415926535
Currency (Moeda)	8 bytes	De –922337203685477,5808 até 922337203685477,5807	Dim Debt@ Debt@ = 7600300,50
String (String)	1 bytes por caractere	De O até 65.535 caracteres	Dim Dog$ Dog$= "pointer"
Boolean (Booleano)	2 bytes	Verdadeiro ou Falso	Din Flag as Boolean Flag = True
Date (Data)	8 bytes	De 1 de janeiro de 100 até 31 de dezembro de 9999	Dim Birthday as Date Birthday = #3-1-63#
Variante (Variant)	16 bytes(com tipo de dada números); 22 bytes + 1 byte por caractere (com strings)	Todas as faixas de tipo de dados	Dim Total Total = 289,13

O tamanho de armazenagem da variável é medido em bytes - o espaço de armazenagem total exigido para armazenar 8 bits (aproximadamente 1 caractere)

Você pode especificar alguns tipos de dados fundamentais adicionando um caractere de declaração de tipo ao nome da variável. Por exemplo, você pode declarar uma Variável como um número inteiro adicionando o caractere % ao final do nome

Assim, em Visual Basic, as duas instruções de declaração a seguir são equivalentes:

```
Dim I as Integer
Dim I%
```

Para mais informação sobre os tipos de dados fundamentais, os tamanhos de variáveis e as faixas suportadas, procure *Variable* na Ajuda on-line do Visual Basic.

As variáveis também podem ser globais por natureza, ou estar disponíveis em todas as procedures e módulos de formulário de um programa. (Módulos são arquivos especiais que contêm declarações e procedures não associadas a um formulários em particular.) Para que uma variável tenha esse escopo, precisa ser declarada no nível de módulo. Para mais informação sobre como criar uma variável global, procure Public na Ajuda on-line do VB

4.10 Constantes: Variáveis que não se Alteram

Se uma variável no seu programa contiver um valor que nunca será alterado (tal como o símbolo π, uma entidade matemática fixa), você poderá armazenar o valor como uma constante, em vez de uma variável. Uma *constante* é um nome significativo que toma o lugar de um número ou uma string de texto que não são alterados. As constantes são úteis, pois aumentam a legibilidade do código de programa, podem poupar memória e fazem alterações globais mais fáceis de serem cumpridas depois. As constantes operam como as variáveis, mas não podem ter os seus valores modificados durante a execução do programa. Elas são declaradas pela palavra-chave Const, como mostra o exemplo a seguir:

```
Const Pi = 3.14159265
```

A instrução apresentada cria uma constante chamada Pi que pode ser utilizada no lugar do valor de símbolo π no código de programa. Para criar uma constante disponível em todas as procedures de um programa, você devera criar a constante em um módulo padrão colocando a sua frente a palavra-chave Public. Por exemplo:

```
Public Const Pi = 3.14159265
```

4.11 Trabalhando com Operadores do Visual Basic

Uma *fórmula* é uma instrução que combina números, variáveis, operadores e palavras-chave, ou alguns destes elementos, para criar um novo valor. O Visual Basic contém vários elementos de linguagem projetados para o uso em fórmulas. Nesta seção, você praticará trabalhando com operadores matemáticos, os símbolos utilizados para amarrar as partes de uma fórmula. Com algumas execuções, os símbolos matemáticos que você utilizará são aqueles utilizados no seu dia a dia, e as suas operações são claramente intuitivas. Você verá cada um deles nos exercícios a seguir.

O Visual Basic fornece os seguintes operadores:

Operador	Operação Matemática
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
\	Divisão de inteiro (número inteiro)
Mod	Resto de Divisão
^	Exponenciação
&	Concatenação de string (combinação)

É sempre importante testar cada cálculo no programa para verificar se todo o programa está funcionando corretamente. Testar apenas uma parte do programa não é suficiente

4.12 Utilizando Operadores Avançados: \, Mod, ^ e &

Além dos quatro operadores matemáticos básicos, o Visual Basic inclui quatro operadores avançados, que desempenham a divisão de inteiros (\ ), divisão de restos (Mod), exponenciação (^) e concatenação de string (&). Esses operadores são úteis em fórmulas com finalidade especialmente matemática e nos aplicativos de processamento de texto. A Figura 4.12 apresenta um programa que usa os operadores avançados.

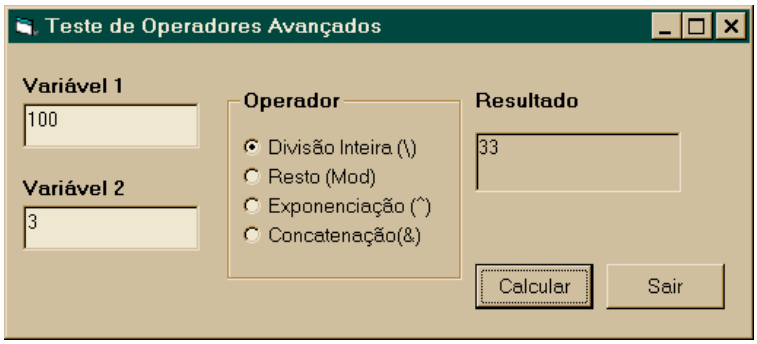


Figura 4.12 – Programa para demonstração dos operadores avançados

A rotina de execução do programa fica como segue:

```
Private Sub Command1_Click()
Dim First, Second   'declara as variáveis
  First = Text1.Text  'busca os números das caixas de texto
  Second = Text2.Text
  'se for dado um clique no primeiro botão, faça a divisão de inteiros
  If Opção1.Value = True Then
    Label1.Caption = First \ Second
  End If
  'se for dado um clique no segundo botão, calcule o resto da divisão
  If Opção2.Value = True Then
    Label1.Caption = First Mod Second
  End If
  'e for dado um clique no terceiro botão, faça a exponenciação
  If Opção3.Value = True Then
    Label1.Caption = First ^ Second
  End If
  'e for dado um clique no quarto botão, faça a concatenação das strings
  If Opção4.Value = True Then
    Label1.Caption = First & Second
  End If
End Sub
```

4.13 Precedência de Operador

Você viu sete operadores matemáticos e um operador de string. O Visual Basic permite que você misture quantos operadores matemáticos quiser em uma fórmula, contanto que cada variável e cada expressão numérica esteja separada uma da outra por um operador. Por exemplo, veja a seguir uma fórmula aceita pelo Visual Basic:

Total = 10 + 15 * 2 / 4 ^ 2

Essa fórmula processa vários valores e atribui o resultado a uma variável chamada Total. Mas como tal expressão é avaliada pelo Visual Basic? Em outras palavras, quais operadores matemáticos o Visual Basic utiliza primeiro quando resolve a fórmula? Você pode não ter percebido, mas a ordem da avaliação tem grande importância neste exemplo.

A ordem do operador de avaliação é sempre importante para se ter em mente quando fórmulas as matemáticas estiverem sendo construídas

O Visual Basic resolve esse dilema estabelecendo uma *ordem de precedência* especifica para as operações matemáticas. Uma lista de regras diz ao Visual Basic qual operador deve ser utilizado primeiro quando ele for avaliar uma expressão que contenha mais de um operador. A tabela a seguir apresenta a ordem, do primeiro ao ultimo operador, em que eles serão avaliados. (Os operadores de mesmo nível nesta tabela são avaliados da esquerda para a direita, como aparecem na expressão).

Operador(es)	Ordem de precedência
( )	Os valores entre parêntese são sempre avaliados primeiro.
^	A exponenciação é avaliada em segundo lugar.
-	A negação (criar um número negativo) é avaliada em terceiro.
* /	A multiplicação e a divisão são avaliados em quarto.
\	A divisão de inteiro é avaliada em quinto.
Mod	A divisão de restos é avaliada em sexto.
+ -	A adição e a subtração são avaliados por último.

4.14 Funções Matemáticas do Visual Basic

De vez em quando você pode querer fazer uma pequena manipulação extra de números nos seus programas. Você precisará converter um valor para um tipo diferente, calcular uma expressão matemática complexa ou introduzir uma função aleatória aos seus programas. As funções do Visual Basic a seguir pode ajudá-lo a trabalhar com números nas suas fórmulas. Como em qualquer função, as funções matemáticas precisam ser utilizadas em uma instrução de programa e retornarão um valor ao programa. No Quadro abaixo, o argumento *n* representa o número, a variável ou a expressão que você quer que a função avalie.

Função	Finalidade
Abs(*n*)	Retorna o valor absoluto de *n.*
Atn(*n*)	Retorna o arco-tangente de *n*, em radianos.
Cos(*n*)	Retorna o Co-seno do ângulo *n*, expresso em radianos.
Exp(*n*)	Retorna a constante *e* elevado a potência *n.*
Rnd(*n*)	Gera um número aleatório entre 0 e 1.
Sgn(*n*)	Retorna –1 se *n* for menor que zero; 0 se *n* for zero; e +1 se *n* for maior que zero.
Sin(*n*)	Retorna o seno do ângulo *n*. O ângulo *n* é expresso em radianos.
Sqr(*n*)	Retorna a raiz quadrada de *n.*
Str(*n*)	Converte um valor numérico em uma string.
Tan(*n*)	Retorna a tangente do ângulo *n.* O ângulo *n* é expresso em radianos.
Val(*n*)	Converte um valor de string em número.

5.0 UTILIZANDO ESTRUTURA DE DECISÃO

Nas lições anteriores, você utilizou várias ferramentas do Microsoft Visual Basic para processar as entradas de dados do usuário. Você utilizou menus, objetos e caixas de diálogo para exibir escolhas para o usuário e processou as entradas de dados utilizando propriedades e variáveis. Nesta lição, você aprenderá a desviar condicionalmente uma seção específica do código de programa baseado na entrada de dados feita pelo usuário, ao avaliar uma ou mais propriedades ou variáveis utilizando expressões condicionais e, então, executar uma ou mais instruções de programa baseado nos resultados.

5.1 Programação Orientada por Evento

Os programas que você escreveu até aqui neste livro exibiram menus, objetos e caixas de diálogo na tela e encorajaram os usuários a manipular os elementos da tela na ordem que eles bem quisessem. Os programas colocaram o usuário no comando, esperaram pacientemente por uma resposta e, então, processaram prognosticamente as entradas de dados. Nos círculos de programação, essa metodologia é conhecida como *programação orientada por evento.* Você constrói um programa criando um grupo de objetos "inteligentes" que sabem como responder quando o usuário interage com eles e, então, você processa as entradas de dados utilizando as procedures de evento associadas a esses objetos.

Os programas em Visual Basic são orientados por eventos

As entradas de dados do programa podem também vir do próprio sistema do computador. Por exemplo, o seu programa pode ser notificado quando uma mensagem no correio eletrônico chegar, ou quando um certo período de tempo terminar no relógio do sistema. Esses eventos são disparados pelo computador e não pelo usuário. Independentemente de como um evento é disparado, o Visual Basic reage chamando a procedure de evento associada ao objeto que reconheceu o evento. Embora, até aqui, você tenha lidado a princípio com eventos Click e Change, os objetos do Visual Basic podem responder a vários tipos de eventos.

A natureza orientada por evento do Visual Basic significa que a maioria da computação feita nos seus programas será realizada pelas procedures de evento.

Esses blocos de código evento-específicos processam entradas de dados, calculam novos valores, exibem as saídas de dados e lidam com outras tarefas. Na lição anterior, você aprendeu a utilizar variáveis, operadores e fórmulas matemáticas para executar cálculos nas suas procedures de evento. Nesta lição, você aprenderá a utilizar *estruturas de decisão* para comparar variáveis, propriedades e valores, e a executar uma ou mais instruções com base nos resultados. Na próxima lição, você aprenderá a utilizar *loops* para executar um grupo de instruções várias e várias vezes até que seja satisfeita a condição. Juntas, essas estruturas de controle de fluxo poderosas irão ajudá-lo a construir as suas procedures, de maneira que possam responder a quase todas as situações.

5.2 Eventos Suportados pelos Objetos do Visual Basic

Cada objeto do Visual Basic possui um conjunto pré-determinado de evento aos quais ele pode responder. Esses eventos são limitados para cada evento na caixa de listagem drop-down Proc (procedure) na janela de código. Você pode escrever uma procedure de evento para qualquer um desses eventos e, se um evento ocorrer no programa, o Visual Basic executará a procedure de evento que estiver associado a ele. Por exemplo, um objeto caixa de listagem suporta os eventos Click, DblClick, Drag-Drop, DragOver, GotFocus, KeyDown, KeyPress, KeyUp, LostFocus, MouseDown, MouseMove e MouseUp. Embora muito provavelmente você não tenha precisado programar para mais de um ou dois desses eventos no seu aplicativo, é bom saber que você tem muitas opções quando cria elementos na sua interface. A Figura 5.2 mostra uma listagem parcial dos eventos para um objeto caixa de listagem na janela de código.

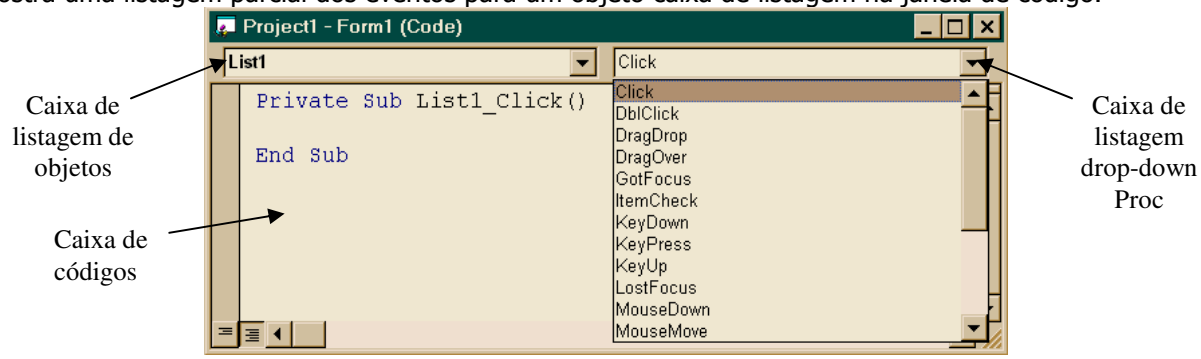


Figura 5.2 - Janela de código e eventos de uma caixa de listagem

5.3 Utilizando Expressões Condicionais

Uma das ferramentas mais úteis para processamento de informação na procedure de evento é a expressão condicional. Uma *expressão condicional* é uma parte de uma instrução de programa completa que faz uma pergunta True-or-False sobre uma propriedade, sobre uma variável ou sobre outra parte de dados no código de programa. Por exemplo, a expressão condicional:

$$Price < 100$$

avalia para True (Verdadeiro) se a variável Price contiver um valor menor que 100, e avalia para False (Falso) se Price contiver um valor maior ou igual a 100. Você pode utilizar os operadores de comparação, que estão listados a seguir, em uma expressão condicional.

As expressões condicionais fazem perguntas do tipo True-or-False (Verdadeiro ou Falso)

Operador de comparação	Significado
=	Igual a
< >	Diferente de
>	Maior que
<	Menor que
>=	Maior ou igual a
<=	Menor ou igual a

Expressões que podem ser avaliadas como True ou False também são conhecidas como expressões Booleanas, e o resultado True ou False pode ser atribuído a uma variável ou propriedade Booleana. Você pode atribuir valores Booleanos a certas propriedades de objeto, variáveis do tipo variant ou variáveis Booleanas que tenham sido criadas com o uso da instrução Dim e das palavras-chave As Bolean

5.4 Estruturas de Decisão *If ... Then*

As expressões condicionais podem controlar a ordem na qual as instruções são executadas quando utilizadas em um bloco especial de instruções conhecido como *estrutura de decisão.* A estrutura de decisão *If...Then* permite que você avalie uma condição no programa e tome uma medida com base no resultado. Na sua forma mais simples, a estrutura de decisão *If...Then* é escrita em uma única linha:

If *condição* Then *instrução*

em que, *condição* é uma expressão condicional e *instrução* é uma instrução válida de programa em Visual Basic. Por exemplo,

```
If Score >= 20 Then Label1.Caption = "Você ganhou!"
```

é uma estrutura de decisão *If ... Then* que utiliza a expressão condicional

```
Score >= 20
```

para determinar se o programa deverá configurar a propriedade Caption do objeto Label1 para "Você ganhou!". Se a variável Score contiver um valor maior ou igual a 20, o Visual Basic configura a propriedade Caption, caso contrário, pula a instrução de atribuição e executa a próxima linha na procedure de evento. Esse tipo de comparação sempre resulta em um valor True ou False. Uma expressão condicional nunca tem como resultado "talvez".

As estruturas de decisão If ... Then permitem que você acrescente lógica aos seus programas

5.5 Testando Várias Condições em um Estrutura de Decisão *If...Then*

O Visual Basic também suporta uma estrutura de decisão *If...Then* que permite que você inclua várias expressões condicionais. Esse Bloco de instruções pode ser de várias linhas e conter as palavras-chave EIseIf, Else e End If.

As cláusulas ElseIf e Else permitem que você faça perguntas adicionais na estrutura If ... Then

```
If condição1 Then
        instruções executadas se a condiçãol for True
ElseIf condição2 Then
        instruções executadas se a condição2 for True
        [Cláusulas adicionais ElseIf e instruções podem ser inseridas aqui]
Else
        instruções executadas se nenhuma das condições for True
End If
```

Nessa estrutura, *condição1* é avaliada primeiro. Se essa expressão condicional for True, o Bloco de instruções abaixo dela será executado, uma instrução por vez. (Você pode incluir uma ou mais instruções de programa). Se a primeira condição não for True, a segunda expressão condicional *(condição2)* é avaliada. Se a segunda condição for True, o segundo bloco de instruções é executado (Você pode acrescentar condições ElseIf e instruções adicionais se tiver mais condições para avaliar). Finalmente, se nenhuma das expressões condicionais for True, as instruções abaixo da palavra-chave Else são executadas. Toda estrutura é encerrada pelas palavras-chave End If.

A ordem das expressões condicionais nas suas cláusulas If ... Then e ElseIf é crucial

No próximo exercício, você utilizará a estrutura de decisão *If...Then* para validar os usuários quando entrarem no programa. Você poderá utilizar uma lógica de programa semelhante, se escrever um aplicativo para rede. Abra um novo formulário e insira nele um botão de comando onde em sua rotina de eventos Click deve-se digitar a seguinte instrução:

```
Private Sub Command1_Click()
UserName = InputBox("Entre com o seu primeiro nome.")
        If UserName = "Laura" Then
                MsgBox ("Bem-vinda Laura! Pronta para iniciar o seu PC?")
                Form1.Picture = _
                LoadPicture("c:\visual basic\pcomputr.wmf")
        ElseIf UserName = "Marc" Then
                MsgBox ("Bem-vindo Marc! Pronto para exibir o seu Rolodex?")
                Form1.Picture = _
                LoadPicture("c:\visual basic\rolodex.wmf")
        Else
                MsgBox ("Desculpe, eu não o reconheço.")
                End   'sair do programa
        End If
End Sub
```

Depois do programa Meu Programa Login executado e com a usuária Laura tendo utilizado seu Logon fica a janela apresentada na Figura 5.5.

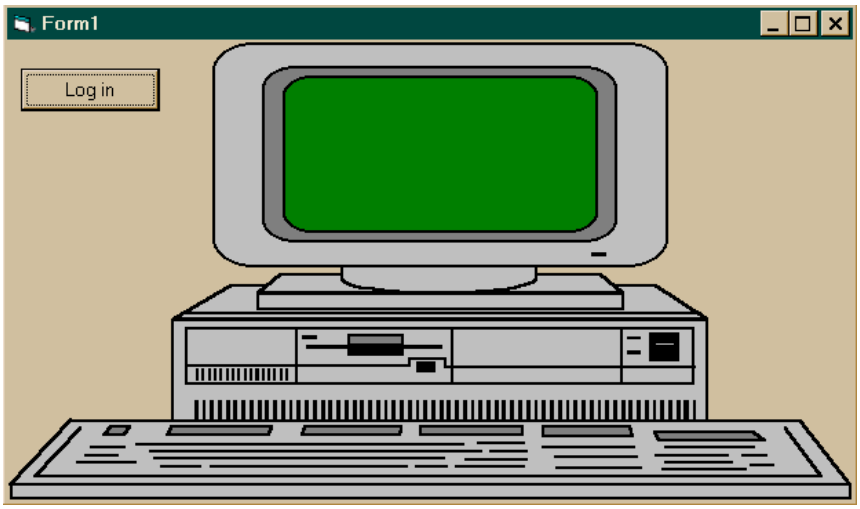


Figura 5.5 - Janela do programa Meu Programa Login com a imagem que é apresentada quando Laura entra no sistema

As linhas de programa podem ter 1023 caracteres na janela de código do Visual Basic, porém, geralmente, é mais fácil trabalhar com linhas menores que 80 caracteres. Você pode dividir as instruções longas de programa em múltiplas linhas utilizando o caractere de continuação de

linha (_) no final de cada linha de instrução, exceto na última linha. (Contudo, você não pode utilizar um caractere de continuação de linha para dividir uma string que estiver entre aspas)

5.6 Utilizando Operadores Lógicos em Expressões Condicionais

O Visual Basic permite que você teste mais de uma expressão condicional nas suas cláusulas *If ... Then* e *ElseIf*, se quiser concluir mais de um critério de seleção na sua estrutura de decisão. As condições extras são ligadas utilizando-se um ou mais dos seguintes operadores lógicos:

Os operadores lógicos permitem que você acrescente testes às suas expressões

Operador lógico	Significado
And	Se ambas as expressões condicionais são True, então o resultado é True.
Or	Se uma expressão condicional é True, então o resultado é True.
Not	Se a expressão condicional é False, então o resultado é True. Se a expressão condicional é True, então o resultado é False.
Xor	Se uma e apenas uma expressão condicional é True, então o resultado é True. Se ambas são True ou ambas são False, então o resultado é False.
Eqv	Segue-se abaixo as combinações que podem ser feitas com Eqv

Se expressão1 for	E expressão2 for	O resultado será
True	True	True
True	False	False
False	True	False
False	False	True

Quando o seu programa avalia uma expressão complexa que mistura diferentes tipos de operador, ele avaliará primeiro todos os operadores matemáticos, em segundo, todos os operadores de comparação e, em terceiro, todos os operadores lógicos

A próxima tabela lista alguns exemplos de operadores lógicos em funcionamento. Nas expressões, considera-se que a variável Vehicle contém um valor "Bike" e a variável Price contém o valor 200.

Expressão lógica	Resultado
Vehicle = "Bike" And Price < 300	True (ambas as expressões são True)
Vehicle = "Car" Or Price < 500	True (uma condição é True)
Not Price < 100	True (a condição é False)
Vehicle = "Bike" Xor Price < 300	False (ambas as condições são True)

No exercício a seguir, você modificará o programa Meu Programa Login fazendo com que o usuário insira uma senha durante o processo de validação. Uma caixa de entrada de dados obtém a senha do usuário e você modifica as cláusulas *If...Then* e *ElseIf* na estrutura de decisão, de maneira que utilizem o operador And para verificar a senha. As linhas de comando depois da inserção da instrução para verificação da senha fica da seguinte forma:

```
Private Sub Command1_Click()
UserName = InputBox("Entre com o seu primeiro nome.")
Pass = InputBox("Digite a sua senha.")
        If UserName = "Laura" And Pass = "May17" Then
                MsgBox ("Bem-vinda Laura! Pronta para iniciar o seu PC?")
                Form1.Picture = _
                LoadPicture("c:\Visual Basic\pcomputr.wmf")
        ElseIf UserName = "Marc" And Pass = "trek" Then
                MsgBox ("Bem-vindo Marc! Pronto para exibir o seu Rolodex?")
                Form1.Picture = _
                LoadPicture("c:\Visual Basic\rolodex.wmf")
```

```
        Else
                MsgBox ("Desculpe, eu não o reconheço.")
                End   'sair do programa
        End If
End Sub
```

5.7 Estruturas de Decisão *Select Case*

O Visual Basic também permite que você controle a execução das instruções nos seus programas utilizando as estruturas de decisão *Select Case*. Uma estrutura *Select Case* é semelhante à estrutura *If ... Then ... EIseIf*, porém é mais eficiente quando o desvio depende de uma variável-chave, ou *test case.* Você também pode utilizar estruturas *Select Case* para tornar o seu código de programa mais legível e eficiente.

As estruturas de decisão Select Case baseiam as decisões de desvio em uma variável- chave

A sintaxe para uma estrutura *Select Case* é:

```
Select Case variável
Case valor1
        instruções de programa executadas se valor1 for igual à variável
Case valor2
        instruções de programa executadas se valor2 for igual à variável
Case valor3
        instruções de programa executadas se valor3 for igual à variavel
.
.
.
End Select
```

Uma estrutura Select Case começa com as palavras-chave *Select Case* e termina com as palavras-chave End Select. Você substitui *variável* do *Select Case* por uma outra variável, por uma propriedade ou por outra expressão que deve ser o valor-chave, ou caso de teste, para a estrutura. Você substitui *valor1, valor2 e valor3* por números, strings ou outros valores relacionados para o caso de teste que está sendo considerado. Se um dos valores for igual à variável, as instruções abaixo da cláusula Case serão executadas e o Visual Basic continuará executando o código de programa após a instrução End Select. Você pode incluir qualquer número de cláusulas Case em uma estrutura *Select Case* e pode incluir mais de um valor na cláusula Case. Se você listar valores múltiplos após um caso, separe-os por vírgulas.

5.8 Utilizando Operadores de Comparação com a Estrutura *Select Case*

O Visual Basic permite que você utilize operadores de comparação para incluir uma faixa de valores de teste em uma estrutura *Select Case*. Os operadores de comparação do Visual Basic que podem ser utilizados são =, <, >, <>, > = e <=. Para utilizar os operadores de comparação, você precisa incluir a palavra-chave *Is* ou a palavra-chave *To* na expressão para identificar a comparação que está fazendo. A palavra-chave *Is* dá instrução ao compilador para comparar a variável de teste com a expressão listada após a palavra-chave *Is*. A palavra-chave *To* identifica uma faixa de valores. A estrutura a seguir utiliza *Is*, *To* e vários operadores de comparação para testar a variável Idade e para exibir uma das cinco mensagens:

```
Select Case Idade
        Case Is < 13
        Label1.Caption = "Aproveite a sua infância!"
        Case 13 To 19
        Label1.Caption = "Aproveite a sua adolescência!"
        Case 21
        Label1.Caption = "Você pode beber vinho em suas refeições”
        Case Is > 100
        Label1.Caption = "Você está bem!"
```

```
        Case Else
        Label1.Caption = " É uma boa idade para se ter."
End Select
```

Uma estrutura Select Case suporta operadores de comparação exatamente como a estrutura If ...Then

Se o valor da variável Idade for menor que 13, a mensagem "Aproveite a sua infância!" será exibida. Para as idades de 13 até 19, a mensagem "Aproveite a sua adolescência!" é exibida e assim por diante.

Uma estrutura de decisão *Select Case* geralmente é muito mais clara que uma estrutura *If....Then*, e é mais eficiente quando você estiver tomando três ou mais decisões de desvio baseadas em uma variável ou propriedade. Contudo, quando estiver fazendo duas ou mais comparações, ou quando estiver trabalhando com vários valores diferentes, você provavelmente utilizará a estrutura de decisão *If...Then*.

No exercício seguinte, você verá como pode utilizar uma estrutura *Select Case* para processar as entradas de dados de uma caixa de listagem. Você utilizará as propriedades List1.Text e List1.ListIndex para coletar as entradas de dados e, então, utilizará uma estrutura *Select Case* para exibir uma saudação em uma das quatro linguagens.

Objeto	**Propriedade**	**Configuração**
Label1	Caption	"Programa de Boas-vindas Internacional"
	Font	Times New Roman, Negrito, 14 pontos
Label2	Caption	"Escolha um país"
Label3	Caption	(em branco)
Label4	Caption	(em branco)
	BorderStyle	1 – FixedSingle
	ForeColor	Vermelho-escuro (&H00000080&)
Comrnand1	Caption	"Sair"

Dê um clique duplo no formulário, e digite, na procedure de evento Form_Load, o código para inicializar a caixa de listagem, veja abaixo:

```
Private Sub Form_Load()
   List1.AddItem "Inglaterra"
   List1.AddItem "Alemanha"
   List1.AddItem "Espanha"
   List1.AddItem "Itália"
   List1.AddItem "Brasil"
End Sub
```

Os valores são carregados na caixa de listagem utilizando-se o método AddItem

Agora, dê um clique duplo na caixa de listagem, e digite, na procedure de evento List1_Click, o código para processar a seleção da caixa de listagem feita pelo usuário:

A propriedade ListIndex contém o número do item de lista selecionado

```
Private Sub List1_Click()
   Label3.Caption = List1.Text
   Select Case List1.ListIndex
   Case 0
      Label4.Caption = "Hello, programmer"
   Case 1
      Label4.Caption = "Hallo, Programmierer"
```

```
    Case 2
      Label4.Caption = "Hola, programador"
    Case 3
      Label4.Caption = "Ciao, programmatori"
    Case 4
      Label4.Caption = "Oi programador"
    End Select
End Sub
```

O programa depois de pronto e executado fica disposto da forma que segue:

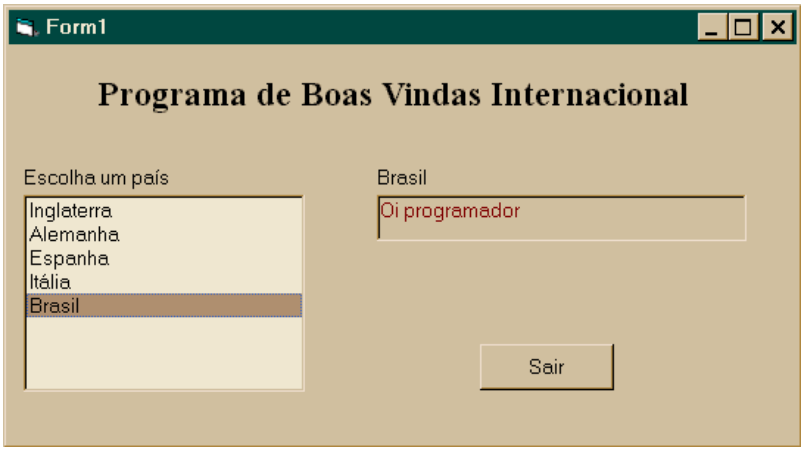


Figura 5.8 - Janela do programa "Boas Vindas Internacional", exemplo de utilização da estrutura de decisão *Select Case.*

6.0 UTILIZANDO LOOPS E TEMPORIZADORES

Na lição anterior, você aprendeu a utilizar estruturas de decisão *If...Then e Select Case* para escolher quais instruções executar no programa. Nesta lição, você aprenderá a executar um bloco de instruções várias e várias vezes utilizando um *loop.* Você utilizará um loop *For...Next* para executar instruções por um certo número de vezes e utilizará um loop *Do* para executar instruções até que uma expressão condicional no loop seja True. Você também aprenderá a utilizar o método Print para exibir texto e números no formulário e a utilizar um objeto temporizador para executar o código em intervalos específicos no seu programa.

6.1 Escrevendo Loops *For...Next*

Um loop *For...Next* permite que você execute um grupo específico de instruções de programa por um certo número de vezes na procedure de evento. Isso pode ser útil se você estiver executando diversos cálculos relacionados, trabalhando com elementos na tela ou processando várias partes das entradas de dados do usuário. Um loop *For...Next* é na verdade uma forma abreviada de escrever uma longa lista de instruções de programa. Uma vez que cada grupo de instruções na lista faria essencialmente a mesma coisa, o Visual Basic permite que você defina um grupo de instruções e solicite que ele seja executado quantas vezes quiser.

No loop For...Next, começar e terminar determinam quanto tempo o loop deve ser executado

A sintaxe do loop For...Next é parecida com:

```
For variável = começar To terminar
        instruções que devem ser repetidas
```

```
Next variável
```

Nessa sintaxe de instrução, *For*, *To* e *Next* são palavras-chave necessárias, e "=" é o operador necessário. Você substitui *variável* pelo nome de uma variável numérica que controla a contagem do loop atual, e substitui *começar* e *terminar* por valores numéricos representando os pontos de início e fim do loop, respectivamente. A linha ou as linhas entre as instruções *For* e *Next* são as instruções que são repetidas toda vez que o loop é executado. Por exemplo, o loop *For...Next* a seguir emite uma sucessão rápida de quatro bipes no alto-falante do computador:

```
For i = 1 To 4
  Beep
Next i
```

Este loop equivale funcionalmente a escrever quatro vezes a instrução Beep na procedure. Para o compilador é o mesmo que,

```
Beep
Beep
Beep
Beep
```

A variável utilizada no loop é *i*, uma simples letra que, por convenção, representa o primeiro contador de números inteiros no loop *For...Next*. Toda vez que o loop for executado, a variável de contador será incrementada em um (a primeira vez que se passar pelo loop, a variável conterá um valor de 1, o valor de *começar;* a última vez terá o valor de 4, o valor de *terminar).* Como será visto nos exemplos a seguir, você pode utilizar a variável de contador com grande vantagem nos seus loops.

Abra um novo formulário, insira nele um botão de comando e configure sua propriedade Caption para Loop. Em seguida digite em sua procedure de evento Command1_Click a seguinte instrução:

```
Private Sub Command1_Click()
  For i = 1 To 10
    Print "Linha"; i
  Next i
End Sub
```

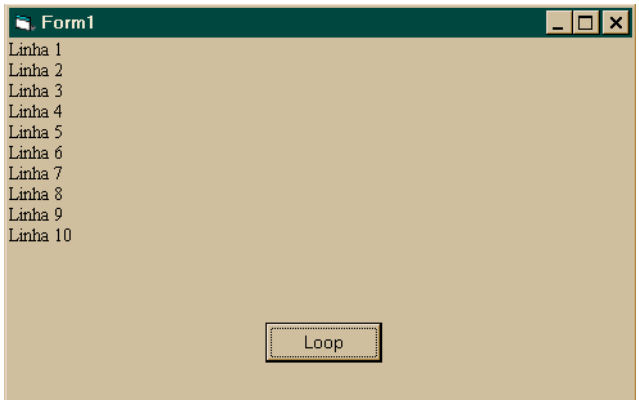


Figura 6.1.1 - Janela do programa utilizando a instrução *For...Next*

O Visual Basic permite que você altere as propriedades e atualize as variáveis-chaves em um loop. Agora, você modificará o programa da Figura 6.1.1, de maneira que ele altere a propriedade *FontSize* no loop *For...Next*. A propriedade *FontSize* especifica o tamanho do texto no formulário. O procedure depois das modificações fica da seguinte forma:

```
Private Sub Command1_Click()
  For i = 1 To 10
    FontSize = 10 + i
    Print "Linha"; i
  Next i
```

```
End Sub
```

A Figura 6.1.2 apresenta o programa depois da inserção da instrução e executado.

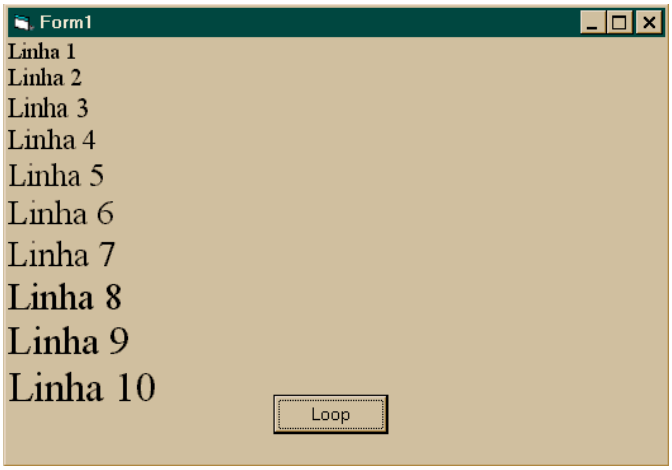


Figura 6.1.2 - Janela do programa utilizando a instrução *For ... Next,* com inserção da instrução que modifica a propriedade *FontSize* (tamanho da fonte) do formulário

O método Print suporta os símbolos de ponto-e-virgula (;) e virgula (,) para separar os elementos em uma lista de expressão. O ponto-e-vírgula coloca os elementos lado a lado e a virgula coloca os elementos separados por um espaço de tabulação. Você pode utilizar qualquer combinação dos símbolos ponto-e-vírgula e vírgula para separar os elementos de lista da expressão

6.2 Loops *For...Next* Complexos

A variável de contador em um loop *For...Next* pode ser uma ferramenta poderosa nos seus programas. Com um pouco de imaginação, é possível utilizá-la para criar várias seqüências úteis de números nos seus loops. Para criar um loop com um padrão de contador que não seja 1, 2, 3, 4, e assim por diante, você pode especificar um valor diferente para *começar* no loop e, então, utilizar a palavra-chave Step para incrementar o contador em intervalos diferentes. Por exemplo, o loop

```
For i = 5 To 25 Step 5
    Print i
Next i
```

imprime a seguinte seqüência de números no formulário:

5
10
15
20
25

Você pode criar uma seqüência diferente de números para a sua variável de contador do For...Next utilizando a palavra-chave Step

Você também pode especificar valores decimais no loop. Por exemplo, o loop *For...Next,*

```
For i = 1 To 2.5 Step 0.5
        Print i
Next i
```

imprime os seguintes números no formulário:

1,0
1,5
2,0
2,5

Além de exibir a variável de contador, você pode utilizar o contador para configurar propriedades, calcular valores ou processar arquivos.

Você pode utilizar a palavra-chave Step com valores decimais

6.3 Instrução *Exit For*

A instrução *Exit For* permite que você saia do loop *For...Next* antes do loop terminar de ser executado. Isso lhe permite responder um evento específico que ocorre antes do loop ser executado um número de vezes. Por exemplo, no loop *For...Next* a seguir

```
For i = 1 To 10
        InpName = InputBox("Digite o seu nome ou Concluido para sair.")
        If InpName  = "Concluido"  Then Exit For
      Print InpName
Next i
```

o loop solicita dez nomes ao usuário e os imprime no formulário, a menos, que a palavra *Concluido* seja inserida (no caso em que o programa salta para primeira instrução seguinte a instrução Next). As instruções *Exit For* geralmente são utilizadas com a instrução *If*. Você irá achá-las úteis quando lidar com casos especiais que surgem em um loop.

6.4 Escrevendo Loops *Do*

Como uma alternativa para o loop *For...Next*, você pode escrever um loop *Do* que executa um grupo de instruções até que uma certa condição seja True no loop. Os loops *Do* são valiosos porque, em geral, você não pode saber com antecedência quantas vezes um loop poderá ser repetido. Por exemplo, você pode querer permitir que o usuário insira nomes em um banco de dados até que ele digite a palavra *Concluido* na caixa de entrada de dados. Nesse caso, você poderá utilizar o loop *Do* para repetir a operação indefinidamente até que a string de texto "Concluido" seja inserida.

Os loops Do executam o código até que uma condição específica seja satisfeita

Um loop *Do* possui vários formatos, dependendo de onde e como a condição de loop for avaliada. A sintaxe mais comum é,

```
Do While condição
        bloco de instruções que devem ser executadas
Loop
```

Por exemplo, as instruções seguintes são um loop *Do* que processará a entrada de dados até que seja inserida a palavra *Concluido:*

```
Do While TnpName <> "Concluido"
        InpName = InputBox("Digite o seu nome ou Concluído para sair. " )
        If InpName <> "Concluido " Then Print InpName
Loop
```

A colocação do teste condicional afeta a maneira como o loop Do é executado

A instrução condicional nesse loop é InpName <> "Concluido", que o compilador do Visual Basic traduz para "repita em loop até que a variável InpName não contenha a palavra *Concluido".* Isso traz à tona um fato interessante dos loops *Do*: se a condição no topo do loop não for True quando a instrução *Do* for avaliada em um primeiro momento, o loop *Do* nunca será executado. Nesse exemplo, se a variável InpName contivesse a string de texto "Concluido" antes do loop ser iniciado (talvez a partir de uma atribuição anterior na procedure de evento), o Visual Basic saltaria todo o loop e continuaria na linha abaixo da palavra-chave Loop. Observe que esse tipo de loop exigiu uma estrutura extra *If...Then* para evitar que o valor de saída fosse exibido quando o usuário o digitasse.

Se você quiser que o loop seja sempre executado pelo menos uma vez no programa, coloque o teste condicional no final do loop. Por exemplo, o loop,

```
Do
  InpName = InputBox("Digite o seu nome ou Concluido para sair.")
  If InpName <> "Concluido" Then Print InpName
Loop While InpName <> "Concluido"
```

é essencialmente o mesmo que o loop *Do* mostrado anteriormente, porém a condição de loop é testada após o nome ter sido recebido pela função InputBox. Isso tem a vantagem de atualizar a variável InpName antes do teste condicional no loop, assim, o valor "Concluido" pré-existente não fará com que o loop seja saltado. Testar a condição de loop no final assegura que o seu loop seja executado pelo menos uma vez, porém você sempre precisará adicionar algumas instruções extras para processar os dados.

6.5 Evitando um Loop sem Fim

Devido à natureza impiedosa dos loops Do, é muito importante projetar as suas condições de teste, de maneira que cada loop possua um ponto de saída verdadeiro. Se um teste de loop nunca for avaliado como False, o loop será executado sem fim e o seu programa não mais responderá à entrada de dados.

Certifique-se de que cada loop tenha uma condiçã o de saída legítima

Considere o seguinte exemplo:

```
Do
   Number = InputBox("Insira um numero para ser elevado ao quadrado. Digite -1 para sair.”)
   Number = Number * Number
   Print Number
Loop While Number >= 0
```

Nesse loop, o usuário insere um número após o outro e o programa elevará ao quadrado cada número e irá imprimi-lo no formulário. Infelizmente, quando o usuário já tiver digitado o suficiente, ele não poderá sair, pois a condição de saída anunciada não funciona. Quando o usuário inserir -1, o programa elevará ao quadrado e a variável Number terá atribuído o valor 1. (o problema pode ser resolvido configurando-se uma condição de saída diferente.) Os loops sem fim devem ser observados quando você estiver escrevendo loops *Do*. Felizmente, eles são muito fáceis de serem identificados se você testar completamente os seus programas.

O exercício a seguir mostra como você pode utilizar um loop *Do* para converter uma temperatura Fahrenheit em Ceisius. Este programa simples solicita que o usuário insira os dados através de uma função InputBox, converte a temperatura e exibe o resultado em uma caixa de mensagem. O programa também demonstra como você pode ocultar um formulário configurando a propriedade Visible para False.

Abra um formulário e digite em sua procedure de evento Form_Load a seguinte instrução e depois execute o programa para visualizar o resultado. Para uma temperatura de 32º Fahrenheit tem-se um temperatura em ºC igual a zero. Para finalizar o programa não digite número algum.

```
Private Sub Form_Load()
   Prompt = "Digite uma temperatura em graus Fahrenheit."
  Do
```

```
    FTemp = InputBox(Prompt, "Fahrenheit para Celsius")
    If FTemp <> "" Then
      Celsius = Int((FTemp - 32) * 5 / 9 )
      MsgBox (Celsius), , "Temperatura em Celsius"
    End If
  Loop While FTemp <> ""
  End
End Sub
```

6.6 Utilizando a Palavra-Chaves *Until* em Loops *Do*

Os loops *Do* com os quais você trabalhou até aqui utilizaram a palavra-chave *While* para executar um grupo de instrução, enquanto a condição loop permanecesse True. O Visual Basic também permite que você utilize a palavra-chave *Until* nos loops *Do* para repetir a operação indefinidamente até que uma certa condição seja True. A palavra-chave *Until* pode ser utilizada no inicio ou no final do loop *Do* para se testar uma condição, exatamente como palavra-chave *While*. Por exemplo, o loop *Do* a seguir utiliza a palavra-chave *Until* para fazer um loop repetidamente, até que o usuário insira a palavra *Concluido* na caixa de entrada de dados:

```
Do
  InpName = InputBox("Digite o seu nome ou Concluido para sair.")
  If InpName <> "Concluido" Then Print InpName
Loop Until InpName = "Concluido"
```

Como você pode observar, o loop que utiliza a palavra-chave *Until* é muito semelhante ao loop *Do* que utiliza a palavra-chave *While*, exceto que a condição a ser testada geralmente contém um operador de oposição – o operador = versus o operador <>, neste caso. Se utilizar a palavra-chave *Until* fizer mais sentido para você, sinta-se a vontade, para usá-la com as condições de teste nos seus Loop *Do*.

7.0 TRABALHANDO COM FORMULÁRIOS, IMPRESSORAS E SUB-ROTINAS DE TRATAMENTO DE ERROS

Você aprendeu a utilizar uma variedade de instruções, funções e estruturas de controle do Microsoft Visual Basic para executar trabalhos úteis nos seus programas. Você irá se concentrar novamente na interface do usuário e aprenderá a criar efeitos impressionantes e aplicativos "à prova de bala". Nesta lição, você aprenderá a acrescentar mais formulários a uma interface para manipular entrada e saída de dados, ou mensagens. Você também aprenderá a enviar dados de um programa para uma impressora e a utilizar sub-rotinas de tratamento de erros para processar resultados inesperados.

7.1 Acrescentando Novos Formulários ao Programa

Cada programa que você escreveu até aqui utilizou apenas um formulário para entrada e saída de dados. Em muitos casos, um formulário será suficiente para a comunicação com o usuário. Mas se você quiser acrescentar mais telas de informação ao programa, o Visual Basic permite que se acrescente um ou mais formulários extras. Cada novo formulário é considerado um objeto e mantém os seus próprios objetos, propriedades e procedures de evento. O primeiro formulário no programa é intitulado Form1 e os formulários subseqüentes são intitulados Form2, Form3 e assim por diante. Você cria novos formulários dando um clique no comando Add Form no menu Project, ou dando um clique no botão Add Form na barra de ferramentas. A tabela a seguir relaciona várias utilizações práticas para os formulários extras nos seus programas.

Cada novo formulário tem um único nome e seu próprio conjunto de objetos, propriedades e procedures de evento

Formulário ou formulários	Descrição
Tela introdutória	Uma tela que exibe uma mensagem de boas-vindas, arquivos gráficos ou informações sobre direitos autorais, mostrada inicialmente.
Instruções de programa	Uma tela que exibe informações e dicas sobre como o programa funciona.
Caixas de diálogo	Caixas de diálogo personalizadas que aceitam entradas de dados e exibem saídas de dados no programa
Conteúdos de documento e arquivos gráficos	Uma tela que exibe o conteúdo de um ou mais arquivos utilizados no programa.

7.2 Como os Formulários São Utilizados

Em Visual Basic, você tem uma flexibilidade significativa quando utiliza formulários. Você pode deixar visível todos os formulários no programa ao mesmo tempo, ou pode carregar ou descarregar formulários, conforme o programa necessitar. Se você exibir mais de um formulário por vez, poderá permitir que o usuário alterne entre os formulários, ou poderá controlar a ordem na qual os formulários são utilizados. Um formulário que deve ser utilizado quando for exibido na tela é conhecido como formulário *modal* - o formulário que retém o foco até que o usuário dê um clique em OK, dê um clique em Cancelar ou o despache de outra forma. Um formulário que o usuário pode alternar é conhecido como formulário *não-modal, ou sem modo.* A maioria dos aplicativos para o Microsoft Windows utiliza formulários não-modais quando exibem informações, pois dão maior flexibilidade ao usuário, assim o não--modal é o padrão quando você cria um novo formulário. Você também pode configurar de maneira independente qualquer propriedade no formulário, incluindo legenda, tamanho do formulário, estilo de borda, cores de fundo e de primeiro plano, fonte de exibição e figura de fundo do formulário.

Os novos formulários podem ser modais ou não-modais

7.3 Instruções do Formulário em Código de Programa

Após criar um novo formulário no ambiente de programação, você pode carregá-lo na memória e acessá-lo utilizando instruções específicas nas procedures de evento. A instrução que você utiliza para carregar um novo formulário tem a sintaxe:

Load *nomeformulário*

em que, *nomeformulário* é o nome do formulário que você quer carregar. Por exemplo, a instrução

Load Form2

carregaria o segundo formulário de um programa na memória, quando a instrução fosse executada pelo Visual Basic. Após você ter carregado um formulário, você pode utilizá-lo a partir de qualquer procedure de evento no programa e pode acessar qualquer propriedade ou método que quiser. Por exemplo, para configurar a propriedade Caption do segundo formulário no seu programa para "Ordenar os Resultados", você poderá digitar a seguinte instrução de programa em qualquer procedure de evento:

Form2.Caption = " Ordenar os Resultados "

A instrução Load traz um novo formulário para a memória e o método Show exibe um formulário carregado

Quando estiver pronto para exibir um formulário carregado, você o chama utilizando o método Show e observa se é modal ou não-modal. A sintaxe para o método Show é,

Nomeformulário.Show modo

em que, *nomeformulário* é o nome do formulário e modo é 0 para não-modal (padrão) ou 1 para modal. Por exemplo, para exibir Form2 como formulário não-modal, você poderá utilizar o método Show sem especificar um modo:

Form2.Show

O padrão para o novo formulário é não-modal

Para exibir o Form2 como um formulário modal, você deverá digitar esta instrução:

Form2.Show 1

Quando você exibe um formulário como modal ele se torna o único formulário ativo, ou seja, enquanto você não realizar as tarefas contidas nesse formulários os outros não poderão ser ativados (acessados). Isto é muito usado quando você deseja que o usuário responda as questões daquele formulário, antes de tentar sair do respectivo formulário. Geralmente, quando o formulário é exibido como modal e o usuário tenta clicar fora dele para ativar outra janela, o computador emite um som de beep.

Se você utilizar o método Show antes de utilizar a instrução Load, o Visual Basic carregará e exibirá automaticamente o formulário especificado. O Visual Basic fornece uma instrução Load separada para permitir que os programadores pré-carreguem os formulários na memória, de maneira que o método Show funcione rapidamente e os usuários não observem qualquer retardamento de desempenho. É uma boa idéia pré-carregar os seus formulários, especialmente se eles contiverem vários objetos ou arquivos gráficos

7.4 Ocultando e Descarregando Formulários

Você pode ocultar os formulários utilizando o método *Hide* e pode descarregá-los utilizando a instrução *Unload*. Essas palavras-chave são o oposto de *Show* e *Load*, respectivamente. Ocultar (hiding) um formulário o torna invisível, mas mantendo-o na memória para uso posterior no programa. (Ocultar um formulário é idêntico a torná-lo invisível utilizando a propriedade *Visible*). Descarregar (*unload*) um formulário o remove da memória. Isso libera memória RAM utilizada para armazenar os objetos e os gráficos no formulário, mas não libera o espaço utilizado pelas procedures de evento do formulário, esses estão sempre na memória. Você poderá utilizar as palavras-chave *Hide* e *Unload* para ocultar e descarregar o Form2 da seguinte maneira:

Form2.Hide
Unload Form2

O método Hide torna o formulário invisível. Após descarregar um formulário, serão perdidos os seus valores e as propriedades do tempo de execução. Se você recarregar o formulário, ele conterá os valores originais definidos no código de programa. A instrução Unload remove o formulário da memória

Você pode utilizar também a propriedade *Visible* para mostrar ou ocultar um formulário. A instrução é da seguinte forma:

Nomeformulário.Visible = Operador Lógico (True ou False)

7.5 Formulários MDI: Janelas que Possuem Relacionamento Pai-Filho

Na maioria dos programas que você escreverá, possivelmente criará um formulário "caseiro" padrão, onde o usuário faz parte do seu trabalho e, então, acrescentará formulários com a finalidade específica para lidar com a entrada e saída de dados do programa. Contudo, o Visual Basic também permite uma hierarquia, que é um relacionamento especial entre formulários em um programa que funciona melhor como grupo. Esses formulários especiais são denominados MDI (Multiple Document Interface – Interface para Múltiplos documentos) e são distinguidos pelos seus papeis como formulários pais e filhos. Você cria um formulário-pai MDI dando um clique no comando MDI Form no menu Project;

você cria um formulário-filho MDI dando um clique no comando Form no menu Project, e, então, configurando a propriedade MDI Child do formulário para True. Os formulários MDI têm função semelhante aos formulários comuns durante o tempo de execução, com as seguintes exceções:

- Todos os formulários-filhos são exibidos dentro do espaço da janela do seu formulário-pai;
- Quando um formulário-filho é minimizado, ele encolhe até o tamanho de um pequeno barra de título no formulário MDI-pai, em vez de aparecer como um botão na barra de tarefas;
- Quando o formulário-pai é minimizado, o pai e todos os filhos aparecem como um botão na barra de tarefas;
- Todos os menus-filhos são exibidos na barra de menus do formulário-pai. Quando um formulário-filho é maximizado, sua legenda é exibida na barra de títulos do pai;
- Você pode exibir todos os formulários-filhos de um formulário-pai configurando a propriedade AutoShow Children do formulário-pai para True;
- Os formulários pais e filhos são especialmente úteis em aplicativos centrados em documentos, nos quais muitas janelas são utilizadas para exibir ou editar um documento. Para aprender mais sobre formulários MDI e suas aplicações, procure MDI na Ajuda online do Visual Basic.

7.6 Enviando as Saídas do Programa para uma Impressora

O Visual Basic permite que você envie as saídas de dados para uma impressora instalada utilizando o método Print. Você aprendeu sobre o método Print, anteriormente, quando o utilizou em um loop para exibir o texto no formulário. Para enviar uma saída para uma impressora conectada, você utiliza o método Print com o objeto Printer. Por exemplo, a seguinte linha envia a string de texto "Mariners" para a impressora padrão do Windows:

```
Printer.Print = "Mariners"
```

O objeto Printer controla as impressões

Antes de imprimir, você também pode utilizar o objeto Printer para ajustar certas características de fonte. Por exemplo, o código a seguir imprime "Mariners" no tipo de 14 pontos:

```
Printer.FontSize = 14
Printer.Print = "Mariners"
```

No geral, o objeto Printer possui várias dezenas de propriedades e métodos que você pode utilizar para controlar diferentes aspectos da impressão. Ao contrário de muitos formulários e objetos que você cria utilizando os controles na caixa de ferramentas, as propriedades Printer não podem ser configuradas utilizando-se a janela Properties. Cada propriedade deve ser configurada de acordo com o código de programa no tempo de execução. (A procedure de evento Form_Load é um bom lugar para colocar as configurações padrões que você utilizará toda vez que o seu programa for executado.) Entretanto, muitas das propriedades e dos métodos para o objeto Printer são semelhantes às palavras-chave que você já vem utilizando com os formulários e objetos criados por meio dos controles da caixa de ferramentas. As tabelas a seguir listam algumas das propriedades Printer e dos métodos mais úteis.

Para obter uma listagem completa das propriedades e métodos do objeto Printer, procure Printer na Ajuda online do Visual Basic. Você também pode utilizar as propriedades para determinar as capacidades da sua impressora

Propriedade	Descrição
FontName	Configura o nome da fonte para o texto.
FontSize	Configura o tamanho da fonte para o texto.
FontBold	True configura o estilo da fonte para negrito.
FontItalic	True configura o estilo da fonte para itálico.
Page	Contém o número de página que está sendo impresso.

Método	Descrição
Print	Imprime o texto especificado na impressora.
NewPage	Inicia uma nova página no trabalho da impressão.
EndDoc	Sinaliza o final de um trabalho da impressão.
KillDoc	Termina um trabalho da impressão atual.

No exercício a seguir, você acrescentará um suporte de impressão básico para um programa. Você utilizará as propriedades FontName, FontSize e FontBold para mudar o estilo do texto, o método Print para enviar as definições para a impressora, e o método EndDoc para sinalizar o final do trabalho da impressão.

```
Printer.FontName = "Arial"
Printer.FontSize =14
Printer.FontBold = True
Printer.Print  "Imprimindo Arquivo"
Printer.FontBold = False
Printer.Print "Teste de Impressão"
Printer.EndDoc
```

Utilizar uma das fontes TrueType fornecida pelo Windows tomará o seu programa compatível com a maioria das impressoras

A finalidade de cada uma dessas instruções é clara. As primeiras três instruções configuram as propriedades FontName, FontSize e FontBold do objeto Printer para as configurações suportadas pela impressora padrão. A fonte Arial é fornecida na maioria das versões do Windows e, uma vez que Arial é uma fonte TrueType, você pode utilizá-la para exibir informações na tela e enviar informações para a impressora, você não está limitado às fontes embutidas da impressora. A quarta instrução utiliza o método Print para enviar a palavra "Teste de Impressão" para a impressora. Ela imprime a palavra em negrito na primeira linha da página atual e, então instrui a impressora para mover-se para o início da segunda linha. A fonte da impressora é então configurada para normal e a definição da palavra é impressa. Finalmente, o método EndDoc termina o trabalho da impressão.

Se você quiser permitir que o usuário imprima várias definições de palavras em uma página, adie o uso do método EndDoc até que o usuário dê um clique em Sair para finalizar o programa. Se a sua impressora não estiver pronta para imprimir, o Windows poderá retornar um erro para o Visual Basic, indicando que o seu programa não está pronto para ser manipulado. Isso poderá resultar em um erro de tempo de execução, ou em uma falha de programa. Você aprenderá a lidar com erros de tempo de execução associados a drives de disco, impressoras e outros dispositivos mais adiante nesta lição

7.7 Imprimindo um Formulário Inteiro Utilizando o Método PrintForm

Como uma alternativa para a impressão individual de linhas utilizando o método Print, você pode enviar todo o conteúdo de um ou mais formulários para a impressora utilizando o método PrintForm. Esta técnica permite que você disponha os elementos de texto, gráficos e interface do usuário que você quiser no formulário e, então, envie o formulário inteiro para a impressora. Você pode utilizar a própria palavra-chave PrintForm para imprimir o formulário atual, ou você pode incluir o nome de um formulário para imprimir um formulário específico. Por exemplo, para imprimir o conteúdo de um segundo formulário em um programa, você poderá inserir a instrução,

```
Form2.PrintForm
```

em qualquer procedure de evento do programa.

O método PrintForm imprime o seu formulário na resolução atual do seu monitor, em geral 96 pontos por polegada. O método PrintForm imprime somente os objetos que estiverem atualmente visíveis no formulário. Para remover objetos indesejados de uma saída de impressão (tal como um botão de comando), configure a propriedade visible dos objetos para False antes de chamar o método PrintForm. Após enviar o formulário para a impressora, você pode fazer com que os objetos fiquem visíveis novamente

7.8 Processando Erros Utilizando as Sub-rotinas de Tratamento de Erros

Você já teve uma experiência com um erro de tempo de execução no programa Visual Basic? *Erro de tempo de execução, ou falha de programa,* é um evento inesperado que acontece quando o seu programa está sendo executado e que o Visual Basic não consegue superar. Você pode ter experimentado a primeira falha no seu programa nesta lição, quanto tentava imprimir, e nada funcionava direito. (Talvez a impressora estivesse sem papel, ou estivesse desligada e você recebeu uma mensagem de erro de impressora do Visual Basic ou do Windows). Um erro de tempo de execução acontece em qualquer momento em que o Visual Basic execute uma instrução que, por algum motivo, não esteja completa *"como deveria estar".* Não é que o Visual Basic seja rígido demais para lidar com essa falha, é só que o compilador não sabe o que fazer quando algo de errado acontece.

Uma sub-rotina de tratamento de erros ajuda o seu programa a se recuperar dos erros de tempo de execução

Felizmente, você não tem de conviver com erros ocasionais que provocam falhas nos seus programas. O Visual Basic permite que você escreva sub-rotinas especiais, chamadas *tratamento de erros,* para responder aos erros de tempo de execução. A sub-rotina de tratamento de erro lida com um erro de tempo de execução dizendo *ao programa* como continuar quando uma das suas instruções não funcionar. As sub-rotinas de tratamento de erros são colocadas nas mesmas procedures de evento em que as instruções potencialmente instáveis estiverem, e elas lidam, ou *capturam,* um problema utilizando um objeto de tratamento de erro especial chamado *Err*. *Err* possui uma propriedade *Number* que identifica o erro e permite que o seu programa responda a ele. Por exemplo, se um disco flexível provocar um erro, a sua sub-rotina de tratamento de erros deverá exibir uma mensagem de erro personalizada e, então, desativar as operações de disco até que o usuário resolva o problema.

7.9 Quando utilizar as Sub rotinas de Tratamento de Erros

Você pode utilizar as sub-rotinas de tratamento de erros em qualquer situação na qual uma ação inesperada possa resultar em erro de tempo de execução. Em geral, as sub-rotinas de tratamento de erros são utilizadas para processar eventos externos que influenciam um programa – por exemplo, os eventos provocados por uma falha de drive de rede, por uma porta de drive de disco flexível aberta ou por uma impressora que está fora de linha. A próxima tabela lista os problemas potenciais que podem ser endereçados pelas sub-rotinas de tratamento de erros.

A maioria dos erros de tempo de execução é provocada por eventos externos

Problemas	Descrição
Problemas de rede	Os drives de rede ou recursos que falham, ou "caem", inesperadamente.
Problemas em discos flexíveis	Discos não formatados ou formatados incorretamente, portas de drive abertas ou setores ruins de disco.
Impressoras off-line	Impressoras que estão off-line, sem papel ou de alguma forma não disponível.
Erros de estouro de dados	Imprimir ou desenhar muita informação no formulário.
Erro de falta de memória	Falta de espaço para aplicativo ou recursos no Windows.
Problemas com área de transferência	Problemas com transferência de dados ou com a Área de Transferência do Windows.
Erros de lógica	Erros de lógica ou sintaxe não detectados pelo compilador e pelos testes anteriores (tal como um nome de arquivo digitado incorretamente)

7.10 Definindo a Armadilha: A Instrução On Error

A instrução de programa utilizada para detectar um erro de tempo de execução é *On Error*. Você coloca *On Error* na procedure de evento, exatamente antes de utilizar a instrução com a qual estiver

preocupado. A instrução *On Error* define, ou *ativa,* uma armadilha de evento dizendo ao Visual Basic para onde desviar, caso encontre um erro. A sintaxe para a instrução *On Error* é

On Error GoTo *Label*

em que, *Label* é o nome da sua sub-rotina de tratamento de erros.

A instrução On Error identifica a sub-rotina de tratamento de erros

As sub-rotinas de tratamento de erros são digitadas próximo da parte inferior de uma procedure de evento, seguidas pela instrução *On Error*. Cada sub-rotina de tratamento de erros tem o seu próprio rótulo, que é precedido por dois-pontos (:) para identificação – por exemplo, ErrorHandler: ou PrinterError:. Uma sub-rotina de tratamento de erros geralmente possui duas partes. A primeira parte pode, em geral, utilizar a propriedade *Err.Number* na estrutura de decisão (tal como, *If...Then* ou *Select Case*) e, então, exibir uma mensagem ou configurar uma propriedade com base no erro. A segunda parte é uma instrução *Resume* que envia o controle de volta para o programa, de maneira que o programa possa continuar. Na instrução *Resume*, você pode utilizar a palavra-chave *Resume* sozinha, utilizar as palavras-chave *Resume Next* ou utilizar a palavra-chave *Resume* com um rótulo para o qual você gostaria que fosse desviado, dependendo de qual parte do programa você quer que continue.

Resume, Resume Next e o rótulo Resume devolvem o controle ao programa

A palavra-chave *Resume* devolve o controle para a instrução que provocou o erro (na esperança de que a condição de erro seja reparada ou que não ocorra novamente). Utilizar a palavra-chave *Resume* é uma boa estratégia, se você pedir ao usuário que resolva o problema, como, por exemplo, fechar a porta do drive de disco flexível, ou verificando uma impressora. As palavras-chave *Resume Next* devolvem o controle para a instrução *seguinte* à que provocou o erro. Utilizar as palavras-chave *Resume Next* é a estratégia a ser tomada se você quiser saltar o comando e continuar trabalhando. Você também pode colocar após a palavra-chave *Resume* o rótulo para o qual gostaria que fosse desviado. Isso lhe dá a flexibilidade de mover-se de qualquer lugar na procedure de evento para onde quiser. A localização típica para o desvio é a última linha da procedure.

O exemplo a seguir mostra como você pode criar uma sub-rotina de tratamento de erros para se recuperar de erros associados aos drives de disco flexível. Você acrescentará a sub-rotina de tratamento de erros ao programa para tentar carregar um metafile do Windows de um disco no drive A. Você pode utilizar a mesma técnica para acrescentar um suporte de tratamento de erro ao programa Visual Basic apenas altere os números e as mensagens de erro.

O programa a seguir utiliza um número de erro (da propriedade Err.number) para diagnosticar um erro de tempo de execução. Para ver a listagem completa dos números de erro, pesquise error codes na Ajuda on-line do Visual Basic

Abra um novo for formulário, e nele insira um botão de comando, um label e uma caixa de imagem. Na procedure de evento click do botão digite a instrução abaixo, o label terá descrito o texto da Figura # e caixa de imagem fica como está.

```
On Error GoTo DiskError
Image1.Picture = LoadPicture("a:\prntout2.wmf")
Exit Sub  'sair da procedure
DiskError:
If Err.Number = 71 Then  'Se o disco não estiver pronto
   MsgBox ("Por favor, feche a trava do drive ."), , _
    "O disco não está pronto"
   Resume
Else
```

```
    MsgBox ("Não posso achar o arquivo prntout2.wmf no caminho A:\."), , _
     "Arquivo Não Encontrado"
    Resume StopTrying
  End If
  StopTrying:
```

Essa sub-rotina de tratamento de erros lida com os problemas de drive de disco flexível. Se a mensagem de erro ainda aparecer, poderá estar ocorrendo um erro diferente no seu programa. Tente entrar no seu código de programa útil ando a janela Debug. Os erros são fáceis de identificar utilizando expressões de observação e o processo mostrará graficamente a você como funciona a sub-rotina de tratamento de erros

7.11 Outras Técnicas sobre Sub-rotinas de Tratamento de Erros

O objeto *Err* contém algumas outras propriedades que você poderá utilizar para exibir informações adicionais nas suas sub-rotinas de tratamento de erros. A propriedade *Err.Description* contém a mensagem de erro retornada ao Visual Basic quando ocorre um erro de tempo de execução. Você pode utilizar essa mensagem como uma fonte adicional de informação para o usuário, independentemente de você querer ou não responder ao erro no programa. Por exemplo, a sub-rotina de tratamento de erros a seguir utiliza a propriedade *Description* para exibir uma mensagem de erro, se um erro ocorre quando você carrega um arquivo gráfico de um disco flexível.

```
On Error GoTo DiskError
Imagel.Picture = LoadPicture("a:\prntout2.wmf")
Exit Sub   'sair da procedure
DiskError:
MasgBox (Err.Description), , "Erro de carregamento".
Resume     'tentar a função LoadPicture novamente
```

Você pode utilizar essa técnica para capturar problemas com disco flexível, tais como discos não formatados, arquivos faltantes ou trava de drive aberta. A sub-rotina de tratamento de erros utiliza a instrução *Resume* para tentar a operação de carregar novamente, quando o usuário resolver o problema e der um clique em OK na caixa de mensagem. Quando o arquivo eventualmente for carregado, a instrução Exit Sub encerrará a procedure de evento.

A propriedade Err.Description contém uma explicação sobre o erro de tempo de execução. Se você ficar preso em um loop de erro não conseguir sair, pressione CTRL + BREAK

7.12 Especificando um Período de Nova Tentativa

Outra estratégia que você pode utilizar na sub-rotina de tratamento de erros é tentar uma operação algumas vezes e, então, "saltar" o problema, caso não seja resolvido. Por exemplo, a sub-rotina de tratamento de erros a seguir utiliza uma variável de contador intitulada "Retries" para rastrear o número de vezes que uma mensagem de erro foi exibida e, então, o programa salta a instrução Load se esta falhar duas vezes:

```
Retries = 0        'inicializar variável de contador
On Error GoTo DiskError
Imagel.Picture = LoadPicture("a:\prntout2.wmf")
Exit Sub                          'sair da procedure

DiskError
MsgBox (Err.Description), , "Carregando Erro"
Retries = Retries + 1     'incrementar contador em condição de erro
If Retries >= 2 Then
Resume Next
Else
Resume
End If
```

Essa é uma técnica útil se o erro que você está tratando for um problema que pode ser ocasionalmente resolvido pelo usuário. Uma coisa importante para se lembrar aqui é que *Resume* tenta várias vezes a instrução que provocou o erro, e *Resume Next* salta a instrução e vai para a próxima linha na procedure de evento. Quando você utilizar *Resume Next*, certifique-se de que a próxima instrução realmente seja aquela que quer executar e, quando continuar, certifique-se de não executar acidentalmente a sub-rotina de tratamento de erros de novo. Uma boa maneira de saltar uma sub-rotina de tratamento de erros é utilizar uma instrução *Exit Sub*, ou você pode ainda utilizar *Resume Next* com um rótulo que direciona o Visual Basic para continuar executando o que estiver abaixo da sub-rotina de tratamento de erros.

8.0 ACRESCENTANDO ARQUIVOS GRÁFICOS E EFEITOS ESPECIAIS

Para a maioria dos programadores, acrescentar elementos gráficos e efeitos especiais a um aplicativo é a parte mais excitante e viciante da programação. Felizmente, criar efeitos gráficos úteis e impressionantes com o Microsoft Visual Basic é gratificante e fácil. Nesta lição, você aprenderá a acrescentar gráficos interessantes aos seus programas. Você aprenderá, à força, a criar elementos gráficos em um formulário e construir botões de comando gráficos.

8.1 Acrescentando Arquivos Gráficos Utilizando os Controles Line e Shape

Você já aprendeu a acrescentar bitmaps, ícones e metafiles do Windows a um formulário criando objetos caixa de figura e imagem. Acrescentar elementos gráficos pré-confeccionados aos seus programas é fácil no Visual Basic, e você já praticou a utilização deles em quase todas as lições. Agora você aprenderá a criar elementos gráficos originais, utilizando os controles Line e Shape. Essas ferramentas úteis estão na caixa de ferramentas e você pode utilizá-las para construir uma variedade de imagens de diferentes formatos, tamanhos e cores. Os objetos que você cria utilizando esses controles têm algumas limitações não podem receber o foco durante a execução e não podem ser exibidos no topo dos outros objetos, mas são poderosos, rápidos e fáceis de usar.

Os controles Line e Shape permitem que você cria imagens geométricas

8.2 O Controle Line

O controle Line pode ser utilizado para se criar uma linha reta em um formulário. Você pode, então, configurar uma variedade de propriedades para alterar a aparência do objeto linha que você criou, exatamente como faz com os outros objetos. As propriedades mais importantes do objeto linha são BorderWidth, BorderStyle, BorderColor e Visible. A propriedade BorderWidth ajusta a espessura da linha no seu formulário; isso é especialmente útil quando se está criando um sublinhado ou uma linha que separa um objeto de outro. A propriedade BorderStyle permite que se faça a linha sólida, pontilhada ou tracejada, e a propriedade BorderColor permite que você defina a cor da linha em qualquer cor padrão do Visual Basic. Finalmente, a propriedade Visible permite que você oculte ou exiba a linha quando tornar-se necessário no seu programa. Você terá a chance de trabalhar com o controle Line depois que aprender um pouco sobre o controle Shape.

8.3 O Controle Shape

Pode-se utilizar o controle Shape para se criar retângulos, quadrados, ovais e círculos em formulários. Você utiliza o controle Shape para desenhar a imagem que quiser e, então, utiliza a janela Properties para ajustar as características da imagem. A propriedade Shape controla a forma da imagem; é possível selecionar um formato retângulo, retângulo arredondado, quadrado, quadrado arredondado, oval ou circular, depois de criar um objeto. Você pode construir imagens complexas desenhando várias formas e linhas. Outras propriedades importantes (do objeto forma incluem FillColor, que permite que se especifique a cor do objeto; FillStyle, que permite que se especifique o padrão para a cor de preen-

chimento; e BorderColor, que permite que se especifique uma cor separada para a borda do formato. Um objeto *shape* também pode ter uma propriedade *Visible*, a qual permite que você oculte ou exiba o seu elemento gráfico quando necessário.

O exercício a seguir irá lhe dar alguma prática utilizando os controles Line e Shape. Você irá utilizá-los para criar uma tela introdutória de boas-vindas usada por uma empresa fictícia chamada ENGAJE – Soluções Agrícolas. A tela de boas-vindas será parecida com a da próxima figura.

Figura 8.3 – Janela introdutória de um programa, dando as boas vindas ao usuário

Insira no formulário uma Linha, dois Shapes, dois Labels e dois botões de comando, e configure-os da seguinte forma:

Objeto	**Propriedade**	**Configuração**
Line1	BorderWidth	5
	BorderColor	Azul Escuro
Shape1	Shape	0 – Retângulo
	FillColor	Amarelo escuro
	FillStyle	6 – Cross
	BorderColor	Amarelo escuro
Shape2	Shape	4 – Retângulo arredondado
	FillColor	Azul claro
	FillStyle	6 – Cross
	BorderColor	Azul claro
Label1	Caption	"ENGAJE – Soluções Agrícolas"
	ForeColor	Azul escuro
	Font	Times New Roman, Negrito, 24 pontos
	Alignment	2 – center
Label2	Caption	"Projetando sistemas que realmente funcionam"
	Font	Times New Roman, Itálico, 12 pontos
	Alignment	1 – Right Justify
Comando1	Caption	"Sair"
Comando2	Caption	"Continuar"
Form1	Caption	"Bem-Vindo"

Agora, execute o programa, ele deverá ter a mesma configuração apresentada na Figura 8.3.

8.4 Criando Botões de Comando Gráficos

Você tem usado os botões de comando neste livro para dar ao usuário um método intuitivo de emitir comandos. Como você aprendeu, cria-se um botão de comando utilizando-se o controle CommandButton e, então, configura-se a propriedade Caption do botão com uma palavra que descreva o comando que o botão executa. Como uma alternativa para criar botões com base em texto, o Visual Basic permite que você utilize o controle Image para criar botões gráficos nos seus programas. Um botão gráfico contém uma imagem, que é uma representação visual do comando executado pelo botão. Por exemplo, o botão contendo o ícone de um disco flexível representaria um comando que salva as informações em disco. Os botões gráficos podem ser colocados individualmente em programas, ou podem ser agrupados

em coleções chamadas *barras de ferramentas.* A barra de ferramentas do Visual Basic é um exemplo desse tipo de grupo de botão. Nesta seção, você aprenderá a criar botões de comando gráficos autênticos que são "pressionados" e "realçados" quando você dá um clique neles, exatamente como os botões que você viu em outros aplicativos para Windows.

Você pode utilizar o controle Image para criar botões de comando no estilo da barra de ferramentas

8.5 Detectando um Evento MouseDown

Para dar aos seus botões gráficos uma aparência e um "sentido" realísticos, o seu programa deverá responder imediatamente quando o usuário colocar o ponteiro do mouse sobre um botão gráfico e pressionar o botão do mouse. Até aqui você tem utilizado o evento *Click* para tomar as ações nos seus programas, mas neste caso o *Click* não será suficiente. O seu programa precisa responder quando o usuário primeiro *pressionar* o botão, e não depois que soltar o botão. Você pode rastrear a atividade do mouse no Visual Basic utilizando o evento *MouseDown*. *MouseDown* é um evento especial que executa uma procedure de evento sempre que o usuário colocar o ponteiro do mouse sobre um objeto no formulário e, então, pressionar o botão do mouse. Se você escrever uma procedure de evento especial para o evento *MouseDown* (tal como Image1.MouseDown), o seu programa poderá tomar a ação quando o usuário pressionar o botão do mouse enquanto o ponteiro do mouse estiver sobre o objeto. Quando estiver criando botões de comando gráficos, você pode querer que o seu programa altere a figura no botão quando o usuário der um clique (para dar ao botão aquela aparência de "estar pressionado") e, então, execute o comando especificado no programa.

O evento MouseDown detecta a primeira metade do clique no botão do mouse. Além de reconhecer os eventos MouseDown, o seu programa pode reconhecer os eventos MouseUp (gerado quando o usuário libera o botão do mouse) e MouseMove (gerado Quando o usuário move o mouse)

8.6 Permuta de Botões

Como fazer para que os botões de comando gráficos pareçam estar pressionados e realçados quando é dado um clique no programa? Como você suspeitava, as figuras de ícone não são modificadas na tela quando o usuário dá um clique nos ícones. Os ícones são substituídos por outros ícones, ou são *permutados,* pela procedure de evento MouseDown. Como mostra a figura a seguir, cada botão de comando gráfico tem três estados: para cima, para baixo e desativado.

Para cima é o estado normal, ou realçado a aparência do botão quando está em posição de descanso, ou padrão. Para baixo é o estado selecionado, ou pressionado a aparência do botão quando foi selecionado ou está ativo. Desativado é um estado opcional que é utilizado quando um botão não estiver disponível para uso no programa. Alguns botões de comando gráficos nunca utilizam esse estado.

Você pode atualizar os estados do botão utilizando a procedure de evento MouseDown

No programa Visual Basic, os estados do botão são controlados pela permuta de ícones com o objeto do tipo imagem utilizado para guardar o botão. A procedure de evento MouseDown associada ao objeto imagem cuida da permuta. Para que a procedure de evento funcione corretamente, ela deve ler o estado atual do botão (para cima ou para baixo), alterar o estado requisitado e, então, executar o comando solicitado (tal como alterar o texto para negrito). Os ícones de botão podem ser carregados durante o tempo de execução, utilizando a função *LoadPicture*, ou podem ser permutados no formulário, utilizando as instruções de atribuição. Alguns programadores colocam todos os três estados de botão no

formulário para que a atualização seja mais rápida. Se você tem a Professional Edition do Visual Basic, poderá encontrar uma grande coleção de botões úteis na pasta do Visual Basic no seu disco rígido.

Você pode criar botões e barras de ferramentas gráficos no formulário do Visual Basic, porém os formulários MDI possuem propriedades incorporadas especais que fazem com que trabalhar com as coleções de botões seja mais fácil

No exercício seguinte, você criará um programa que utiliza três botões de comando gráficos (Negrito, Itálico e Sublinhado) para formatar um texto no formulário. O programa utiliza seis ícones para exibir os botões no formulário e utiliza três procedures de evento MouseDown para atualizar os botões e formatar o texto. A próxima figura mostra o formulário que você construirá.

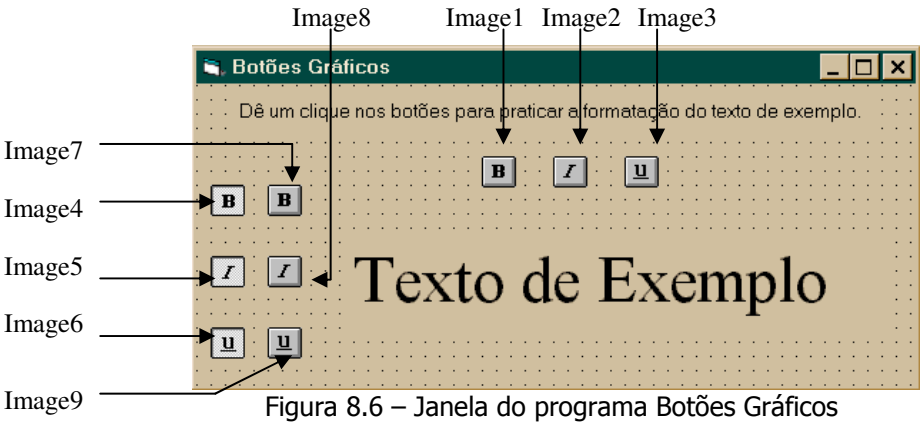


Figura 8.6 – Janela do programa Botões Gráficos

Abra um novo formulário, crie dois rótulo (labels), um na parte superior e o outro na parte inferior do formulário. Crie três pequenas caixas de imagem centralizado abaixo do rótulo superior. Essas caixas de imagem guardarão os botões de Negrito, Itálico e Sublinhado que você utilizará no programa. Crie seis caixas de imagem no lado esquerdo do formulário, procure manter a mesma ordem que apresentada na Figura 8.6.

Essas caixas de imagem guardarão seis estados de botão que você permutará entre as três primeiras caixas de imagem (alternativamente, você poderá carregar esses botões do disco, ou armazenar alguns dos estados em variáveis globais, porém, mantê-los todos no formulário é mais rápido e mais conveniente para este exemplo).

Agora configure as propriedades para os objetos no formulário. Primeiro, deixe invisíveis os seis ícones de permuta no formulário e, então, altere e formate o texto nos rótulos. Depois, grave os estados de abertura dos botões em uma propriedade de caixa de imagem especial chamada *Tag.* Tag é uma propriedade que você pode utilizar para incluir notas descritivas em um objeto quando estiver trabalhando com ele. Tag em geral é utilizada para armazenar o nome do objeto, mas, neste caso, você irá utilizá-la para armazenar os estados "Para Cima" e "Para Baixo" do botão.

A propriedade Tag permite que você coloque uma nota de identificação, ou identificador, em um objeto

Configure as seguintes propriedades para os objetos.

Quando você configurar a propriedade Picture, as caixas de imagem irão se ajustar ao tamanho do ícone da barra de ferramentas

Objeto	Propriedade	Configuração
Label1	Caption	"Dê um clique nos botões para praticar a flutuação do texto de exemplo."
Label2	Caption Font	"Texto de exemplo." Times New Roman, 28 pontos
Form1	Caption	"Botões Gráficos"
Image1	Picture Tag	"bld-up.bmp" "Para cima"
Image2	Picture Tag	"itl-up.bmp "Para cima"
Image3	Picture Tag	"ulin-up.bmp" "Para cima"
Image4	Picture Visible	"bld-dwn.bmp" False
Image5	Picture Visible	"itl-dwn.bmp" False
Image6	Picture Visible	"ulin-dwn.bmp” False
Image7	Picture Visible	"bld-up.bmp" False
Image8	Picture Visible	"itl-up.bmp" False
Image9	Picture Visible	"ulin-up.bmp" False

Quando terminar, o seu formulário estará parecido com o da Figura 8.6.

A procedure de evento MouseDown fornece quatro variáveis que você pode utilizar nos seus programas

Digite as seguintes instruções de programa na procedure de evento Image1_MouseDown().

```
If Imagel.Tag  = "Para Cima" Then
        Image1.Picture = Image4.Picture
        Label2.FontBold = True
        Imagel.Tag = "Para Baixo"
Else
        Image1.Picture = Image7.Picture
        Label2.FontBold = False
        Imagel.Tag = "Para Cima"
End If
```

Essa estrutura de decisão *If* simples processa dois tipos de cliques no botão Negrito (Bold) que o usuário poderá dar no programa. Se o botão Negrito já estiver no estado Para Cima, a procedure substitui o ícone *BId-up.bmp* pelo ícone *BId-dwn.bmp*, altera o texto para negrito e configura a tag da caixa de imagem para "Para Baixo". Se o botão já estiver no estado Para Baixo, a procedure substitui o ícone BId-dwn.bmp pelo ícone *BId-up.bmp*, cancela a configuração de negrito e configura a tag da caixa de imagem para "Para Cima". Qualquer que seja o estado do botão, a estrutura de decisão alterará para o estado oposto.

A propriedade FontBold controla o formato negrito

Abra a caixa de listagem drop-down Object na janela de código e, então, selecione o objeto Image2. Abra a caixa de listagem dropdown Proc e, então, selecione o evento MouseDown.

A procedure de evento Image2_MouseDown aparecerá.

A propriedade Tag é utilizada para determinar o estado atual do botão

Digite as seguintes instruções de programa no evento Image2_MouseDown :

```
If Image2.Tag = "Para Cima" Then
        Image2.Picture = Image5.Picture
        Label2.FontItalic = True
        Image2.Tag = "Para Baixo"
Else
        Image2.Picture = Image8.Picture
        Label2.FontItalic = False
        Image2.Tag = "Para Cima "
End If
```

Abra a caixa de listagem drop-down Object e, então, selecione o objeto Image3. Abra a caixa de listagem drop-down Proc e, então, selecione o evento MouseDown. Quando a procedure de evento Image3 MouseDown aparecer, digite as seguintes instruções de programa:

```
If Image3.Tag = "Para Cima" Then
        Image3.Picture = Image6.Picture
        Label2.FontUnderline = True
        Image3.Tag = "Para Baixo"
Else
        Image3.Picture = Image9.Picture
        Label2.FontUnderline = False
        Image3.Tag = "Para Cima"
End If
```

A propriedade Picture é utilizada para alterar entre os botões. Se você quiser aguardar que o botão seja liberado para depois formatar o formato, você poderá ainda manipular a permuta de ícone na procedure de evento Mousedown, porém poderá utilizar o evento MouseUp para alterar a fonte. Como programador, você tem o controle total sobre a maneira pela qual os botões são utilizados

8.7 Desenho com o Visual Basic

Existem dois tipos de objetos nos quais podemos desenhar no Visual Basic: formulários e quadros de figura. Já estamos acostumados com os formulários; no entanto, apenas utilizamos quadros de figura para carregar imagens. Apartir de agora vamos iniciar com um quadro de figura e alguns gráficos elementares que começam com o desenho de um único ponto.

8.8 Desenho de Pontos

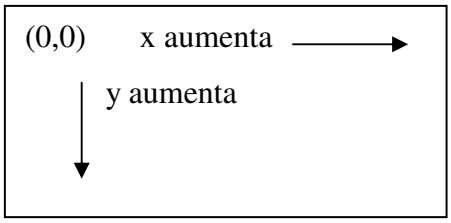
As funções gráficas do Visual Basic que realmente desenhar em formulários ou em quadros de figura são muito semelhantes às funções correspondentes no QuickBasic (última versão do Basic para o MS-DOS). Por exemplo, a primeira função gráfica que veremos aqui é PSet(), que marca um pixel na tela.

Para usar Pset(), inicie o Visual Basic e atribua a este projeto o nome, digamos, "Graphics", alterando a legenda do formulário original, Form1, para Graphics. Em seguida, encontre a propriedade AutoRedraw na janela de propriedades e defina-a para True. Essa propriedade indica que o Visual Basic deverá redesenhar os gráficos no formulário se parte dele ou todo for temporariamente encoberto por outra janela.

O sistema de coordenadas no Visual Basic usa t*wips* (1/1440 de polegada) por definição original. Você pode querer saber por que twips são a medida básica em vez de, digamos, pixeis da tela. A resposta é que o ambiente Visual Basic pode operar de uma maneira mais independente de dispositivo possível; ou

seja, poderemos desenhar na impressora tão fácil quanto na tela. No entanto, como as impressoras podem ter uma resolução muito mais alta do que as telas (as impressoras a laser normalmente possuem uma resolução de pelo menos 300 pontos por polegada), precisamos de um sistema de medida que seja ainda mais preciso do que o melhor que os nossos dispositivos podem gerar, motivo pelo qual o Visual Basic usa twips. Naturalmente, como programador, você pode estar acostumado a outras unidades, como pixeis, centímetros ou até mesmo pontos. (Um *ponto,* normalmente usado na medição de dimensões de uma fonte, é 1/72 de polegada, e um twip é 1/20 de um ponto.) Veremos mais tarde que é possível mudar a unidade de medida básica para pixeis, centímetros ou outras unidades que desejarmos.

O sistema de coordenadas começa em (0,0) em um formulário ou quadro de figura; x e y aumentam para a direita e para baixo, da seguinte forma:



Existe uma forma de trasferir o ponto (0,0) para a base esquerda de um formulário ou de um picture, é usar a seguinte instrução:

Objeto.Scale (Xinicial, Ymáximo) - (Xmáximo, Yinicial)

Vamos usar esse fato para desenhar nosso pixel, que podemos colocar no centro exato do formulário. Para isso, podemos obter o tamanho do formulário pela janela de propriedades. Por exemplo, poderíamos ter um formulário com 4.320 twips de altura x 7.200 twips de largura (isto é, 3 polegadas por 5 polegadas). Nesse caso, nosso comando PSet() poderia ser este:

Pset (4320 / 2, 7200 /2)

Esse comando coloca o pixel no centro do formulário a sintaxe de PSet() é:

PSet (x, y)

No entanto, existe outra maneira que podemos usar para centralizar o pixel sem saber o tamanho do formulário de antemão, usando as propriedades ScaleWidth e ScaleHeight.

Existem quatro propriedades embutidas que poderão lhe dizer as dimensões de um formulário ou quadro de figura: Height, Width, ScaleHeight e ScaleWidth. Pode parecer que as propriedades naturais a usar para determinar como centralizar um pixel sejam Width e Height, mas estas correspondem à largura e altura externas da janela (isto é, incluindo a barra de titulo, barra de menus etc. As dimensões reais da área do cliente (a área da qual temos controle) é ScaleWidth de largura e ScaleHeight de altura (em twips), como mostra a Figura 8.8. Por esse motivo, nosso comando PSet() poderia se parecer com este:

PSet (ScaleWidth / 2, ScaleHeight / 2)

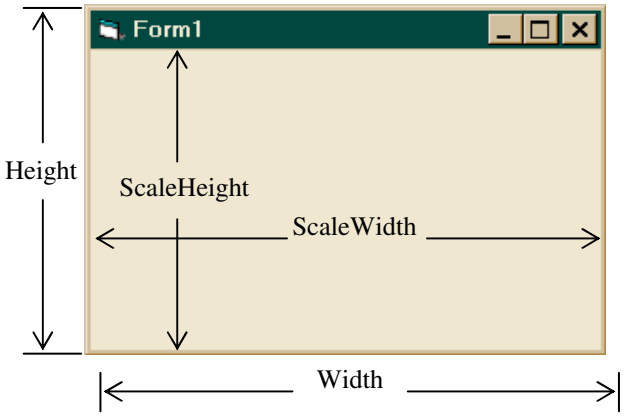


Figura 8.8 - Propriedades de altura e largura dos formulários.

Como queremos executar essa linha assim que o formulário aparece na tela, iremos colocá-la na procedure Sub Form_Load(), que é executada quando o formulário é inicialmente mostrado. Coloque nosso comando PSet() aqui:

```
Sub Form_Load()
PSet (ScaleWidth / 2, ScaleHeight / 2)
End Sub
```

Quando rodarmos o programa, o pixel aparecerá no centro do formulário. Observe que existem outros eventos que também poderíamos ter usado para mostrar esse pixel, como o evento Form_Click(), que significa que o ponto apareceria quando você desse um clique em qualquer lugar na área do cliente no formulário.

8.9 Seleção de Cores

Existem várias maneiras de definir nossas próprias cores no Visual Basic. Talvez a mais fácil seja usar os valores predefinidos. Cada valor de cor no Visual Basic é um inteiro longo, e muitos valores estão prontos para usarmos, como as constantes chamadas RED, BLUE e assim por diante, como mostra o Quadro 8.9.1. Além disso, podemos descobrir como o usuário definiu o sistema de cores (através do painel de controle do Windows) e moldar nossa aplicação para combinar usando as constantes de cor do sistema do Visual Basic. Por exemplo, você pode definir a propriedade de cor de fundo de um formulário (isto é, BackColor) para o padrão de todo o sistema definindo-a como sendo igual a Window_Background.

Quadro 8.9.1 Cores Predefinidas no Visual Basic

Constante	Valor	Descrição
VbBlack	0x0	Preto
VbRed	0xFF	Vermelho
VbGreen	0xFF00	Verde
VbYellow	0xFFFF	Amarelo
VbBlue	0xFF0000	Azul
VbMagenta	0xFF00FF	Magenta
VbCyan	0xFFFF00	Ciano
VbWhite	0xFFFFFF	Branco

Vamos fazer com que nosso programa imprima um ponto vermelho. A sintaxe geral de PSet() aparece na linha seguinte:

```
[object.]Pset [Step] (x!, y!)[,color&]
```

Aqui (x!, y!) representa o local do pixel com que queremos trabalhar. Temos que notar que essas variáveis são medidas com relação ao objeto (formulário ou quadro de figura) em que estamos desenhando. Em outras palavras, quando estamos definindo pixels em um quadro de figura, (x!, y!) representa o local do pixel em relação ao canto superior esquerdo do quadro (que é (0,0)). Além disso, o valor da cor, *color&,* é sempre um inteiro longo.

A palavra-chave *Step* é uma que será vista em muitos dos métodos de desenho, e indica que as coordenadas especificadas referem-se à posição ativa do gráfico, cujas coordenadas são armazenadas nas propriedades *CurrentX* e *CurrentY* dos formulários e quadros de figura. Como veremos, você pode desenhar linhas ou outras figuras gráficas usando a posição ativa do gráfico. Para definir essa posição, defina as propriedades *CurrentX* e *CurrentY*. Além disso, usar métodos gráficos como *PSet()* pode definir a posição ativa do gráfico. Quando você define um pixel com *PSet()* em algum objeto, a posição ativa do gráfico é definida para esse pixel. Entretanto, não usaremos *Step* aqui. Para tornar nosso pixel vermelho, podemos simplesmente usar a constante predefinida *vbRed*, da seguinte forma:

```
Sub Form_Load()
    PSet (ScaleWidth / 2, ScaleHeight / 2)
    Picture1.PSet(Picture1.ScaleWidth/2, Picturel.ScaleHeight/2), VbRed
End Sub
```

Podemos especificar a cor usando a função RGB. Essa função apanha três argumentos (um valor de cor vermelha, um valor de cor verde, e um valor de cor azul) da seguinte forma:

RGB (RedVal%, GreenVal%, BlueVal%)

Cada um desses valores de cor pode variar de um mínimo de 0 (isto é, quando a cor é inteiramente excluída) até um máximo de 255 (quando está mais forte). Para um ponto vermelho puro, podemos usar RedVal = 255, GreenVal = 0 e BlueVal = 0, da seguinte forma.

```
Sub Form_Load()
    PSet (ScaleWidth / 2, ScaleHeight / 2)
    Picture1.PSet(Picture1.ScaleWidth/2, Picturel.ScaleHeight/2), RGB(255,0,0)
End Sub
```

Existem ainda outra opção para definir cores. A última maneira de especificar cores no Visual Basic é com a função QBColor(). O QuickBasic possui 16 cores predefinidas, e você também poderá acessá-las no Visual Basic (facilitando a passagem de código que use gráficos do QuickBasic para o Visual Basic) por meio da função QBColor(). Essa função retorna um inteiro longo correspondente ao valor RGB do Visual Basic. Você usa QBColor() da seguinte forma: QBColor(nnn%), onde nnn% é o número de cor do QuickBasic, de 0 a 15, associado com a cor que você deseja, como mostra o Quadro 8.9.2.

Quadro 8.9.2 – Cores do QuickBasic para a função QBColor()

Número da Cor	**Cor**
0	Preto
1	Azul
2	Verde
3	Ciano
4	Vermelho
5	Magenta
6	Amarelo
7	Branco
8	Cinza
9	Azul-claro
10	Verde-claro
11	Ciano-claro
12	Vermelho-claro
13	Magenta-claro
14	Amarelo-claro
15	Branco intenso

Por exemplo, poderíamos passar nosso ponto para vermelho, além das outras maneiras, da seguinte forma:

```
Sub Form_Load()
    PSet (ScaleWidth / 2, ScaleHeight / 2)
    Picture1.PSet (Picture1.ScaleWidth / 2, Picture1.ScaleHeight / 2), QBColor(4)
End Sub
```

8.10 Desenho de Linhas

Uma nova ferramenta na caixa de ferramentas do Visual Basic lhe permite desenhar linhas em tempo de projeto: a ferramenta de controle *Line*. Usá-la é muito fácil; quando você dá um clique duplo, uma pequena linha aparece com duas alças de redimensionamento. Basta dimensionar a linha como deseja que apareça no formulário. Esse procedimento facilita a inclusão de sublinhados ou outros reforços no projeto de suas janelas. Aqui, no entanto, estamos interessados em desenhar linhas no código; para desenhar linhas no código, podemos usar o método *Line*, cuja sintaxe é:

[objeto.]Line [[Step](x1!, y1!)] – [Step](x2!, y2!)[,[color&],B[F]]]

Novamente, observe que este é um método, o que significa que você pode especificar em que objeto deseja desenhar as linhas. (Como sempre, se não especificar um objeto, o Visual Basic achará que você quer usar o formulário ativo.) E, como antes, podemos usar a palavra-chave Step aqui, se desejarmos. Para desenhar uma linha, naturalmente existem dois pontos envolvidos, o início e o final da linha, (x1!, y1!) e (x2!, y2!), respectivamente. Quando você usar Step na frente de um deles, isso indicará que o ponto deve ser considerado em relação à posição ativa do gráfico, conforme especificado por (CurrentX, CurrentY). Além disso, podemos especificar a cor da linha com o argumento color&. Os dois últimos argumentos, B e F são usados para desenhar retângulos (boxes), e nós os veremos na próxima seção.

Isso significa que se quisermos desenhar uma única linha diagonalmente pelo formulário, poderíamos fazer da seguinte forma, usando Line:

```
Line (0, 0) - (ScaleWidth, ScaleHeight)
```

E somente isso é necessário, os pontos inicial e final da linha (o Visual Basic desenha a linha de modo que o primeiro ponto, mas não o segundo, seja incluído). Por exemplo, vamos remover temporariamente o quadro de figura da nossa aplicação Graphics, e mudar a procedure Sub Form_Load() para esta:

```
Sub Form_Load()
    Line (0, 0)- (ScaleWidth, ScaleHeight)
End Sub
```

O formulário depois da execução do programa fica da seguinte forma:

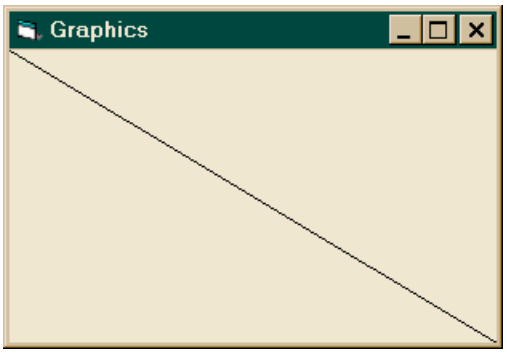


Figura 8.10 – Janela Form1 com uma linha diagonal

Podemos afetar a espessura da linha com a propriedade *DrawWidth*, poderíamos usar a seguinte instrução para mudar a espessura da linha:

```
DrawWidth = n (n um número inteiro)
```

Podemos também afeta o estilo de uma linha. Em outras palavras, podemos especificar se a linha é sólida ou composta de pontos e traços. O Quadro 8.10 apresenta os estilos fornecidos pelo Visual Basic.

Quadro 8.10 – Estilos de Linhas – Propriedade DrawStyle

DrawStyle	**Resulta em**
0	Linha sólida (estilo original de DrawStyle)
1	Linha tracejada
2	Linha pontilhada
3	Linha traço-ponto
4	Linha traço-ponto-ponto
5	Linha transparente (não aparece)
6	Linha interna

Uma linha interna (DrawStyle = 6) merece alguma explicação especial. Quando você desenha um boxe com uma linha grossa, a linha normalmente está centralizada na borda do boxe. O boxe termina ligeiramente maior do que deveria, pois a linha grossa está metade dentro e metade fora do boxe. Por outro lado, uma linha interna e desenhada de modo que fique inteiramente dentro do boxe, até mesmo para linhas grossas.

8.11 Desenho de Retângulos

Você viu que o controle Line lhe permite desenhar linhas em tempo de projeto. Outro controle gráfico, o controle Shape, permite desenhar uma série de formas, incluindo retângulos, elipses, quadrados, círculos, retângulos arredondados e quadrados arredondados. Você seleciona a forma desejada em tempo de projeto definindo a propriedade Shape do controle *Shape*. Como essas formas são controles, você pode definir suas propriedades *Top, Left, Width, Height*, *Bottom e Right* em um programa. Suponha que você desenhe um retângulo como Shape1; para duplicar a espessura desse retângulo quando o usuário clicar o formulário, você poderá executar o seguinte código:

```
Sub Form_Click()
    Shape1.Width = 2 * Shape1.Width
End Sub
```

Os controles Shape normalmente não são usados para produzir saída de programa; em vez disso, são usados para melhorar o projeto e o visual das suas Janelas (você pode dar destaque a um grupo de controles se agrupá-los com uma forma qualquer, por exemplo). Para produzir retângulos gráficos normais, você usa o método *Line* novamente. Pode parecer estranho que você só precise usar o método Line uma vez para desenhar um quadro, mais é exatamente isso que a opção B nos permite fazer:

```
[objeto.]Line [[Step](x1!, y1!) - Step(x2!, y2!)[,[color&],B[F]]]
```

Quando você especifica uma linha no Visual Basic, especifica os pontos finais. Semelhantemente, quando você especifica um retângulo, faz sentido que você só precise especificar dois pontos: os cantos superior esquerdo e inferior direito.

Como os dois padrões são unicamente especificados por dois pontos, o Visual Basic (bem como muitos outros tipos de Basic, incluindo o QuickBasic) permite que você desenhe retângulos e linhas com Line. Vejamos como isso funciona modificando nossa aplicação Graphics para desenhar alguns retângulos, da seguinte forma:

```
Sub Form_Load()
    DrawWidth = 8
    Line (0, 0) - (ScaleWidth / 2, ScaleHeight / 2), , B
    Line (ScaleWidth/4, ScaleHeight/4)-(3*ScaleWidth/4, 3* ScaleHeight / 4), , B
    Line(ScaleWidth/2, ScaleHeight/2)-(ScaleWidth, ScaleHeight),, B
End Sub
```

Isso gera a janela mostrada na Figura 8.11.1. As linhas grossas nas bordas estão metade dentro e metade fora da área do cliente. Isso pode ser consertado usando o estilo de desenho Inside Line, da seguinte forma:

```
Sub Form_Load()
    DrawWidth = 8
    DrawStyle =6
    Line (0, 0) - (ScaleWidth / 2, ScaleHeight / 2), , B
    Line (ScaleWidth/4, ScaleHeight/4)-(3*ScaleWidth/4, 3* ScaleHeight / 4), , B
    Line(ScaleWidth/2, ScaleHeight/2)-(ScaleWidth, ScaleHeight),, B
End Sub
```

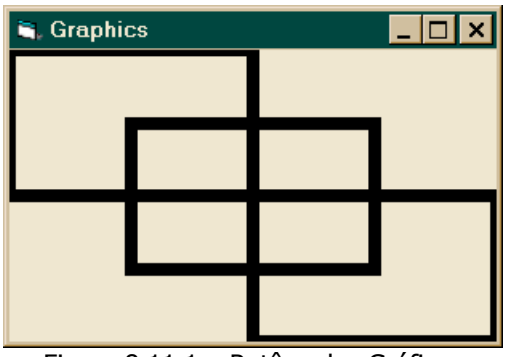


Figura 8.11.1 – Retângulos Gráficos

Agora os retângulos são desenhados de modo que a borda espessa esteja dentro dos limites, como mostra a Figura 8.11.2. É assim que funcionam as linhas internas. Elas não superpõem os limites da figura. Se quisermos, também poderemos trocar o modo de desenho (isto é, usar um valor de caneta diferente); por exemplo, podemos usar uma Invert Pen, que inverte o que estiver abaixo do desenho (DrawMode = 6)

```
Sub Form_Load()
    DrawWidth = 8
    DrawStyle = 6
    DrawMode = 6
    Line (0, 0) - (ScaleWidth / 2, ScaleHeight / 2), , B
    Line (ScaleWidth/4, ScaleHeight/4)-(3*ScaleWidth/4, 3* ScaleHeight / 4), , B
    Line(ScaleWidth/2, ScaleHeight/2)-(ScaleWidth, ScaleHeight),, B
End Sub
```

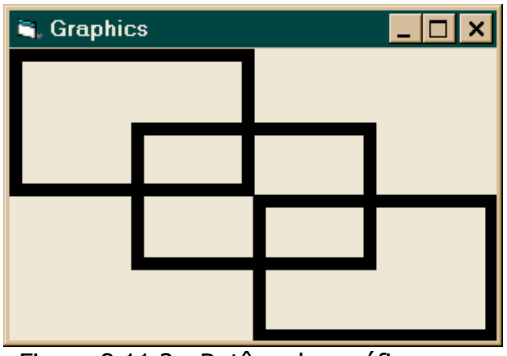


Figura 8.11.2 - Retângulos gráficos com Inside Lines.

Isso produz o resultado da Figura 8.11.3. Em geral, podemos produzir todos os tipos de retângulos desse lado, tão fácil quanto podemos desenhar linhas. De fato, também podemos preencher os retângulos, se quisermos, e veremos isso em seguida.

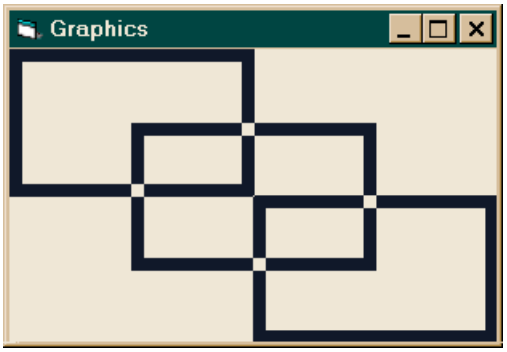


Figura 8.11.3 - Retângulos.

8.12 Preenchendo Figuras com Padrões

Desenhando boxes, desenhamos figuras gráficas fechadas pela primeira vez, mas o Visual Basic nos permite ir além disso. Podemos especificar um padrão de preenchimento de modo que nossos boxes sejam preenchidos automaticamente. De fato, nossos boxes já foram automaticamente preenchidos, mas o padrão de preenchimento original é transparente. No entanto, isso e apenas um dos oito padrões de preenchimento. Os outros incluem linhas horizontais ou verticais, linhas diagonais ou, é claro, o preenchimento de figuras com uma cor sólida. Essas opções aparecem na Quadro 8.12

Quadro 8.12 – Opções de Preenchimento – Propriedade FillStyle

Valor de FillStale	Padrão Resultante
0	Sólido
1	Transparente (definição original)
2	Linhas horizontais
3	Linhas verticais
4	Diagonais para cima
5	Diagonais para baixo
6	Hachuras
7	Hachuras diagonais

Você seleciona um dos padrões de preenchimento do Quadro 8.12 com a propriedade FillStyle. Vejamos isso em ação. Podemos desenhar oito retângulos no formulário principal da aplicação Graphics, e depois preenchê-los, um por um, com os padrões diferentes, correspondentes a diferentes valores da propriedade FillStyle. Novamente, vamos usar a procedure Sub Form_Load() para ver isso funcionando. Podemos mudar essa procedure para desenhar duas fileiras de quatro retângulos e preencher cada um com um padrão de preenchimento diferente:

```
Sub Form_Load()
    SX = ScaleWidth
    SY = ScaleHeight
    For i= 0 To 3
        FillStyle = i
        Line ((2*i+1)*SX/9, SY/5)-((2*i+2)*SX/9, 2*SY/5), ,B
        FillIStyle = i + 4
        Line ((2*i+1)* SX/9, 3*SY/5) - ((2*i+2)*SX/9, 4*SY/5), ,B
    Next i
End Sub
```

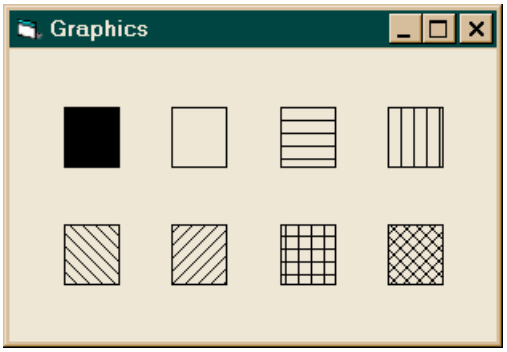


Figura 8.12 – As Diferentes opções de FillStyle

8.13 Desenho de Círculos

Desenhar círculos e muito fácil com o método Circle:

[objeto] .Circle [Step] (x!, y!), raio! [,[color&][,[start!][, [end!] [,[aspect!]]]]

Aqui, (x!, y!) representa o centro do circulo, e raio! o seu raio (tudo em twips). Além disso, você pode desenhar arcos especificando ângulos (em radianos) inicial e final para o desenho. Vejamos como isso funciona desenhando alguns círculos. Podemos começar com um estilo de preenchimento de diagonais descendo e um circulo vermelho simples, assim:

```
Sub Form_Load()
    FillStyle = 5   'Diagonais descendo
    ForeColor = RGB(255, 0, 0) 'Vermelho
    Circle (ScaleWidth/4, ScaleHeight/4), ScaleHeight/5
End Sub
```

Isso produz um círculo vermelho preenchido com diagonais descendo, mas o padrão de preenchimento é preto, e não vermelho. Se estivéssemos desenhando boxes, poderíamos consertar isso com a opção F, mas não existe uma opção F aqui; em vez disso, podemos definir a propriedade FillColor para combinar com a ForeColor ativa, da seguinte forma:

```
Sub Form_Load()
    FillStyle = 5           'Diagonais descendo
    ForeColor = RGB(255, 0, 0)       'Vermelho
    FillColor = ForeColor
    Circle (ScaleWidth/4, ScaleHeight/4), ScaleHeight/5
End Sub
```

Agora, o círculo e o padrão de preenchimento dentro dele são vermelhos. No entanto, o método Circle é ainda mais poderoso; também podemos desenhar elipses e arcos com ele. Para desenhar elipses, usamos o argumento aspect com Circle. A *razão de aspecto* indica a razão vertical por horizontal para elipses. Quando estamos desenhando um círculo, a razão é definida para o valor básico de 1; no entanto, podemos desenhar uma elipse com a mesma facilidade. Por exemplo, para desenhar um circulo e depois uma elipse com a altura sendo o dobro da largura, podemos usar uma razão de aspecto igual a dois:

```
Sub Form_Load()
    Circle (ScaleWidth/4, ScaleHeight/4), ScaleHeight/5
    FillStyle = 2
    Circle (ScaleWidth/2, ScaleHeight/2), ScaleHeight/3,,,,2 'Elipse
End Sub
```

A tela gráfica resultante aparece na Figura 8.13.1. Podemos também desenhar arcos ou seja, círculos parciais especificando os ângulos start! [inicial] e end! [final]. Esses ângulos são medidos em radianos, que vão de 0 até dois pi (2π). Por exemplo, vamos incluir um arco que vai de 0 (isto é, da posição de três horas) até pi (isto é, em sentido anti-horário até a posição das nove horas), da seguinte forma:

```
Sub Form_Load()
    Circle (ScaleWidth/4, ScaleHeight/4), ScaleHeight/5
    FillStyle = 2
    Circle (ScaleWidth/2, ScaleHeight/2), ScaleHeight/3,,,,2 'Elipse
    Circle (3*ScaleWidth/4, 3*ScaleHeight/4), ScaleHeight/5,, 0,3.141     'Arco
End Sub
```

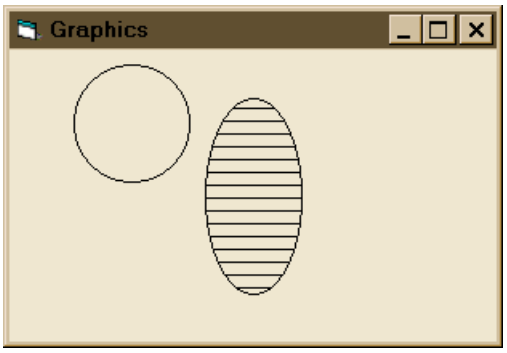


Figura 8.13.1 – Janela onde se projetou um círculo e uma elipse

As figuras resultantes aparecem na Figura 8.13.2. Embora especifiquemos um padrão de preenchimento, o arco não é preenchido, pois não é uma figura fechada.
E isso encerra os principais métodos de desenho embutidos, PSet, Line e Circle. Como vimos, é possível produzir alguns efeitos interessantes com eles.

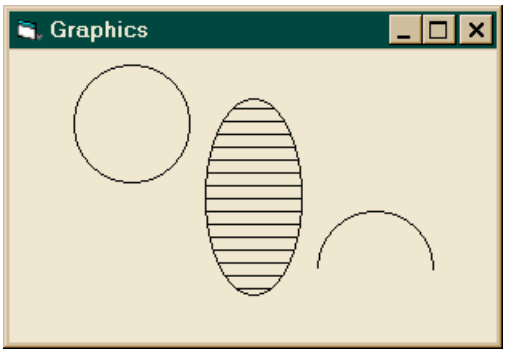


Figura 8.13.2 – Círculo, elipse e arco

9.0 TRABALHANDO COM ARQUIVO TEXTO E BANCOS DE DADOS

Nesta lição você aprenderá a trabalhar com informações armazenadas em arquivos de texto e em bancos de dados no seu sistema. Você aprenderá a abrir um arquivo de texto e exibir o seu conteúdo utilizando um objeto caixa de texto, e aprenderá a criar um novo arquivo de texto no disco. Você também aprenderá a abrir bancos de dados existentes utilizando um objeto dados, a pesquisar itens específicos e a acrescentar e excluir registros de bancos de dados. O Microsoft Visual Basic foi especialmente projetado para criar interfaces personalizadas, ou *front-ends (auxiliares),* para os bancos de dados existentes, assim, se você quiser personalizar ou dispor os dados que acabou de criar com outro aplicativo, tal como o Microsoft Access, você pode começar imediatamente.

9.1 Exibindo Arquivos de Texto Utilizando um Objeto Caixa de Texto

A maneira mais simples de exibir um arquivo de texto no programa é utilizar um objeto caixa de texto. Você pode criar objetos caixa de texto em uma variedade de tamanhos e, se o conteúdo do arquivo

de texto não couber na caixa de texto, você pode acrescentar barras de rolagem na caixa de texto, de maneira que o usuário possa examinar o arquivo inteiro. Para carregar o conteúdo do arquivo de texto na caixa de texto, você precisa utilizar três instruções e uma função. Essas palavras-chave estão descritas na tabela a seguir e serão demonstradas no primeiro exercício desta lição.

Você pode utilizar quatro palavras-chave do Visual Basic com os arquivos de texto

Palavra-chave	Descrição
Open	Abre um arquivo de texto para entrada e saída de dados
Line Input	Lê uma linha de inserção do arquivo de texto
EOF(n)	Verifica se é final de arquivo
LOF(n)	Verifica o tamanho do arquivo, em bytes
Close	Fecha o arquivo de texto

9.2 Abrindo um Arquivo de Texto para Entrada de Dados

Um arquivo de texto consiste de uma ou mais linhas de números, palavras ou caracteres, eles são distintos dos arquivos de documentos, os quais contêm códigos de formatação, e dos arquivos executáveis, os quais contêm instruções para o sistema operacional. Os arquivos de texto típicos no seu sistema serão identificados pelo Explorer do Windows como "Documento de Texto, ou terão a extensão .txt, .ini, .dat ou .bat**.** Uma vez que os arquivos de texto contêm somente caracteres comuns e reconhecíveis, você pode facilmente exibi-los utilizando objetos caixa de texto.

Os arquivos de texto contêm caracteres reconhecíveis

Você pode permitir que o usuário decida qual arquivo de texto deve abrir no programa, utilizando um objeto de diálogo comum que solicite ao usuário o nome de caminho do arquivo. Os objetos de diálogo comuns suportam o método ShowOpen, que exibe a caixa de diálogo comum Abrir na tela. O nome de caminho que o usuário selecionar na caixa de diálogo é retornado ao programa na propriedade filename e você pode utilizar esse nome para abrir o arquivo. O objeto de diálogo comum não abre o arquivo, apenas fornece o nome do caminho.

Um objeto caixa de diálogo exibe a caixa de diálogo comum Abrir

9.3 A Instrução Open

Após obter o nome de caminho do usuário, abra o arquivo no programa utilizando a instrução Open. A sintaxe da instrução Open é:

Open *caminho* For *modo As #NúmeroDoArquivo*

em que, *caminho* é um nome de caminho válido do Microsoft Windows, *modo* é a palavra-chave que indica como o arquivo será utilizado (você utilizará os modos Input e Output nesta lição, existindo ainda o comando Append - que insere dados após o último dado do arquivo) e *NúmeroDoArquivo* é um número inteiro de 1 até 255. O número do arquivo estará associado ao arquivo quando for aberto. Você, então, pode utilizar esse número de arquivo no seu código, quando precisar se referir ao arquivo aberto. Não há nada de especial sobre os números de arquivo. O Visual Basic apenas os utiliza para não perder de vista os diferentes arquivos que você abrir no seu programa.
Uma instrução Open típica que utiliza um objeto de diálogo comum é parecida com:

```
Open CommonDialog1.filename For Input As #1
```

Aqui a propriedade CommonDialog1.filename representa o caminho, Input é o modo, e 1 é o número do arquivo.

Os arquivos de texto que são abertos utilizando-se essa sintaxe são chamados arquivos seqüenciais, pois os seus conteúdos devem ser trabalhados em uma ordem seqüencial. Por outro lado, você pode acessar as informações no arquivo do banco de dados em qualquer ordem

O exercício a seguir demonstra como você pode utilizar um objeto de diálogo comum e a instrução Open para abrir um arquivo de texto. O exercício também demonstra como você pode utilizar as palavras-chave Line Input e EOF para exibir o conteúdo de um arquivo de texto em uma caixa de texto e como você pode utilizar a palavra-chave Close para fechar um arquivo.

Insira em um formulário um Label, uma caixa de texto e uma caixa de diálogo comum. Crie um menu, de nome Arquivo, com as opções Abrir, Fechar e Sair, e suas rotinas são apresentadas abaixo:

```
Private Sub mnuItemFechar_Click()
    txtArquivo.Text = ""        'limpa a caixa de texto
    lblArquivo.Caption = "Carregar um arquivo de texto com o comando Abrir."
    mnuItemFechar.Enabled = False 'atenua o comando Fechar
    mnuItemAbrir.Enabled = True   'ativa o comando Abrir
    txtArquivo.Enabled = False     'desativa a caixa de texto
End Sub

Private Sub mnuItemAbrir_Click()
    Wrap$ = Chr$(13) + Chr$(10)  'cria caracter de mudança de linha
    CommonDialog1.Filter = "Arquivos texto (*.TXT)|*.TXT"
    CommonDialog1.ShowOpen    'exibe a caixa de diálogo Abrir
    If CommonDialog1.filename <> "" Then
            Form1.MousePointer = 11  'exibe a ampulheta
            Open CommonDialog1.filename For Input As #1
            On Error GoTo TooBig: 'cofingura o manipulador de erro
            Do Until EOF(1)       'então lê linhas do arquivo
                    Line Input #1, LineOfText$
                    AllText$ = AllText$ & LineOfText$ & Wrap$
            Loop
            lblArquivo.Caption = CommonDialog1.filename
            txtArquivo.Text = AllText$  'exibe o arquivo
            txtArquivo.Enabled = True
            mnuItemFechar.Enabled = True
            mnuItemAbrir.Enabled = False 'permite rolagem
CleanUp:
            Form1.MousePointer = 0   'inicializa o mouse
            Close #1              'fecha o arquivo
    End If
    Exit Sub
TooBig:         'manipulador de erro exibe mensagem
    MsgBox ("O arquivo especificado é muito grande.")
    Resume CleanUp: 'então pula para a rotina de inicialização
End Sub


Private Sub mnuItemSair_Click()
    End    'sai do programa
End Sub
```

O programa depois de executado ficará da seguinte forma:

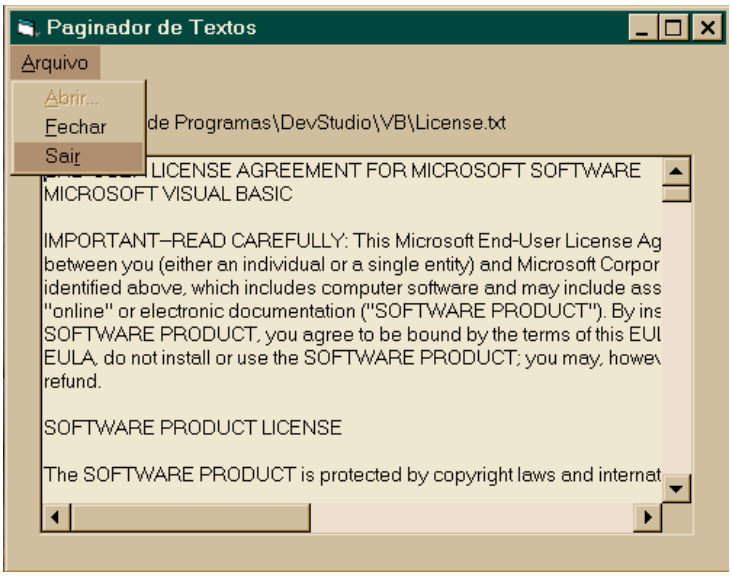


Figura 9.3 - Janela do programa Paginador de Texto

9.4 Criando um Novo Arquivo de Texto em Disco

Para criar um novo arquivo de texto em disco utilizando o Visual Basic, você usará muitos objetos e palavras-chave que utilizou no último exemplo. Criar novos arquivos em disco e salvar os dados neles será útil se você planeja gerar relatórios personalizados ou arquivos de transações, salvar cálculos ou valores importantes, ou criar um processador ou editor de textos para uma finalidade especial. Veja agora alguns passos que você precisará seguir no programa:

1. Obtenha dados do usuário, ou execute um cálculo matemático, ou ambos;
2. Atribua os resultados do seu processador para uma ou mais variáveis;

Você utiliza as palavras-chaves For Output na instrução Open quando quiser criar um novo arquivo. Caso você queira adicionar dados a um arquivo, sem perder o conteúdo do mesmo use as palavras-chaves For Append

3. Solicite ao usuário um caminho utilizando a caixa de diálogo comum Salvar Como. Utilize o método ShowSave para exibir a caixa de diálogo;
4. Utilize o caminho recebido na caixa de diálogo para abrir o arquivo para saída de dados;
5. Utilize a instrução Print# para salvar os dados no arquivo aberto.

A instrução Print# envia a saída de dados para o arquivo especificado

6. Feche o arquivo quando terminar, com a instrução Close #.

Faça como no exemplo anterior, substituindo os menus existentes pelos menus mostrados na Figura 9.4. As rotinas ficam dispostas da seguinte forma:

```
Private Sub mnuItemData_Click()
   Wrap$ = Chr$(13) & Chr$(10) 'adiciona data ao string
   txtNota.Text = Date$ & Wrap$ & txtNota.Text
End Sub

Private Sub mnuItemSair_Click()
   End            'sai do programa
End Sub

Private Sub mnuItemSalvar_Click()
'nota: o arquivo inteiro é armazenado em uma string
   CommonDialog1.Filter = "Text files (*.TXT)|*.TXT"
   CommonDialog1.ShowSave     'exibe o diálogo salvar
   If CommonDialog1.filename <> "" Then
```

```
    Open CommonDialog1.filename For Output As #1
    Print #1, txtNota.Text  'salva a string em um arquivo
    Close #1            'fecha o arquivo
  End If
End Sub
```

O programa depois de executado fica da seguinte forma:

Figura 9.4 - Janela do programa Nota Rápida

9.5 Gerenciando Bancos de Dados

Um banco de dados é uma coleção organizada de informações armazenada eletronicamente em um arquivo. Você pode criar bancos de dados poderosos utilizando uma variedade de produtos de banco de dados, incluindo o Microsoft Access, Microsoft FoxPro, Btrieve, Paradox e dBASE. É possível utilizar os bancos de dados cliente-servidor Open Database Connectivity (ODBC, ou Conectividade Aberta de Banco de Dados), como o Microsoft SQL Server. Se você trabalha regularmente com os bancos de dados especialmente com os bancos de dados relacionados acima, poderá utilizar o Visual Basic como uma ferramenta poderosa para otimizar e exibir os seus dados. Uma vez que o Visual Basic implementa a mesma tecnologia de banco de dados incluída no Microsoft Access (um mecanismo de banco de dados intitulado Microsoft Jet), você pode criar aplicativos básicos de banco de dados personalizados com apenas algumas linhas de código de programa.

O Visual Basic pode ler e gravar uma variedade de formatos de banco de dados

Nesta seção, você aprenderá a utilizar um objeto dados do Visual Basic para gerenciar o banco de dados Biblio.mdb que está incluído no software Visual Basic e dentro do Access. O Biblio.mdb é um banco de dados do Access que contém uma bibliografia dos livros sobre programação de banco de dados. Você aprenderá a exibir vários registros no banco de dados, pesquisar uma informação específica no banco de dados, acrescentar registros, excluir registros indesejados e fazer cópia de segurança de arquivos. Quando terminar, você estará apto a colocar essas habilidades em prática nos seus próprios projetos de banco de dados.

9.6 Criando Aplicativos de Banco de Dados Personalizados

Um aplicativo de banco de dados personalizado é um programa que pega os campos e os registros de um banco de dados e os exibe de uma maneira que seja significativa para um grupo específico de usuários. Por exemplo, uma biblioteca pública pode criar uma versão personalizada do seu catálogo de cartões para um grupo de pesquisadores científicos. Os aplicativos de banco de dados personalizados apresentam, em geral, uma variedade de comandos aos seus usuários. Tais comandos permitem que os usuários visualizem filtros, pesquisem, imprimam, acrescentem e excluam registros, e façam cópias de segurança do banco de dados. Devido às peculiaridades de seu design, ou devido à sua evolução posterior, alguns bancos de dados são organizados de uma maneira que se tornam difíceis de usar na sua forma original, ou no ambiente do banco de dados. Com o Visual Basic, você pode construir um aplicativo de

banco de dados personalizado que exiba exatamente a informação que os usuários desejam, e que forneça exatamente os comandos de que eles precisam para processar os dados.

Aplicativos de banco de dados personalizados apresentam listas personalizadas do campos e registros do banco de dados

9.7 Utilizando Controles de Limite para Exibir Informação de Banco de Dados

A maioria dos objetos que você cria utilizando os controles da caixa de ferramentas do Visual Basic possui uma habilidade incorporada para exibir informações de banco de dados. Na terminologia do banco de dados, esses objetos são chamados *controles de limite.* Um objeto é o limite para um banco de dados quando a sua propriedade *DataSource* estiver configurada para um nome de banco de dados válido e sua propriedade *DataFiel* estiver configurada para uma tabela válida no banco de dados. Uma *tabela* é um grupo de campos e registros que você ou alguém define quando o banco de dados é criado. Você pode ligar o seu programa Visual Basic a um banco de dados utilizando um objeto dados. Após a conexão ter sido estabelecida, você pode exibir a informação do banco de dados utilizando objetos criados com qualquer um dos controles padrões.

Os controles de limites processam automaticamente as informações do banco de dado

Os controles DBCombo e DBList são semelhantes aos controles Combo e List, mas contêm propriedades adicionais que permitem que eles manipulem várias tabelas de dados de banco de dados. Para mais informação sobre esses controles avançados, procure por DBCombo e DBList na Ajuda OnLine do Visual Basic

9.8 Utilizando Objetos Caixa de Texto para Exibir Dados

O programa a seguir utiliza um objeto dados e quatro objetos caixa de texto para exibir quatro campos do banco de dados Biblio.mdb. Ele demonstra como você pode criar um aplicativo de banco de dados personalizado para visualizar somente a informação que você deseja. Nesse aplicativo, a propriedade *ReadOnly* do objeto dados é configurada como *True*, de maneira que a informação no banco de dados possa ser visualizada, mas não alterada. Para permitir que os usuários façam algumas alterações no banco de dados Biblio.mdb, configure a propriedade *ReadOnly* como *False* utilizando a janela Properties.

Para manter inalterado o banco de dados Biblio.mdb original, utilize o Explorer do Windows para fazer uma cópia de segurança antes de completar os exercícios seguintes. Se a sua cópia do Biblio.mdb não estiver na subpasta de instalação do Visual Basic, atualize a propriedade DataBaseName do objeto dados com o nome de caminho correto no seu programa

O programa depois de executado fica disposto como apresentado na Figura abaixo:

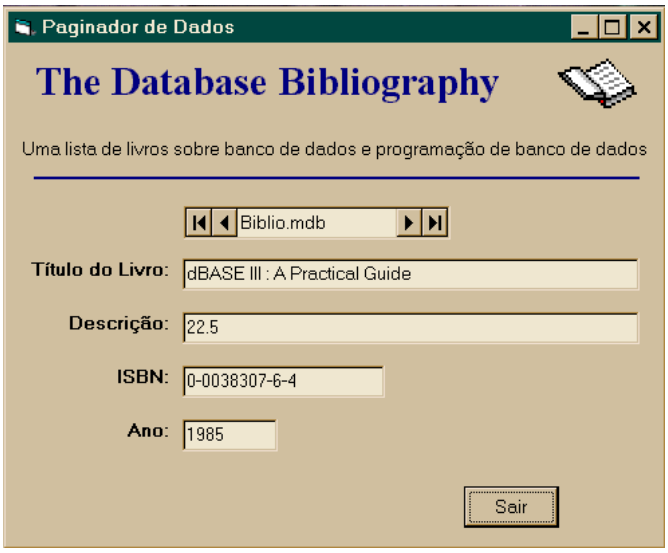


Figura 9.8 - Janela do programa Paginador de Dados

Este programa contém somente uma linha de código de programa (a instrução End), mas ainda assim nos fornece alguma informação. Um ponto a favor é que exibe apenas os campos que você quer ver no banco de dados Biblio.mdb. Utilizando um objeto dados e várias caixas de texto com limite de dados, você pode criar um aplicativo de banco de dados efetivo.

9.9 Utilizando um Objeto Recordset

No programa Paginador de Dados, você utilizou uma propriedade de dados intitulada *RecordsetType* para identificar a informação do seu banco de dados como uma tabela. No Visual Basic, *Recordset* é um objeto que representa a parte do banco de dados com que você está trabalhando no programa. O objeto *Recordset* inclui propriedades especiais e métodos que permitem que você pesquise, ordene, acrescente e exclua registros. No exercício a seguir, utilizaremos o objeto *Recordset* para pesquisar os títulos de livro no banco de dados Biblio.mdb.

O objeto Recordset representa os dados com que você está trabalhando no programa

Ative o código do botão de comando Localizar para abrir a procedure de evento cmdLocalizar_Click. Em seguida digite as seguintes instruções de programa na procedure de evento.

```
prompt$ = "Digite o título completo do livro."
'Obtém a string a ser utilizada na pesquisa do campo Title
SearchStr$ = InputBox(prompt$, "Pesquisa de Livro")
datBiblio.Recordset.Index = "Title"  'utiliza a tabela Title
datBiblio.Recordset.Seek "=", SearchStr$ 'e pesquisa
If datBiblio.Recordset.NoMatch Then     'se não achar
        MsgBox ("Desculpe, não pude achar o livro.")
        datBiblio.Recordset.MoveFirst 'vá para o primeiro 'registro
End If
```

O método Seek pesquisa um registro correspondente

Essa procedure de evento exibe uma caixa de diálogo de pesquisa para obter uma string de pesquisa do usuário (SearchStr$), e utiliza o método *Seek* para pesquisar o campo Title do banco de dados do início ao fim até que encontre um correspondente ou chegue ao final da lista. Se o campo correspondente não for encontrado, uma mensagem é exibida e o primeiro registro no *Recordset* aparece. As propriedades *Recordset* e os métodos a seguir são utilizados na procedure de evento:

Propriedade do Método Recordset	Descrição
Index	Uma propriedade utilizada para definir o campo do banco de dados que será utilizado na pesquisa.
Seek	Um método utilizado para pesquisar o registro. Além do "=", os operadores relacionais > =, >, < = e < podem ser utilizados para comparar a string de pesquisa com o texto no banco de dados.
NoMatch	Uma propriedade configurada como True, se não for encontrado o correspondente na pesquisa.
MoveFirst	Um método que torna o primeiro registro no Recordset o registro atual.

A Figura 9.9.1 apresenta a janela do programa Paginador de Dados, com a inserção do botão Localizar. Caso o usuário digite um nome não existente o programa retorna a caixa de diálogo apresentada na Figura 9.9.2.

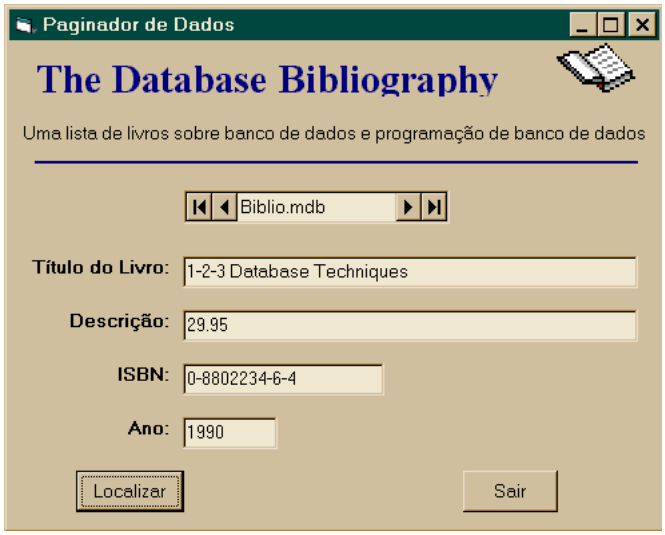


Figura 9.9.1 - Janela do programa Paginador de Dados com a opção Localizar

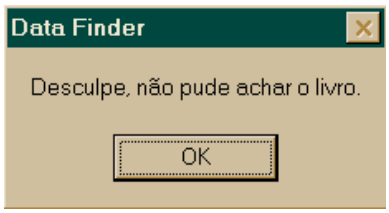


Figura 9.9.2 - Caixa de diálogo, mostrada se o nome do livro não existe no banco de dados

9.10 Acrescentando registros ao Banco de Dados Biblio.mdb

Para acrescentar um novo registro ao banco de dados, configure a propriedade *ReadOnly* do objeto dados como False no modo projeto e, então, utilize o método *AddNew* na procedure de evento para abrir um novo registro no banco de dados. Quando aparecer um registro em branco no formulário, o usuário preencherá os campos necessários e, quando terminar, irá para um registro diferente no banco de dados. A maneira mais fácil do usuário ir para um registro diferente é dar um clique em um dos botões no objeto dados. Quando o usuário passar para um registro diferente, o novo registro será inserido no banco de dados em ordem alfabética.

O exercício a seguir demonstra como a propriedade *ReadOnly* e o método *AddNew* podem ser utilizados para inserir novos registros no banco de dados. A função *InputBox* oferece ao usuário algum "feedback" visual durante o processo. Insira um novo botão de comando com o rótulo "Acrescentar" e digite em seu código a rotina abaixo:

```
prompt$ = "Insira um novo registro e dê um clique na seta esquerda."
reply = MsgBox(prompt$, vbOKCancel, "Acrescentar Registro")
If reply = vbOK Then      'se o usuário dert um clque em OK
    txtTítulo.SetFocus    'mover o cursor para a caixa Título
    datBiblio.Recordset.AddNew   'e obter novo registro
    'definir campo PubID como 14 (este campo é necessário para Biblio.mdb)
    datBiblio.Recordset.PubID = 14  'by biblio.mdb)
End If
```

O método AddNew acrescenta um novo registro ao banco de dados

A procedure exibe, primeiro, uma caixa de diálogo que contém instruções para o usuário sobre entrada de dados. A função MsgBox utiliza o argumento vbOKCancel (uma constante numérica definida pelo Visual Basic) para exibir uma caixa de diálogo que tenha os botões OK e Cancelar. Se o usuário der um clique em OK, um novo registro será criado por meio do método *AddNew*. Se o usuário der um clique em Cancelar, a operação é ignorada. A procedure de evento também utiliza o método *SetFocus* para colocar o cursor na caixa de texto Title. O método *SetFocus* pode ser utilizado para ativar qualquer objeto que possa receber o foco.

A última instrução na estrutura de decisão If utiliza o código de programa para definir o campo PubID no banco de dados Biblio.mdb. Cada novo registro exige esse campo, e devido ao fato do "browser" do seu banco de dados não contê-lo, você deve inserir PubID manualmente. O valor 14 significa que o livro será publicado pela Microsoft Press.

9.11 Excluindo Registros do Banco de Dados Biblio.mdb

Para excluir um registro do banco de dados, exiba o registro que quer excluir e, então, utilize o método *Delete* - com o objeto RecordSet para remover o registro.

Antes de abrir o banco de dados no programa, você deve configurar a propriedade *ReadOnly* do objeto dados como False. (Você já fez isso antes de utilizar o método *AddNew*.) Após excluir o registro, você deverá exibir um novo registro no banco de dados, pois o objeto dados não o faz automaticamente. Geralmente, a melhor técnica é utilizar o método *MoveFirst (ou MoveNext)* para exibir o primeiro registro no banco de dados.

O exercício a seguir mostra como você pode utilizar o Visual Basic para excluir registros do banco de dados Biblio.mdb. Preste bem atenção na utilização da função MsgBox no programa. Uma vez que o objeto dados não oferece o recurso "undo" (voltar à ação anterior), é importante que o programa verifique a intenção do usuário antes de excluir permanentemente um registro do banco de dados.

O método Delete exclui permanentemente o registro do banco de dados. Não dê acesso a este método para os usuários, a menos que você queira que eles excluem registros

Insira um novo botão de comando e em seu código de evento clique digite a seguinte instrução:

```
prompt$ = "Você deseja mesmo excluir este registro?"
reply = MsgBox(prompt$, vbOKCancel, "Excluir registro")
If reply = vbOK Then    'se o usuário der um clique em OK
        datBiblio.Recordset.Delete   'excluir registro atual
        datBiblio.Recordset.MoveNext  'mover para o próximo registro
End If
```

Essa procedure exibe primeiro uma caixa de diálogo que pergunta ao usuário se ele quer excluir o registro atual. Novamente, a função *MsgBox* é utilizada com o argumento vbOKCancel para permitir que o usuário desista da operação de excluir, caso não queira ir até o fim. Se o usuário der um clique em OK, o registro atual será excluído por meio do método *Delete* e o próximo registro será exibido por meio do método *MoveNext*. Se o usuário der um clique em Cancelar, a operação excluir não será realizada. O programa na sua forma final fica da seguinte maneira:

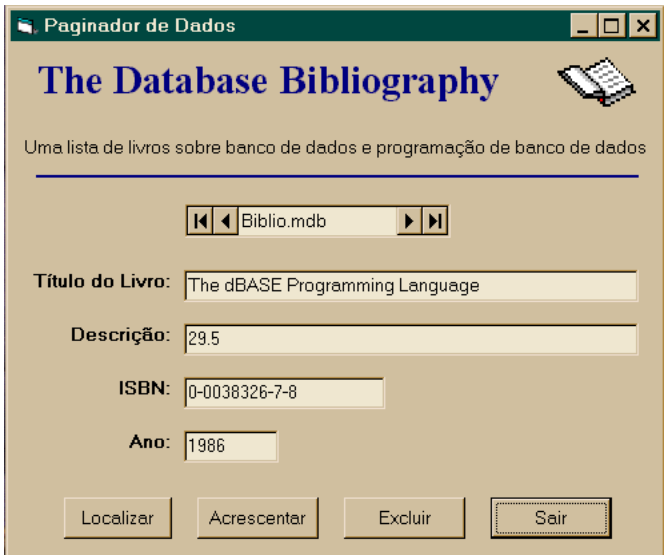


Figura 9.11 - Janela final do programa Paginador de Dados

9.12 Fazendo uma Cópia de Segurança de um Arquivo

Se você é como a maioria das pessoas, as informações que mantém no banco de dados são muito importantes e você estaria encrencado se algo acontecesse com ele. Por essa razão, você deve sempre fazer uma cópia de segurança de cada banco de dados que utiliza, antes de fazer qualquer alteração nele. Se algo der errado quando estiver trabalhando com a cópia, você simplesmente poderá substituir a cópia pelo banco de dados original. A rotina da sua cópia de segurança pode incluir também um programa de cópia de segurança, o Explorer do Windows ou um recurso especial para fazer uma cópia de segurança no seu programa de banco de dados. Como uma segurança adicional, você pode criar uma cópia de segurança de um ou mais arquivos de dentro do programa Visual Basic utilizando a instrução *FileCopy*. *FileCopy* faz uma cópia separada do arquivo (como faz o comando Edit Copy do Windows Explorer) quando você utiliza a instrução com a seguinte sintaxe:

FileCopy caminhofonte caminhodestino

em que, *caminhofonte* é o caminho do arquivo do qual você quer fazer a cópia e *caminhodestino* é o caminho do arquivo que você quer criar.

FileCopy não funciona se o arquivo especificado pelo caminhofonte não estiver aberto

No exercício a seguir, você acrescentará um recurso de cópia de segurança ao programa Meu Programa para Excluir Registros, colocando a instrução FileCopy na procedure de evento Form Load.

A instrução FileCopy faz uma cópia de segurança do arquivo

Digite as seguintes instruções de programa na procedure de evento:

```
prompt$ = "Gostaria de fazer uma cópia de segurança do banco de dados?"
reply = MsgBox(prompt$, vbOKCancel, datBiblio.DatabaseName)
If reply = vbOk Then 'copiar o banco de dados se o usuário der 'um clique em OK
    FileNm$ = InputBox("Insira o nome de caminho para a cópia de segurança.")
    If FileNm$ <> "" Then FileCopy datBiblio.DatabaseName, FileNm$
End If
```

A instrução FileCopy faz uma cópia de segurança do banco de dados

Essa rotina exibe uma caixa de mensagem quando o programa é executado, perguntando ao usuário se ele gostaria de fazer uma cópia de segurança do banco de dados. A função MsgBox é utilizada com o argumento vbOkCancel para dar ao usuário uma chance de cancelar. Desta vez a propriedade *DatabaseName* é também utilizada na função MsgBox para exibir o nome do banco de dados na barra de títulos da caixa de diálogo. Se o usuário der um clique em OK, outra caixa de mensagem pega o caminho do arquivo de cópia de segurança, e a instrução FileCopy copia o arquivo.

10.0 CRIANDO UM PROGRAMA DE INSTALAÇÃO

Uma vez criado um aplicativo em Visual Basic, você pode distribui-lo a outras pessoas. Pode-se distribuir livremente qualquer aplicativo criado no Visual Basic a qualquer pessoa que utilize o Microsoft Windows. Se for distribuir seu aplicativo, é necessário escrever ou utilizar um Programa de Instalação que instale seu aplicativo na máquina do usuário. Este capítulo mostra como criar um Programa de Instalação profissional para um aplicativo.

10.1 Criando um Programa de Instalação

Uma vez criado um aplicativo que você gostaria de distribuir a outros, é necessário criar um Programa de Instalação para instalar corretamente seu aplicativo.

O Setup Wizard do Visual Basic realiza a maior parte do trabalho para você. Ele pode determinar quais arquivos precisarão ser distribuídos, onde deverão ser instalados na máquina do usuário, a entrada dos mesmos no registro do sistema e a criação de um Grupo de programa e vínculos do menu Iniciar no Windows. É necessário decidir qual meio será utilizado para a distribuição do aplicativo. Por exemplo, poderão ser criados disquetes, ou o aplicativo pode ser instalado em um servidor de rede ou na Internet.

Na maior parte dos casos, o Setup Wizard pode manipular tudo o que é necessário para seu Programa de Instalação. Entretanto, caso queira personalizar seu Programa de Instalação para atender alguma necessidade especial ou funcionamento, você poder utilizar o Setup Toolkit para criar um Programa de Instalação melhorado ou modificado.

10.2 O Setup Wizard vs. Setup Toolkit

O Setup Wizard, essencialmente é um programa de ajuda que lhe orienta através das etapas necessárias à criação de um Programa de Instalação profissional para seu aplicativo em Visual Basic. O Programa de Instalação que o assistente cria, compilado a partir do projeto do Setup Toolkit do Visual Basic, que se encontra no diretório \setupkit\setup1 da Instalação do seu Visual Basic. Como em qualquer outro projeto em Visual Basic, os formulários, códigos e o funcionamento do projeto da Instalação podem ser melhorados ou modificados.

Na maior parte dos casos, o Setup Wizard é a melhor maneira de criar uma Instalação para o seu aplicativo. Entretanto, caso queira escrever um Programa de Instalação que tenha recursos e funcionamento não fornecidos pelo Setup Wizard, pode-se fazer isto a partir da modificação do projeto do Setup Toolkit. Não será abordado neste capítulo modificações do projeto do Setup Toolkit.

Cuidado:

Caso seja necessário modificar o projeto do Setup Toolkit para adicionar alguma funcionalidade ainda não fornecida, verifique se o projeto do Setup Toolkit foi copiado para um novo diretório, para servir como backup, antes de modificar o código fonte no diretório Setup1. Qualquer modificação feita no projeto do Setup Toolkit afetará qualquer Programa de Instalação subsequente criado com o Setup Wizard.

O Setup Wizard e o Setup Toolkit criam Programas de Instalação e meios de distribuição apenas para aplicativos em Visual Basic. Para criar Programas de Instalação para outros aplicativos baseado no Windows, utilize o Setup Toolkit fornecido com aquele produto de desenvolvimento ou no Microsoft Windows SDK

10.3 Recursos do Setup Wizard

Utilizando o Setup Wizard pode-se criar facilmente um Programa de Instalação profissional para seu aplicativo. O Setup Wizard executa estes passos com a sua entrada:

- Cria um Programa de Instalação principal (Setup.exe). O Setup Wizard compila o projeto do Setup Toolkit para um arquivo executável chamado Setup.exe. Você pode modificar este nome se quiser.
- Compila o arquivo executável do seu aplicativo. Você pode optar por ter seu projeto recompilado pelo Setup Wizard como parte de seu processo.
- Cria opcionalmente um arquivo de dependência para um aplicativo ou componente do Visual Basic. Pode-se optar por criar um arquivo de dependência (.dep) de seu aplicativo ou qualquer componente construído no Visual Basic. Os arquivos de dependência identificam quais arquivos de tempo de execução devem ser incluídos com seu aplicativo ou componente ao serem distribuídos.
- Comprime arquivos e lhes atribui um layout de disco. O Setup Wizard comprime todos os arquivos adequados e determina onde colocá-los nos discos de distribuição, caso sejam aplicáveis.
- Informa o número necessário de discos formatados vazios, se for aplicáveis.
- Copia arquivos de distribuição para discos formatados vazios, caso sejam aplicáveis, e pede que insira disquetes adicionais conforme seja necessário.
- Informa quando os meios de distribuição mestres estiverem completos.
- Copia opcionalmente os arquivos de distribuição para seu disco rígido, que podem então ser copiados diretamente para um compartilhamento de servidor para distribuição através de rede, ou ser transferidos a um sistema de geração de CDS.

O Setup Wizard também fornece os seguintes recursos avançados:

- Suporta a distribuição de seu aplicativo através da Internet, utilizando o descarregamento automático de código a partir do Microsoft Internet Explorer.
- Suporta a Instalação de componentes remotos utilizando Distributed COM (chamado às vezes de DCOM) e Remote Automation.
- Permite a cópia de arquivos para uma estrutura de diretório semelhante à distribuição em disquetes: Disk1, Disk2, Disk3, etc., para a distribuição em um CD ou em uma rede.

Para maiores informações Consulte "Utilizando o Setup Wizard do Visual Basic", posteriormente neste capítulo.

10.4 Criando um Programa de Instalação

Caso opte por criar sua Instalação com o Setup Wizard ou por utilizar diretamente o Setup Toolkit, existem certas etapas que devem ser executadas.

1ª Determine os arquivos que você precisa distribuir. O Setup Wizard determina automaticamente esta informação e, em seguida, gera o arquivo Setup.lst. Além disso, para cada componente ActiveX ou controle criado no Visual Basic para ser usado em outros projetos, o Setup Wizard pode criar um arquivo de dependência (.dep). Por exemplo, todos os arquivos de tempo de execução exigidos por quaisquer controles ActiveX utilizados em seu aplicativo estão listados neste arquivo. O arquivo .dep será então utilizado pelo Setup Wizard, caso seu projeto atual utilize o componente ou controle ActiveX criado por este projeto. Consulte "Arquivos de dependência explicados", posteriormente neste capítulo, para maiores informações.

2ª Crie ou escreva seu Programa de Instalação (Setup.exe). O Setup Wizard cria a Instalação executável a partir do projeto do Setup Toolkit. Caso queira modificar o Programa de Instalação padrão, faça o backup dos arquivos de projeto do Setup Toolkit e efetue as modificações necessárias. Você pode renomear este projeto no momento adequado, desde que modifique o Setup.lst para refletir o novo nome.

3ª Comprima os arquivos adequados. O Setup Wizard comprime e copia os arquivos adequados para os meios de distribuição. Alguns arquivos (Setup.exe e Setup.lst) são copiados para o meio de distribuição descomprimido. Caso utilize o Setup Toolkit, você deverá realizar a compressão manualmente, utilizando o Compress.exe.

4ª Caso utilize discos, determine o layout de disco. O Setup Wizard cuida disso automaticamente, determinando o layout e informando-lhe o número de discos necessários.

5ª Determine onde instalar os arquivos na máquina do usuário. Tipos diferentes de arquivos precisam ser instalados em diretórios diferentes na máquina do usuário. Por exemplo, no Windows 95 os

arquivos de Instalação e de programas são usualmente instalados em Arquivos de Programas\Nome de seu Aplicativo (Program Files\YourAppName) e os arquivos do sistema e de dependência são copiados para o diretório Windows\system. Você pode especificar outros caminhos no Setup Wizard caso seja necessário ignorar as configurações padrão. Ao utilizar diretamente o Setup Toolkit, você deve adicionar manualmente estas informações ao arquivo Setup.lst.

6ª Modifique Setup.lst para incluir todos os arquivos em sua lista. O Setup Wizard cria de maneira automática o Setup.lst, derivando esta lista a partir das dependências de seu aplicativo, arquivos de programas, etc. Ao utilizar diretamente o Setup Toolkit, você deve criar manualmente esta lista.

7ª Crie os meios de distribuição. O Setup Wizard lhe pergunta o tipo de meio que deseja utilizar. Você pode selecionar discos, um servidor de rede ou distribuição através de descarregamento automático de código através da Internet. Você também pode criar manualmente meios de distribuição.

8ª Teste o Programa de Instalação. Utilizando o Setup Wizard ou o Setup Toolkit, você deve testar a Instalação do seu aplicativo antes da distribuição. Consulte "Testando seu Programa de Instalação", posteriormente neste capítulo para maiores informações.

10.5 Recursos Avançados

Adicionalmente, você pode adicionar recursos avançados a seu Programa de Instalação. Muitos deles estão integrados ao Setup Wizard; entretanto, pode ser necessário modificar manualmente algum aspecto destes recursos. A lista abaixo identifica estes recursos e aponta a seção deste capítulo que os aborda com mais detalhes:

- Criar pacotes para descarga de componentes da Internet. Utilize o Setup Wizard para criar arquivos .cab para os componentes ActiveX usados nos seus aplicativos da Internet. Consulte "Descarga de componentes da Internet", posteriormente neste capítulo para maiores informações.
- Instalar componentes ActiveX. Utilize o Setup Wizard para criar Programas de Instalação tanto para cliente quanto para servidor em um ambiente Remote Automation ou Distributed COM (às vezes chamado DCOM). Consulte "Instalando componentes de Remote Automation e de Distributed COM", posteriormente neste capítulo para maiores informações.
- Instalar componentes de acesso a dados. Caso você tenha utilizado Data Access Objects (DAO) em seu aplicativo, consulte "Instalando Data Access Components", posteriormente neste capítulo para maiores informações sobre como utilizar o Setup Wizard, para instalá-los.
- Distribuir seu aplicativo em uma rede. O Setup Wizard lhe oferece duas opções para a Instalação de seus aplicativos em uma rede. Consulte "Opções de distribuição", posteriormente neste capítulo, para maiores informações.

10.6 Arquivos que Você Está Autorizado a Distribuir

Você pode distribuir livremente qualquer aplicativo ou componente que criar com o Visual Basic. Além de um arquivo executável (.exe), seu aplicativo pode exigir outros arquivos, tais como DLLs, controles ActiveX (.ocx) ou bitmaps (.bmp).

Você pode distribuir legalmente arquivos de aplicativo de exemplo (encontrados no subdiretório \samples) e quaisquer arquivos originalmente copiados para o subdiretório \icons, do diretório principal do Visual Basic, na primeira Instalação do Visual Basic em seu sistema.

Professional e Enterprise Editions

Caso tenha adquirido as Professional ou Enterprise Editions do Visual Basic, você também pode distribuir qualquer arquivo copiado originalmente para os subdiretórios \Graphics e IODBC do diretório principal do Visual Basic, na primeira Instalação do Visual Basic em seu sistema.

Você também pode distribuir outros controles ActiveX, arquivos .exe e .dlls adquiridos. Consulte o contrato de licença do fabricante de cada um dos arquivos que você planeja distribuir, para determinar se existe o direito de distribuí-los junto com seu aplicativo

10.7 Utilizando o Setup Wizard do Visual Basic

O Setup Wizard do Visual Basic toma mais fácil a criação de um Programa de Instalação para seu aplicativo. Como outros assistentes, o Setup Wizard pede informações para criar o que você quer.

Na maioria dos casos, o Setup Wizard é tudo o que você precisa para criar um Programa de Instalação.

➢ **A Para iniciar o Setup Wizard**

1º - Caso o projeto para o qual você quer criar um Programa de Instalação esteja aberto, salve-o e feche o Visual Basic.

2º - Selecione o ícone do Setup Wizard no menu Start do Visual Basic no Windows 9x ou no Windows NT.

Cada tela do Setup Wizard pede informações sobre seu projeto e lhe permitir escolher quais opções você gostaria de compilar na sua Instalação, como seu projeto deve ser distribuído, se deseja que o Setup Wizard compile seu aplicativo, etc.

Cada uma das telas também explica como elas devem ser utilizadas, explica quando determinadas informações são opcionais, e quais informações devem ser inseridas antes que o processo possa continuar (antes que possa prosseguir até a próxima tela). À medida que prossegue através das telas, se achar que precisa de mais informantes, pressione F1 ou clique no botão Help.

Caso seu aplicativo vá ser executado em um sistema operacional BiDi, é preciso incluir manualmente a Vba332me.dll no Setup.lst criado pelo Setup Wizard. Isto pode ser feito adicionando o arquivo na etapa File Summary ao executar o Setup Wizard, editando diretamente o Setup.lst ou adicionando uma entrada para o arquivo Vba332me.dll ao arquivo Vbsdep.ini, de modo que ele seja automaticamente adicionado ao Setup.lst sempre que o Setup Wizard for executado

10.8 Arquivos de Dependência Explicados

Um arquivo de dependência (.dep) contém informações sobre as exigências de tempo de execução de um aplicativo ou componente: quais arquivos são necessários, como eles devem ser registrados, e em que local da máquina do usuário devem ser instalados. Você pode criar arquivos .dep de projetos Visual Basic padrão ou – nas Professional e Enterprise Editions do Visual Basic – de controles ActiveX, documentos ActiveX e outros componentes ActiveX.

- **A anatomia da dependência**

O Visual Basic contém três fontes de informação de dependência:

- Arquivos de componente .dep
- Arquivos de projeto .dep
- O arquivo Vb5dep.ini

Arquivos de dependência de componente

Em breve, todo componente do Visual Basic será acompanhado por seu próprio arquivo .dep. Isto se aplica a controles e outros componentes adquiridos e àqueles criados por você no Visual Basic. Por exemplo, todos os controles ActiveX que acompanham o Visual Basic possuem um arquivo .dep. companheiro. Estes arquivos .dep listam todos os arquivos dependentes utilizados pelo controle, mais as informações de versão e registro. O exemplo abaixo é uma parte de um arquivo .dep de controle típico:

```
[RichTx32.ocx]
Dest = $(WinsysPath)
Register = $(DLLSelfRegister)
Version = a.b.c.d
Uses1 = RichEd32.dll
CABFileName = RichTx32.cab
CABINFFiIe = RichTx32.inf
```

Estes arquivos .dep são usados para fornecer informações de dependência sobre o controle ou outro componente. Ao se utilizar um componente em outro projeto, estas informações são incorporadas às informações de dependência do projeto. Em outras palavras, as informações de dependência nas unidades menores fornecem informações de dependência às unidades maiores.

É altamente recomendável que seja gerado um arquivo .dep para qualquer componente criado no Visual Basic. A criação de um arquivo .dep para controles ActiveX, .DLLs de documentos ActiveX e outros componentes ActiveX, especialmente útil quando seu objeto estiver sendo utilizado como uma dependência em um outro projeto.

Você pode gerar um arquivo .dep para seu componente, selecionando a opção Generate Dependency File Only ao executar o Setup Wizard.

Projetar arquivos de dependência

Ao criar um Programa de Instalação para seu aplicativo utilizando o Setup Wizard, os arquivos .dep de quaisquer componentes usado em seu projeto são incorporados ao arquivo Setup.lst do projeto. O Setup Wizard também retira informações de dependência sobre seu aplicativo de referências em seu projeto e do arquivo Vb5dep.ini. O arquivo Setup.lst do projeto contém informações de dependência sobre cada elemento de seu projeto.

Caso seja necessário, você também pode criar um arquivo de dependência para seu aplicativo. Os arquivos .dep para quaisquer componentes usado em seu projeto estão incorporados ao arquivo .dep do projeto.

Você pode gerar um arquivo .dep para seu aplicativo selecionando as opções Create a Setup Program e Generate Dependency File ao executar o Setup Wizard.

O arquivo Vb5dep.ini

O arquivo Vb5dep.ini é uma substituição do arquivo Swdepend.ini usado em versões anteriores do Visual Basic, e seu objetivo essencial é o mesmo: fornecer ao Setup Wizard uma lista de uso geral de dependências e referências utilizadas pelo Visual Basic. Esta lista é criada quando você instala o Visual Basic e reside no subdiretório \SetupKit\Kitfil32, do diretório principal do Visual Basic.

O exemplo abaixo é uma parte de um vb5dep.ini típico:

```
[DA0350.dll]
Dest =$(MSDAOPath)
Uses1 = MSJtEr35.dll
Uses2 = MSJInt35.dll
Register = $(DLLSelfRegister)
CABFileName = MSDAD350.cab
CABINFFiIe = MSDA0350.inf
```

Como se pode ver, o vbsdep.ini é muito semelhante ao exemplo de arquivo .dep de controle mostrado acima. Ao executar o Setup Wizard, ele verificar primeiramente o arquivo Vb5dep.ini para localizar as informações de dependência. Caso elas não estejam disponíveis, e se um determinado componente não tiver seu próprio arquivo .dep, o Setup Wizard lhe informar sobre a dependência em falta, e lhe permitir ignorar esta omissão ou corrigir o problema.

Caso ignore a omissão, seu programa pode não funcionar corretamente uma vez instalado na máquina do usuário. No entanto, caso você esteja ciente de que uma determinada dependência já está carregada na máquina do usuário, poder prosseguir sem problemas.

Pode-se também adicionar informações de dependência de um componente editando o arquivo vb5dep.ini (adicionando manualmente uma entrada para um determinado componente), criando um arquivo .dep para o componente com o Setup Wizard, ou contatando o vendedor do componente e solicitando um arquivo .dep.

Pode ser útil considerar os arquivos .dep como fornecedores de uma lista de dependências para componentes ou aplicativos individuais, e o arquivo Vb5dep.ini como fornecedor de uma lista de dependências para todo o seu ambiente de desenvolvimento Visual Basic.

A ilustração abaixo mostra como o Setup Wizard combina arquivos .dep de componentes com o projeto em Visual Basic e o arquivo Vb5dep.ini, para criar um Programa de Instalação, um Setup.lst e um arquivo .dep de projeto.

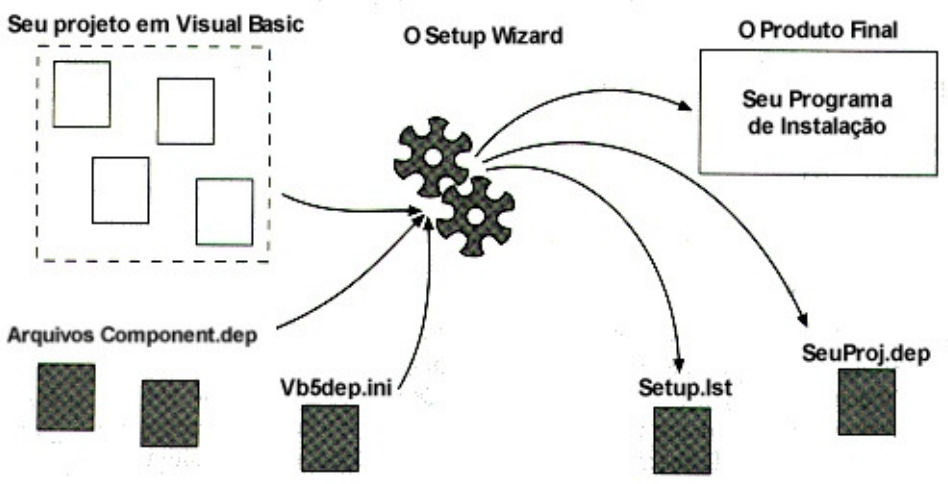


Figura 10.8 – Criando um Programa de Instalação, Setup.lst e arquivo de dependência do projeto Seu projeto em visual Basic

10.9 Gerando Arquivos de Dependência com o Setup Wizard

Pode-se utilizar o Setup Wizard para criar um arquivo .dep ao se criar o Programa de Instalação de seu aplicativo ou usar o assistente para simplesmente criar informações de dependência para um componente ou para o projeto (caso precise do Setup.lst para seu Programa de Instalação personalizado). O arquivo de dependência recebe o mesmo nome que seu aplicativo ou componente, e está localizado no mesmo diretório.

Mesmo que você opte por criar um Programa de Instalação personalizado, deve utilizar o Setup Wizard para gerar um arquivo .dep. Este poderá, então, ser utilizado para auxiliar no desenvolvimento do seu Setup.lst e na distribuição do seu aplicativo

Para criar um arquivo de dependência você precisa:

1º Iniciar o Setup Wizard.

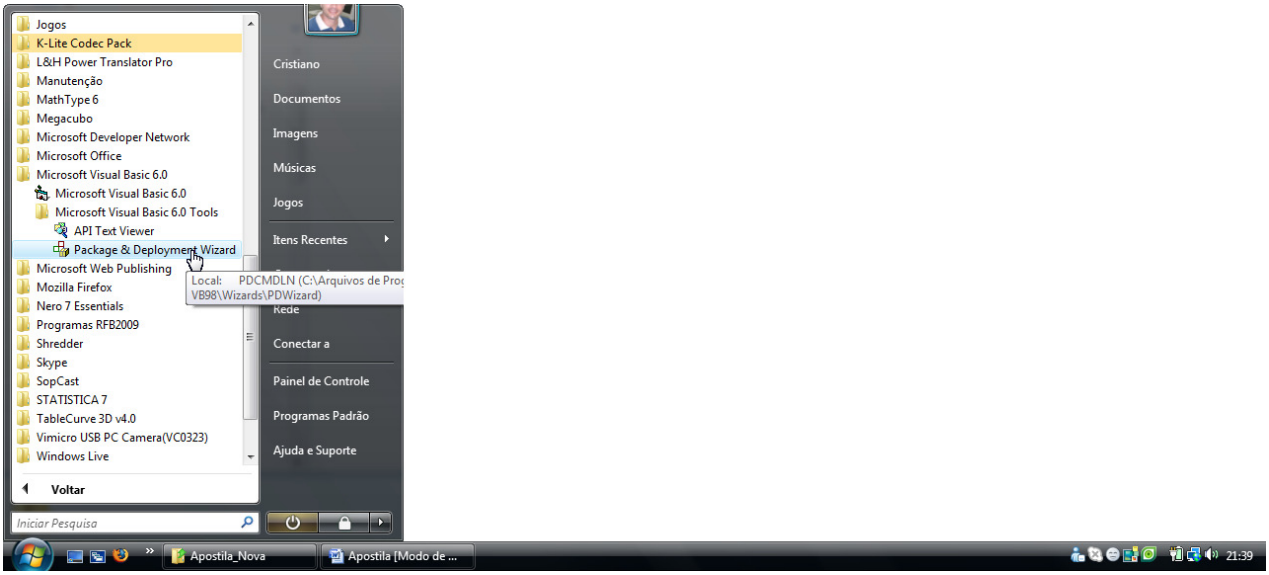


Figura 10.9.1 – Opção de inicialização do Setup Wizard na pasta Programa do menu principal do Windows

2º Clicar no botão Next para selecionar a tela Select Project and Options.

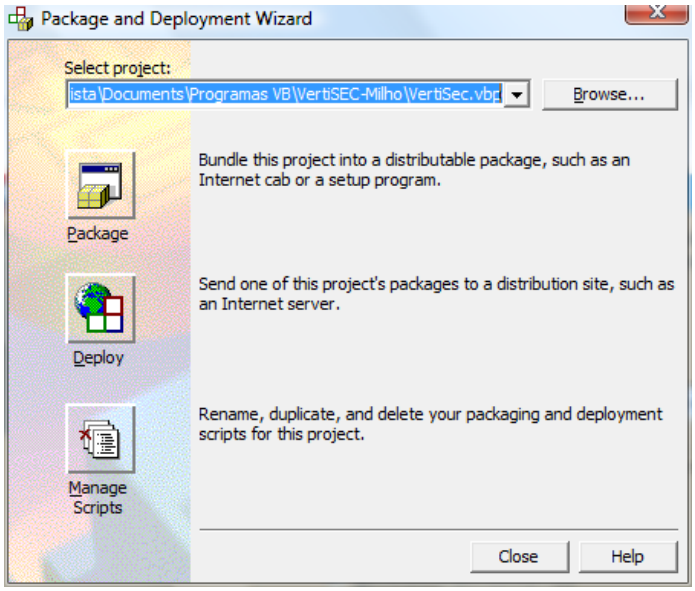


Figura 10.9.2 – A tela Select Project and Options

3º Escolha seu projeto em Visual Basic digitando o caminho e o nome do arquivo na caixa de entrada Project File, ou selecionando o botão Browse.

4º Selecione a opção Generate Dependency File Only para criar somente um arquivo .dep, ou a opção Create a Setup Program/Generate Dependency File para criar um Programa de Instalação e um arquivo .dep de projeto. Clique no botão Next. Caso queira que o Setup Wizard recompile seu projeto, selecione a opção Rebuild the Project.

5º Se estiverem faltando informações de dependência para qualquer componente em seu projeto, você será informado pela tela Missing Dependencies. Caso você saiba que determinados componentes já estão instalados na máquina do usuário, ou que um arquivo listado não tenha arquivos de dependência, você pode clicar neste arquivo para prosseguir. Você também pode prosseguir sem confirmar as dependências em falta, embora seu projeto possa não funcionar corretamente.

Caso estejam faltando informações de dependência para um componente criado por você no Visual Basic (ou o Setup Wizard lhe informar que esta informação está desatualizada), você deverá, primeiro, utilizar o Setup Wizard para criar um arquivo .dep para este componente (usando estas etapas, mas selecionando o arquivo .vbp para o projeto do componente) e, em seguida, reiniciar este processo para o projeto atual

Ao terminar de resolver as dependências que falta, dique em Next para prosseguir.

6º O Setup Wizard lhe apresentará as telas de confirmação de arquivo padrão. Passe por cada uma delas até chegar à tela Finished!. Clique no botão Finish.

Um arquivo de dependência (.dep) é criado com o mesmo nome do seu projeto, no diretório origem do projeto.

10.10 Opções de Distribuição

Você pode distribuir os seus aplicativos em disquetes, CDS ou em uma rede. Caso você use disquetes, poderá também escolher entre os métodos: Disquete do Setup Wizard ou Diretórios de Disco. Caso planeje a distribuição em um CD ou através de rede, você também pode escolher entre os métodos: Single Directory ou Disk Directories.

Caso possua a Professional ou Enterprise Edition, você pode distribuir os seus componentes através da Internet. Para maiores informações, consulte "Descarga de componentes da Internet", posteriormente neste capítulo. "Instalando os componentes Remote Automation e Distributed COM" fornece especificações sobre o trabalho com Remote Automation e com DCOM, que estão disponíveis na Enterprise Edition.

Figura 10.10 – Selecionando opções de distribuição com o Setup Wizard

Se você planeja distribuir seu aplicativo em disquetes, você pode usar a opção Floppy Disk para criar disquetes em seu computador ou pode usar a opção Disk Directories para criar imagens de disquetes que podem ser usadas por um serviço de duplicação de disquetes. Caso planeje distribuir seu aplicativo em uma rede ou em um CD, você também tem duas opções: pode utilizar a opção Single Directory ou a opção Disk Directories. Ambos os métodos copiam os arquivos de Instalação para um diretório temporário em seu computador ou em um servidor da rede. Você tanto pode colocar os arquivos de Instalação no servidor da rede adequado quanto transferi-los para um CD.

- **Disquetes**

Para criar disquetes para a Instalação de seu aplicativo quando não são necessárias imagens de discos, selecione a opção Floppy Disk na tela Distribution Method do Setup Wizard. Esta tela está disponível quando você executa o Setup Wizard e seleciona a opção Create a Setup Program.

O Setup Wizard lhe perguntará qual o tipo de disquete (1,44; 1,2; 720 ou 360) e qual é a unidade de disco existente no seu computador. Como ocorre com a maioria das outras funções do Setup Wizard, será solicitada a confirmação dos arquivos do projeto e dependências. Uma vez confirmados, o Setup Wizard determinar o layout dos discos, comprimir os arquivos adequados e então os copiará para a unidade de disco, pedindo-lhe que insira e remova os disquetes conforme necessário.

Você também pode utilizar a opção Disk Directories para criar imagens de discos do seu Programa de Instalação. O Setup Wizard executa essencialmente as mesmas ações que para a opção Floppy Disk, com a exceções de que as imagens são copiadas para Diretórios separados em um diretório temporário do seu computador ou em um servidor de rede, ao invés de serem copiadas para uma unidade de disquetes. Você pode, a seguir, copiar manualmente os arquivos para discos, colocar os arquivos em um servidor da rede e permitir aos usuários que os copiem para discos (desde que você esteja trabalhando em um ambiente de rede), ou fornecê-los a um serviço de duplicação de disquetes,

- **Diretório único**

Você pode criar uma Instalação em um diretório único selecionando a opção Single Directory. Esta ou a opção Disk Directories podem ser usadas quando se pretende distribuir o aplicativo em uma rede ou em CD. Esta técnica simplesmente copia todos os arquivos de Instalação para um diretório único.

Ao selecionar esta opção, o Setup Wizard lhe pedirá que confirme os arquivos do projeto e dependências, para então, comprimir e copiar os arquivos adequados para um diretório temporário no seu computador ou em um servidor de rede. Como padrão, estes arquivos são copiados para C:\Windows\Temp\SwSetup.

Pode-se também colocar os arquivos em um servidor de rede adequado, ou transferi-los para um CD.

- **Diretórios de disco**

A opção Disk Directories copia os arquivos para uma estrutura de diretório que se parece com a distribuição em discos: Disk1, Disk2, Disk3, etc.

Assim como na opção Single Directory, ao selecionar esta opção, o Setup Wizard vai lhe pedir a confirmação dos arquivos e dependências do projeto e, em seguida, comprimir e copiar os arquivos adequados para um diretório temporário em seu computador ou servidor de rede. Como padrão, os arquivos são copiados para C:\Windows\Temp\SwSetup.

Você pode colocar estes Diretórios (Disk1, Disk2, etc.) em um servidor de rede ou transferi-los para um CD para distribuição. O usuário pode instalar o aplicativo simplesmente abrindo a pasta Disk1 e clicando duas vezes no Programa de Instalação.

10.11 Trabalhando com Modelos do Setup Wizard

Se você tiver um aplicativo que exija a criação do Programa de Instalação para mais de uma vez, você poder optar por salvar os passos já executados através do Setup Wizard em um arquivo (.swt) de modelo. Nas versões anteriores do Visual Basic, a extensão do arquivo de modelo do Setup Wizard era .vbz.

Para criar um modelo para seu Programa de Instalação, estude as etapas do Setup Wizard para criar seu Programa de Instalação e, sem seguida, selecione a opção Save Template na tela Finished! antes de selecionar o botão Finish.

A próxima vez que for necessário criar uma Instalação para seu programa, você poder carregar o arquivo de modelo (.swt) ao invés do arquivo de projeto .vbp na tela Select Project and Options do Setup Wizard e, em seguida, clicar imediatamente no botão Finish, ao invés de passar por cada uma das telas do Setup Wizard.

Devido fato de que o Setup Wizard na versão 5.0 do Visual Basic ter evoluído bastante, os modelos (arquivos .vbz) das versões anteriores do Visual Basic não são compatíveis

10.12 Instalando Data Access Components

Caso seu aplicativo utilize Data Access Objects (DAO), o Setup Wizard solicitará que você escolha o ISAM apropriado e os componentes da área de trabalho.

Você pode escolher um ou mais formatos de banco de dados ISAM. Você deve selecionar pelo menos uma área de trabalho; o Setup Wizard não lhe permitirá anular a seleção de ambas as opções de área de trabalho - você deve selecionar uma ou ambas.

Caso seu aplicativo exija drivers ODBC, você deve instilá-los separadamente. Veja abaixo para maiores informações.

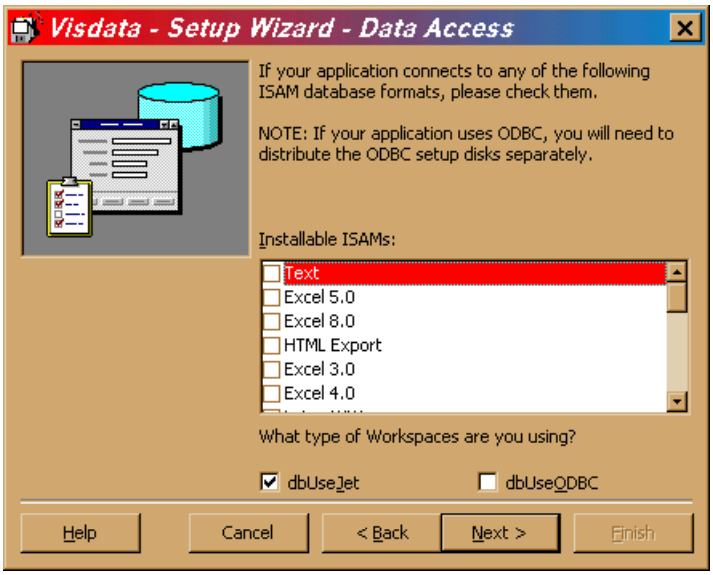


Figura 10.12 – Opções Data Access

- **Criando um disco de Instalação para aplicativos ODBC**

Nas Professional e Enterprise Editions, caso você crie um aplicativo que utilize ODBC, e você deseja distribuir este aplicativo, um Disco de Instalação ODBC deve ser criado. Antes de instalar seu aplicativo em Visual Basic na máquina do usuário (utilizando um Programa de Instalação que você escreveu), o ODBC precisa ser instalado na máquina do usuário.

Caso seu aplicativo utilize Remote Data Objects (RDO), você deve instruir o usuário a instalar os drivers ODBC antes de instalar seu aplicativo. Caso os drivers ODBC não sejam instalados primeiro, os componentes RDO não serão registrados

Para criar um disco de Instalação ODBC

1. Copie todos os arquivos no subdiretório \Odbc do diretório principal do Visual Basic para um disco ou diretório de rede.
2. Antes de instalar seu aplicativo na máquina do usuário, execute o Setup.exe no disco de Instalação ODBC ou diretório de rede. Utilize a função Shell para executar o Setup.exe do ODBC a partir de seu Programa de Instalação, ou rotule o disco de Instalação do ODBC com um título, como: "Programa de Instalação ODBC: Execute o Setup.exe antes de instalar o aplicativo".

Uma vez colocado os arquivos ODBC em um disco, teste o disco de Instalação do ODBC em uma máquina que não contenha arquivos ODBC.

Para instalar o ODBC em uma máquina de usuário, você precisa usar o Programa de Instalação fornecido no subdiretório o ODBC do diretório principal do Visual Basic. Alguns dos arquivos ODBC estão comprimidos e o Programa de Instalação deve descomprimi-los para instalá-los corretamente

10.13 Instalando os componentes Remote Automation e Distributed COM

Caso seu aplicativo utilize componentes de código ActiveX (anteriormente chamados de servidores OLE) o Setup Wizard lhe oferece os meios de criar Programas de Instalação tanto para o cliente quanto para o servidor em um ambiente de Remote Automation ou um ambiente Distributed COM (chamado também DCOM), com a Enterprise Edition.

Remote Automation e Distributed COM são abordados em detalhe no Guia de Criação de Aplicativos Cliente/Servidor com o Visual Basic, disponível na Enterprise Edition. Este exemplo ilustra como criar Programas de Instalação para o aplicativo servidor e cliente que o utiliza e apresenta as opções tanto para Remote Automation quando para DCOM.

Para criar um Programa de Instalação para o servidor, faça como segue:

1. Inicie o Setup Wizard.
2. Clique no botão Next para selecionar a tela Select Project and Options.
3. Escolha o projeto servidor digitando o caminho e nome do arquivo na caixa de entrada Project File, ou selecionando o botão Browse. Neste exemplo, o projeto chama-se Book_svr.vbp e se encontra em \samples\clisvr\booksale\server.
4. Selecione a opção Create a Setup Program e, em seguida, clique no botão Next para prosseguir.
   Caso queira que o Setup Wizard recompile seu projeto, selecione a opção Rebuild the Project.
5. Prossiga pelo Setup Wizard até a tela Shared ActiveX Applications.

Figura 10.13 – Selecionando opções de aplicativo servidor

Selecione Install as a shared component e. Nesse exemplo não se trabalhou com DCOM, caso tivesse trabalhado apareceria uma mensagem, selecionaria **Yes** se estiver usando Remote Automation ou **No** se estiver usando Distributed COM.

Caso tenha selecionado a opção Remote Automation, os arquivos de Remote Automation Racmgr32.exe, Racreg32.dll, Autmgr32.exe e Autprx32.dll serão adicionados a Setup.lst, e instalados com o aplicativo servidor no diretório \windows\system do servidor remoto.

6. Prossiga com o Setup Wizard para criar o Programa de Instalação para o servidor.

O aplicativo servidor é instalado no computador servidor remoto. O Programa de Instalação para o aplicativo servidor instalar o aplicativo e os arquivos de Remote Automation.

Para criar um Programa de Instalação para o cliente, faça como segue:

1. Antes de prosseguir, certifique-se de ter criado um arquivo .vbr para cada servidor remoto a que você se refira em seu aplicativo. Crie um arquivo .vbr selecionando a opção Remote Server na guia Component da caixa de diálogo Project Properties no Visual Basic e, em seguida, compilando o projeto.
2. Inicie o Setup Wizard.
3. Clique no botão Next para selecionar a tela Select Project and Options.
4. Escolha o projeto cliente digitando o caminho e o nome do arquivo na caixa de entrada Project File ou selecionando o botão Browse, Neste exemplo, o nome do projeto, Book_svr.vbp.
5. Selecione a opção Create a Setup Program e, em seguida, clique no botão Next para prosseguir.

Caso queira que o Setup Wizard recompile seu projeto, selecione a opção Rebuild the Project.

Antes de criar um Programa de Instalação para o cliente, você deve ter compilado o aplicativo pelo menos uma vez no Visual Basic

6. Selecione as opções de meio de distribuição e local e, em seguida, prossiga até a tela ActiveX Components.

Caso o projeto cliente contenha uma referência a um servidor local, o aplicativo servidor será listado na caixa de arquivo.

7. Será pedido que você forneça o arquivo .vbr do aplicativo servidor. Localize o arquivo em seu diretório de origem do projeto de servidor e escolha Open. O arquivo .vbr ser adicionado à lista de arquivo, e a caixa de diálogo Remote Connection Details lhe pedirá as informações da conexão.

Se você estiver usando Distributed COM, escolha-a nas opções de Remote Transport, e em seguida digite o nome do computador do servidor remoto na caixa de entrada Network Address.

Caso esteja usando Remote Automation, selecione-o e, em seguida, especifique o endereço de rede, o protocolo de rede e o nível de autenticação, ou deixe em branco para pedir ao usuário cliente esta informação durante o tempo de execução.

Selecione as opções adequadas e, em seguida, clique no botão OK para retornar à tela ActiveX Components. Se tanto a versão local (.exe) quanto a versão remota (.vbr) do componente servidor remoto aparecerem na lista de componentes, anule a seleção da versão local (.exe), de modo que somente o servidor remoto seja usado pelo seu aplicativo quando instalado.

8. Prossiga com o Setup Wizard para criar a Instalação do aplicativo cliente.

Instale o aplicativo cliente na máquina do usuário cliente e siga os procedimentos de Remote Automation ou Distributed COM para usar os aplicativos cliente e servidor simultaneamente.

10.14 Descarga de Componente da Internet

Com as versões Professional e Enterprise do Visual Basic, é possível criar controles ActiveX, DLLs ActiveX, EXEs ActiveX e documentos ActiveX para serem usados em páginas Web na Internet. Por exemplo, você pode criar um controle ActiveX que, usado em uma página Web armazenada no Internet Explorer. O Setup Wizard lhe oferece um meio de empacotar estes componentes para distribuição através da Internet.

- **Como funciona a descarga de componentes da Internet**

Sem entrar em muitos detalhes, pode ser útil descrever aqui como os componentes ActiveX são usados na Internet.

O Visual Basic permite criar controles ActiveX, DLLs ActiveX, EXEs ActiveX e documentos ActiveX que podem ser usados em um navegador Internet, como por exemplo, o Internet Explorer. Por exemplo, você pode criar um controle que pede ao usuário um determinado tipo de informação e, em seguida, processa esta informação de alguma forma. Para usar este controle no Internet Explorer, você se refere a ele no código HTML subjacente, talvez usando o VBScript. Em outras palavras, seu controle está armazenado na página Web.

A página Web, junto com seu componente e qualquer outro arquivo dependente, reside em um local específico na World Wide Web. Quando o usuário acessa esta página Web, seu controle é ativado. Isto é, ele é descarregado (em forma de um arquivo .cab comprimido) junto com a página Web no computador do usuário. O controle, então verificado quanto à segurança, descomprimido, registrado no Windows, instalado e só então ativado.

Tudo isto ocorre no segundo plano e é controlado pelo aplicativo navegador da Internet. Uma vez que esta forma de distribuição difere do método mais comum de distribuição de aplicativos através de discos ou CDs, novas questões devem ser consideradas. A mais importante delas é a segurança, tanto para o usuário quanto para seu código fonte.

A finalidade do Setup Wizard em tudo isto é empacotar seu componente ActiveX. O Setup Wizard determina quais arquivos seu projeto precisará no tempo de execução, reúne estes arquivos, comprime-os em um arquivo .cab, e gera o exemplo de código HTML que aponta para seu controle.

Empacotando os componentes ActiveX você cria um pacote Internet escolhendo Create Internet Download Setup na tela Select Project and Options do Setup Wizard.

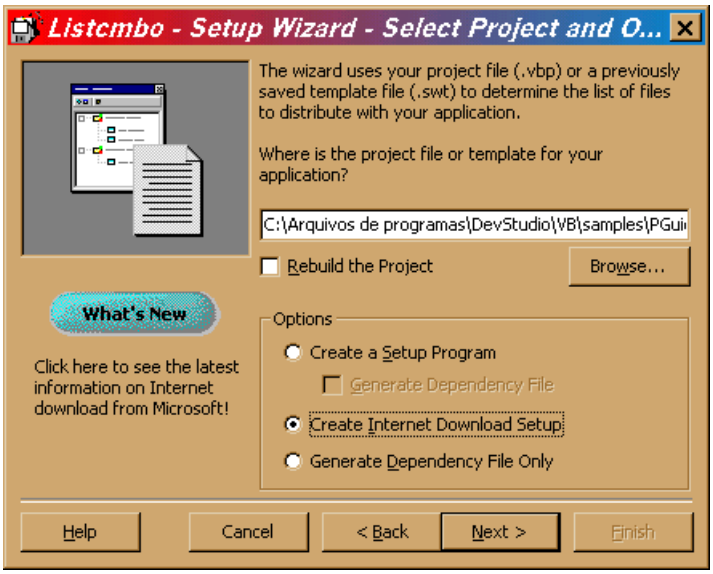


Figura 10.14.1 – Selecionando a opção Create Internet Download

O Setup Wizard criará um arquivo .cab, conhecido como arquivo .cab primário, que contém o seguinte:

- Os componentes do projeto, como por exemplo, o controle ActiveX, DLL ActiveX ou EXE ActiveX.
- O arquivo .inf que contém vínculos até outros arquivos .cab que contém arquivos de suporte e controles do Visual Basic, assim como, outras informações, como por exemplo, se o controle é seguro para script e para inicializações, e informações de registro conforme definidas pelo usuário. Este arquivo substitui o arquivo Setup.lst que o Setup Wizard cria na Instalação padrão.
- Espaço reservado para assinaturas digitais.
- Todos os arquivos que não se encontram em outros arquivos .cab (secundários).

Arquivos .cab secundários

Para projetos de controle ActiveX, EXES ActiveX e DLLS ActiveX, todos os componentes de tempo de execução – como Msvbvms50.dll, controles individuais, Data Access Objects (DAO) e Remote Data Objects (RDO) – são empacotados em arquivos .cab separados, digitalmente assinados pela Microsoft, e colocados no site da Web da Microsoft. Você pode escolher vincular seus arquivos aos arquivos .cab do site da Web da Microsoft, ou pode descarregar cópias deles localmente.

As vantagens de se usar arquivos .cab secundários de um site da Web são:

- Você não precisa distribuir todos os arquivos .cab exigidos por seu aplicativo. O único arquivo que você precisa distribuir é o arquivo .cab primário. O arquivo .inf dentro do arquivo .cab primário aponta para o site da Web da Microsoft e descarrega os arquivos .cab necessários, com base nas necessidades do usuário final.
- Eles oferecem um meio eficiente de entregar as atualizações para o seu produto.

Se você não puder ou não quiser que a Instalação de seu aplicativo exija uma conexão à Internet, você pode colocar os arquivos .cab secundários em um servidor em sua Intranet. Uma intranet permite descarga mais rápida, ao mesmo tempo que permite aos usuários permanecer dentro de uma rede protegida

Questões de segurança

Se seus componentes forem salvos para script e inicialização em uma página Web, você pode fazer com que o Setup Wizard os marque como tal. Ele também reserva espaço no arquivo .cab para assinar digitalmente seu componente.

➢ **Criando um pacote de transferência através da Internet**

Utilize o procedimento abaixo para criar um pacote de descarga através da Internet para seu controle ActiveX, DLL ActiveX, EXE ActiveX e projetos de documento ActiveX.

Para criar um pacote de descarga através da Internet, siga os seguintes passos:

1. Inicie o Setup Wizard.
2. Clique no botão Next para selecionar a tela Select Project and Options.
3. Escolha seu controle ActiveX, DLL ActiveX, EXE ActiveX ou projeto de documento ActiveX do Visual Basic digitando o caminho e nome do arquivo na caixa de entrada Project File, ou selecionando o botão Browse.
4. Selecione Create Internet Download Setup e, em seguida, dique no botão Next para prosseguir até a tela Internet Distribution Location.
5. Será solicitado um local de distribuição na Internet. Este deve ser um local temporário em sua máquina – mais tarde você poderá carregar os arquivos relevantes em seu servidor Web. O padrão é C:\windows\Temp\SwSetup. Selecione o local e, em seguida, clique em Next para prosseguir.
6. Será então solicitado que você descarregue as informações sobre os componentes em seu projeto.

Figura 10.14.2 – Especificando informações de transferência de arquivos .cab secundários

Se você quiser vincular arquivos .cab secundários aos controles Microsoft ou componentes usados em seu projeto, selecione a opção Download from the Microsoft web site. Todos os controles ActiveX que acompanham o Visual Basic contém informações da Internet em seus arquivos dependentes de companheiro (.dep). O Setup Wizard grava as informações adequadas do arquivo .inf no arquivo .cab que ele cria.

Se você quiser usar arquivos .cab secundários não localizados no mesmo diretório que o arquivo .cab primário, selecione a opção Use Alternate Location. Você insere então o caminho do URL ou UNC onde a Instalação da Internet poderá encontrar estes arquivos .cab. Deixar o caminho em branco indica que os arquivos .cab secundários estarão no mesmo diretório que o arquivo .cab primário. Vazio é o padrão.

Selecione a opção adequada.

7. Agora, você pode marcar seus componentes como salvos para inicialização e script.

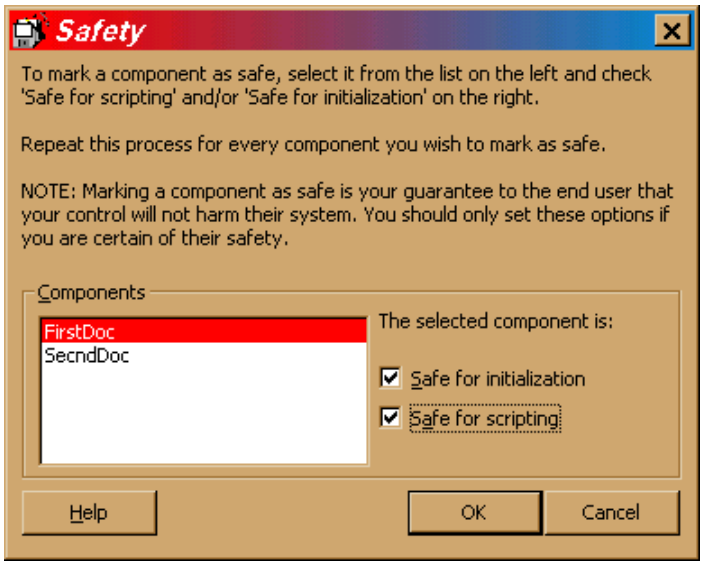


Figura 10.14.3 – Marcando os componentes quanto à segurança

Ao marcar seus componentes como salvos para inicialização e script, você está garantido que seu componente nunca provocará comportamento indesejável em uma máquina de usuário final, mesmo se for usado em uma página Web que você mesmo não tenha criado.

8. Você continuará a percorrer o Setup Wizard para confirmar arquivos e dependências. Ao atingir a tela Finished!, dique em Finish para criar o pacote de descarga através da Internet.

➢ **Distribuindo o pacote de Instalação através da Internet**

O Setup Wizard cria um arquivo .cab que contém seu componente, um arquivo .inf e espaço para assinatura digital. Ele também cria um exemplo de arquivo .htm que pode ser usado para referir-se a seu componente a partir da página Web.

No exemplo acima, criamos um pacote de descarga através da Internet para um documento ActiveX. Foram criados os arquivos abaixo:

Arquivo	Descrição
Actxdoc.cab	O arquivo .cab que contém: o documento ActiveX (.dll) e o arquivo .inf.
Actxdoc.htm	Um arquivo HTML contendo exemplo de código HTML.
FirstDoc.vbd	O arquivo de estado para o objeto Firstdoc no documento ActiveX.
SecndDoc.vbd	O arquivo de estado para o objeto SecndDoc no documento ActiveX.

Você distribui os arquivos .cab e .vbd junto a página Web no site da Web. O arquivo .htm contém exemplo de código HTML que aponta o navegador para os objetos de documento ActiveX. Você pode recortar e colar seções do exemplo de código no código HTML subjacente de sua página Web, para exemplificar qualquer um dos componentes documento ActiveX.

➢ **Recriando o arquivo .CAB**

O Setup Wizard também cria um subdiretório chamado Support que contém todos os arquivos que foram comprimidos no arquivo .cab, Novamente, aproveitando nosso exemplo anterior, os arquivos abaixo seriam encontrados no diretório Support:

Arquivo	Descrição
Actxdoc.dll	O documento ActiveX compilado.
Actxdoc.ddf	O arquivo .ddf , usado pelo programa de compressão.
Actxdoc.inf	O arquivo .inf que contém todas as dependências, versão, Instalação e informações de segurança do componente.

Se for necessário, você pode usar estes arquivos para recompilar o arquivo .cab (por exemplo, se precisar, por qualquer motivo, executar alterações personalizadas no arquivo .inf). Os arquivos .cab são

comprimidos utilizando um programa utilitário chamado Makecab.exe, que está localizado no subdiretório \SetupKit\Kitfil32 do diretório principal do Visual Basic.

Presumindo-se que a localização de Makecab.exe esteja no caminho de seu sistema, você pode recompilar o arquivo .cab ao prompt de comando usando esta sintaxe:

Makecab.exe /f <filename>.ddf

O arquivo .ddf contém informações de compressão usadas pelo programa Makecab.exe,

11.0 BIBLIOGRAFIA

1. Perry, G. **QBasic By Exemple.** Series Programming. Que Corporation. Carnel, 1992. 769p.

2. Holzner, S.; Norton, P. **Visual Basic for Windows.** Editora Campus. Rio de Janeiro. 1994. 673p.

3. Mansfield, R.; Petroutsos, E. **Visual Basic 4 – Ferramentas Poderosas.** Berkeley Brasil Editora. Sâo Paulo. 1996. 958p

4. Halvorson, M. **Microsoft Visual Basic 4 – Passo a Passo.** Editora Makron Books. São Paulo. 1996. 434p.

5. McKelvy, M.; Martinsen, R.; Webb, J. **Usando o Visual Basic 5.0 – O Guia Mais Que Completo.** Editora Campus. Rio de Janeiro. 1997. 1084p.

6. Thomas, Z.; Peterson, K.; Petersen, C.; Shields, J.; Greenberg, S.; Waite, M. **Visual Basic 4 How-to – O Guia Definitivo para o Visual Basic 4.** Editora Ciência Moderna. Rio de Janeiro. 1996

7. Queiroz, D. M.; Pinto, F. A. C. **Engenharia de Sistemas Agrícolas.** UFV. Viçosa-MG. 1992. (Apostila)

8. Microsoft Corporation. **Visual Basic 5.0 – Guia do Programador.** Editada pela Microsoft Corporation nos EUA.

9. SCOTT, J. **Visual Basic para Multimídia.** Editora Campus. Rio de Janeiro. 1995. 662p.

APÊNDICE A

SIMULAÇÃO DE SECAGEM

Elabore um programa de computador para simular a secagem de milho de um silo secador com distribuição de ar radial utilizando o modelo de Hukill.

Um silo, com distribuição de ar realizada por piso perfurado, é simulado por meio das seguintes equações:

$$\mathbf{RU = \frac{U - Ueq}{Uo - Ueq} = \frac{2^D}{2^D + 2^Y - 1}} \qquad \text{(A1)}$$

em que,

- U = teor de umidade do produto numa posição x acima do piso perfurado e após um tempo t de secagem, em decimal b.s.;
- Uo = teor de umidade inicial, decimal b.s.;
- Ueq = teor de umidade de equilíbrio, decimal b.s.;
- RU = razão de umidade do produto, adimensional;
- D = adimensional de profundidade;
- Y = adimensional de tempo.

O adimensional de profundidade, para um silo com distribuição do ar realizada por meio de piso perfurado, é calculado por meio da equação:

$$\mathbf{D = \frac{x \cdot W \cdot Lv \cdot (Uo - Ueq)}{ca \cdot m \cdot t_{1/2} \cdot (To - Tg)}} \qquad \text{(A2)}$$

em que,

- x = distância do ponto de entrada do ar até o ponto em que se deseja determinar o teor de umidade do produto, em m;
- W = peso específico do produto, expresso em Kg de matéria seca por m;
- Lv = calor latente de vaporização do produto, em Kcal/Kg de água evaporada;
- ca = calor específico do ar, Kcal/(Kg . C)
- m = fluxo de massa de ar, em Kg de ar seco/(h.m^2);
- $t_{1/2}$ = tempo de meia resposta, em horas;
- To = temperatura de secagem, em ^{0}C;
- Tg = temperatura na qual o ar está em equilíbrio com o grão com o seu teor de umidade inicial, depois de o ar ter sido resfriado ao longo de uma linha de temperatura de bulbo úmido constante, geralmente admitido como sendo igual à temperatura de bulbo úmido do ar, em ^{0}C;

O dimensional de tampo é calculado por meio da equação:

$$\mathbf{Y = \frac{t}{t_{1/2}}} \qquad \text{(A3)}$$

em que,

t = tempo transcorrido desde o início da secagem, em horas;

Para calcular o teor de umidade de equilíbrio pode-se utilizar várias equações, como por exemplo a proposta por THOMPSON et al. (1928):

$$\mathbf{Ue = 1{,}206 \cdot \left[\frac{-1n(1-UR)}{To+45{,}6}\right]^{0{,}5}} \qquad \text{(A4)}$$

em que,

UR = umidade relativa do ar de secagem, decimal.

O calor latente de vaporização pode ser calculado por meio da equação proposta por THOMPSON et al. (1968):

$$\mathbf{Lv = (606 - 0{,}57 \cdot To) \cdot [1 + 4{,}35 \cdot exp(-28 \cdot 25 \cdot Uo)]} \qquad \text{(A5)}$$

O tempo de meia resposta pode ser determinado por meio da equação proposta por QUEIROZ et alii (1982):

$$\mathbf{t1/2 = exp\left[2{,}413 - \frac{(1{,}6 \cdot Uo)}{1+Uo} + (0{,}003 \cdot To) - \frac{(0{,}1 \cdot Uo \cdot To)}{1+Uo}\right]} \qquad \text{(A6)}$$

O fluxo de massa de ar é determinado por:

$$\mathbf{m = \frac{Q \cdot 60}{A \cdot ve}} \qquad \text{(A7)}$$

em que,

Q = vazão de ar, em m^3/min;
A = área de secador, em m^2;
ve = volume específico do ar ambiente, em m^3/Kg de ar seco.

Com base nessa equações, a secagem em um silo com distribuição de ar por meio de piso perfurado por ser simulada com base nos seguintes parâmetros:

- vazão de ar, em m^3/min;
- área do secador, em m^2;
- altura da camada de grãos, em m;
- Kg de matéria seca do produto por m^3;
- teor de umidade inicial, dec. b.s;
- teor de umidade final, dec. b.s.;
- Tbs do ar de secagem, em ºC;
- Tbu do ar de secagem, em ºC;
- Volume específico do ar ambiente, m^3/Kg.

Num silo com distribuição de ar radial, o fluxo de massa de ar, m da Equação A7, varia pois a área de passagem do ar vai aumentando do centro para a periferia do silo. Se substituirmos a Equação A7 na Equação A2, o adimensional de profundidade passa a ser dado por:

$$\mathbf{D = \frac{(x \cdot A) \cdot ve \cdot W \cdot Lv \cdot (Uo - Ueq)}{ca \cdot Q \cdot 60 \cdot t_{1/2} \cdot (To - Tg)}} \qquad \text{(A8)}$$

O produto x.A da Equação A8 representa o volume de grãos em um silo de fundo plano. Para simular a secagem em um silo com distribuição de ar radial, basta dividir o volume total do silo, dado pala Equação A9, em determinado número de partes iguais, e calcular a distribuição radial do teor de umidade

de forma similar ao cálculo para um silo com distribuição de ar por fundo plano.

$$\mathbf{Vt = n \cdot \left(re^2 - ri^2\right) \cdot h} \qquad \text{(A9)}$$

em que,

Vt = volume total do silo, em m^3;
Re = raio externo do silo, em m;
Ri = raio interno do silo, em m;
H = altura do cilindro do silo, em m.

APÊNDICE B

NÚMERO DE DIAS DISPONÍVEIS PARA TRABALHO COM MÁQUINAS AGRÍCOLAS ✔

Introdução

Um dos parâmetros levados em consideração na seleção de máquinas agrícolas, para uma determinada área é o número de dias de que se dispõe para a realização das operações com máquinas agrícolas.

A estimativa do número de dias disponíveis para a realização de operações com máquinas agrícolas pode ser feita a partir do estudo de umidade do solo, realizado com base nas condições climáticas do local e das características do solo.

Para que um dia seja considerado trabalhável é necessário que satisfaça às seguintes condições:

- a umidade do solo menor ou igual a 90% da disponibilidade máxima de água que o solo pode ter;
- a precipitação ocorrida no dia inferior a 5,0 mm;
- a precipitação ocorrida no dia anterior inferior a 10,0 mm;
- não persistência de mau tempo, isto é, a precipitação do dia anterior maior que 2,0 mm e do dia considerado superior a 0,2 mm.

Balanço de água no solo

O balanço de água no solo pode ser determinado pelo método simplificado de Jensen et al., citado por ASSIS et al. (1989):

$$LAD(j) = LAD\ (j - 1) + PE\ (j) - Etr\ (j) \qquad \text{(B1)}$$

em que,

LAD (J) = lâmina de água atual disponível no solo no final do dia (j), mm;
LAD (j - 1) = lâmina de água atual disponível no solo no final do dia (j - 1), mm;
PE (j) = precipitação efetiva no dia (j), mm;
Etr (j) = evapotranspiração real, no dia (j), mm.

Se a lâmina de água disponível no solo LAD (j) for superior à sua capacidade máxima de retenção de água, o teor de umidade no solo terá valor igual à lâmina correspondente à capacidade de campo, sendo o excesso considerado como perdido por percolação ou escoamento superficial, ASSIS (1988).

Para iniciar o balanço de umidade no solo, considera-se que o solo está na capacidade de campo. Para essa situação, a lâmina total de água disponível é dada por:

$$LADt = 0{,}1 \cdot (CC - PM) \cdot dap \cdot z \qquad \text{(B2)}$$

em que,

CC = capacidade de campo, % em peso;

✔ Parte da apostila de ENG 434 – Engenharia de Sistemas Agrícolas, 1995. Elaborada por Daniel M. Queiros e Francisco de Assis C. Pinto.

PM = ponto de murcha, % em peso;
Dap = densidade do solo, g/cm^3;
Z = profundidade do perfil do solo, cm.

Para efeito da determinação dos dias trabalháveis com máquinas agrícolas, o balanço de umidade é realizado para os primeiros 15 cm de profundidade do solo, que correspondem a faixa de solo em que o rodado das máquinas agrícolas atua.

A evapotranspiração real pode ser estimada por meio da equação:

$$ETR = K \cdot Etp \tag{B3}$$

em que,

Etp = evapotranspiração potencial, mm/dia;
K = coeficiente relacionado com a disponibilidade de água no solo.

Segundo Bernardo, citado por ASSIS (1988), o coeficiente K pode ser estimado por:

$$K = \frac{\ln[LAD(j-1)+1{,}0]}{\ln(LADt+1{,}0)} \tag{B4}$$

A evapotranspiração potencial, calculada por meio do método de radiação, é dada por:

$$ETp = W \cdot \left(a + b \cdot \frac{n}{N}\right) \cdot Ra \tag{B5}$$

em que,

W = fator que depende da temperatura e da altitude;
Ra = quantidade teórica de radiação que alcançaria a superfície da terra na ausência de atmosfera, valor de Angôt (em equivalente lâmina de água), mm/dia;
n = duração do brilho solar recebido, h;
N = duração máxima possível do brilho solar diário, h;
a e b = parâmetros que caracterizam a transmitância da atmosfera.

Para latitudes de 0 a 60 graus, Glover e McCulloch, citados por ASSIS (1988), estabeleceram para a constante **b** o valor 0,52 e para a constante **a**, 0,29 cos φ, sendo φ a latitude do local, em radianos.

A quantidade teórica de radiação, que alcançaria a superfície da terra na ausência da atmosfera, é obtido por:

$$Qo = \frac{1440}{\pi} \cdot Fo \cdot \left(\frac{dm}{d}\right)^2 \cdot \left(H \cdot \mathbf{sen}\,\psi \cdot \mathbf{sen}\,\delta + \cos\psi \cdot \cos\delta \cdot \mathbf{sen}\ H\right) \tag{B6}$$

em que,

Qo = quantidade de radiação, cal/(cm^2 · dia);
Fo = constante solar, igual a 1,94 cal/(cm^2 · dia);
Dm/d = razão entre as distâncias média e instantânea entre a terra e o sol;
H = duração da metade do dia, radianos;
ψ = latitude do local, radianos;
δ = declinação do sol, radianos.

A declinação solar, calculada a partir da equação de Cooper, é dada por:

$$\delta = 23{,}45 \cdot \text{sen} \cdot \left[\frac{2 \cdot \pi}{365} \cdot (284 + nd)\right] \cdot \frac{\pi}{180} \tag{B7}$$

em que,

nd = número do dia do ano, sendo nd = 1 para 1 de janeiro e nd = 365 para 31 de dezembro.

Para a determinação da duração do meio dia, pode ser utilizada a seguinte expressão:

$$\cos H = tg(\delta) \cdot tg(\psi) \qquad (B8)$$

Na Equação B8, a latitude φ tem sinal positivo para o hemisfério norte e sinal negativo para o hemisfério sul.

A quantidade teórica de radiação que alcançaria a superfície da terra, na ausência de atmosfera, em equivalente lâmina de água, pode ser expressa por:

$$Ra = \frac{Qo}{59} \qquad (B9)$$

A duração máxima possível do brilho solar é dada por:

$$N = \frac{2 \cdot H \cdot 180}{15 \cdot \pi} \qquad (B10)$$

A razão entre as distâncias média e instantânea entre a terra e o sol ao quadrado, (dm/d^2), pode ser estimada a partir da seguinte equação:

$$(dm/d^2) = 1{,}000110 + 0{,}034221 \cdot \cos(\xi) + 0{,}00128 \cdot sen(\xi) + 0{,}000719 \cdot \cos(2\xi) + 0{,}000077 \cdot sen(2\xi) \qquad (B11)$$

sendo:

$$\xi = 2 \cdot \pi \cdot \frac{(nd - 1)}{365}$$

O fator W da Equação B5 pode ser determinado a partir da seguinte equação:

$$W = \frac{pvsbs - pvsbu}{pvsbs - pv} \qquad (B12)$$

em que,

pvsbs = pressão de vapor de saturação à temperatura de bulbo seco do ar, mmHg;
pvsbu = pressão de vapor de saturação à temperatura de bulbo úmido do ar, mmHg;
pv = pressão de vapor do ar, mmHg.

A pressão de vapor para uma dada temperatura T, em graus Celsius, pode ser obtida opor meio da equação apresentada por BROKKER et al. (1974):

$$pvs = 51{,}715 \cdot Exp\left[51{,}594 - \frac{6834}{T + 273{,}16} - 5{,}169 \cdot \ln(T + 273{,}16)\right] \qquad (B13)$$

A pressão de vapor do ar pode ser determinada por:

$$pv = \frac{Ur \cdot pvsbs}{100} \qquad (B14)$$

em que,

Ur = umidade relativa do ar, em %.

Diagrama de blocos de um programa de computador para o cálculo do número de dias disponíveis para utilização de máquinas agrícolas

Nas Figuras 1, 2 e 3 são apresentados os diagramas de blocos, referentes ao programa principal e às subrotinas, para calcular o número de dias disponíveis para o trabalho com máquinas agrícolas. Na elaboração dos diagramas foram utilizadas as seguintes variáveis:

Programa Principal

CC - capacidade de campo, % em peso;
Cond - variável que recebe resultado se o dia é 'DIA BOM' ou se é 'DIA RUIM';
dap - densidade do solo, g/cm^3;
dm/d^2 - quadrado da razão entre a distância média e instantânea entre o terra e o sol;
ETP - evapotranspiração potencial, em mm/dia;
Etr - evapotranspiração real, em mm/dia;
H - duração da metade do dia, em radianos;
J - dontador do número de dias simulados;
K - coeficiente para transformação da evapotranspiração potencial em evapotranspiração real, adimensional;
Lat - latitude do local, em graus.
LD - lâmina de água disponível no solo, em mm;
Ldant - lâmina de água disponível no dia anterior, em mm;
LDt - lâmina máxima de água disponível no solo, em mm;
N - duração máxima possível do brilho solar diário, em h;
n - duração do brilho solar diário, em h;
nd - número do dia do ano (nd = 1 para 1 de janeiro);
Ndias - número de dias em que consiste o banco de dados;
Patm - pressão atmosférica, em mmHg;
PE - precipitação no dia atual, em mm;
PEant - precipitação no dia anterior, em mm;
PM - ponto de murcha, %, em peso;
pv - pressão de vapor no ar, em mmHg;
pvsbs - pressão de vapor de saturação à temperatura de bulbo seco em mmHg;
pvsbu - pressão de vapor de saturação à temperatura de bulbo úmido em mmHg;
Qo - quantidade de radiação solar, em $cal/(cm^2 \cdot dia)$;
Ra - quantidade teórica de radiação que alcançaria a superfície da terra na ausência da atmosfera, em equivalente lâmina de água, em mm/dia;
T - temperatura de bulbo seco média diária, em ^0C;
Tbu - temperatura de bulbo úmido do ar, em ^0C;
Tempum - subrotina que calcula a temperatura de bulbo úmido do ar, a partir da temperatura de bulbo seco e umidade relativa;
Ur - umidade relativa média diária, em %;
W - fator que depende da temperatura e altitude, adimensional;
Z - profundidade do perfil do solo, em cm;
δ - declinação solar, em radianos;
ξ - ângulo referente ao dia do ano, em radianos;
φ - latitude, em radianos.

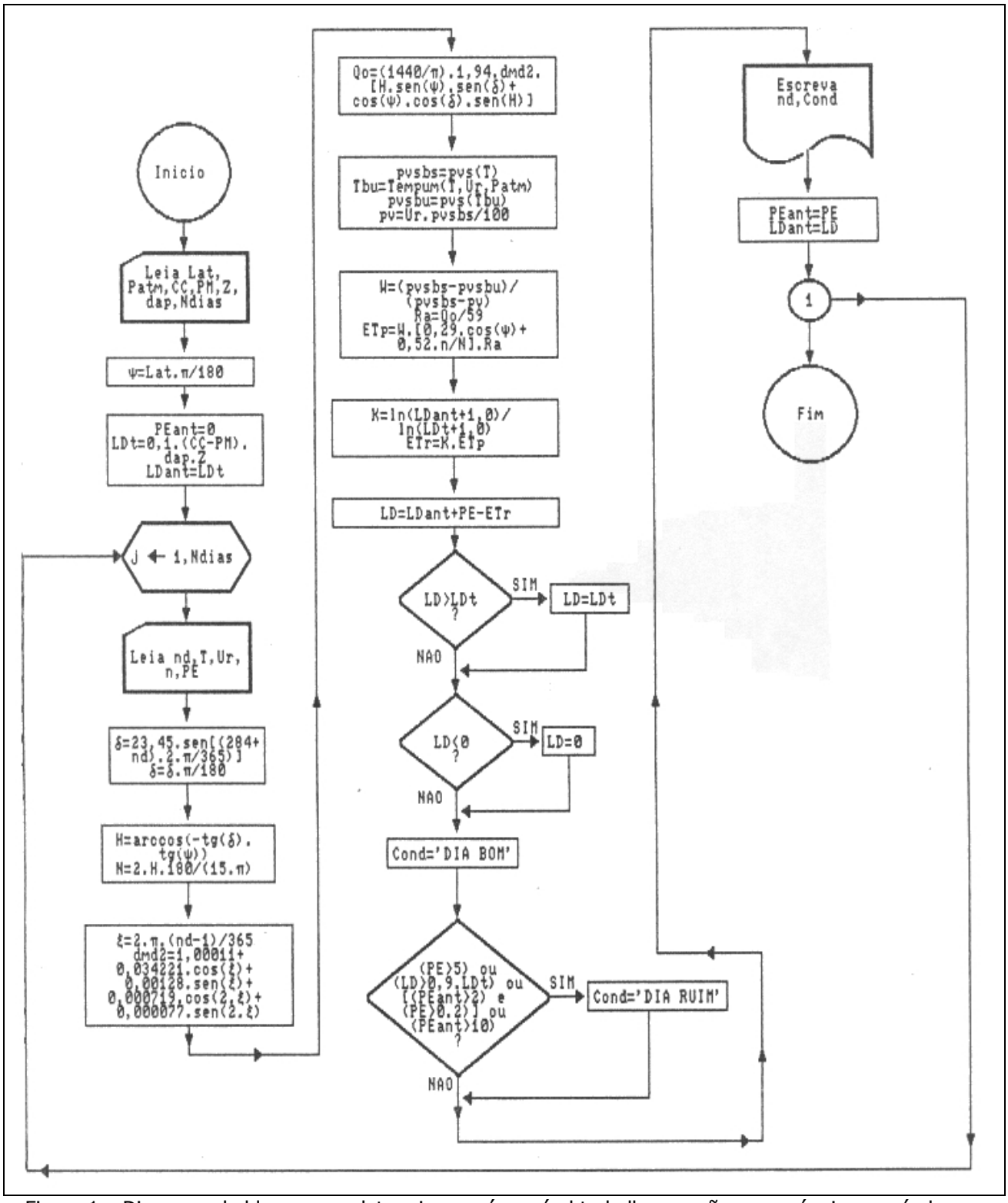


Figura 1 – Diagrama de blocos para determinar se é possível trabalhar ou não com máquinas agrícolas

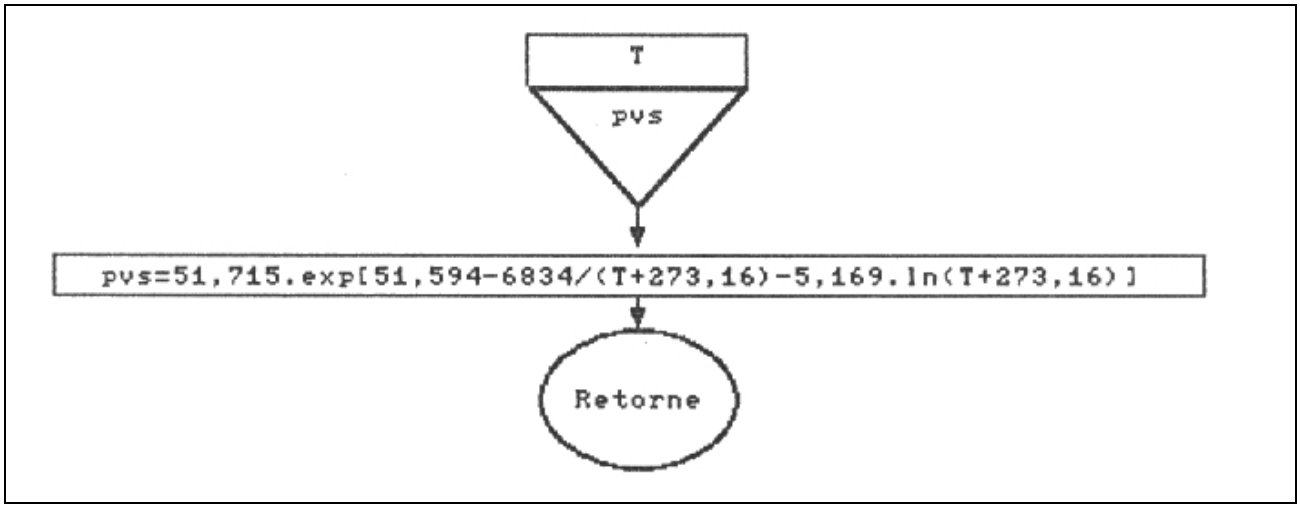


Figura 2 – Diagrama de blocos da função para cálculo da pressão de vapor de saturação

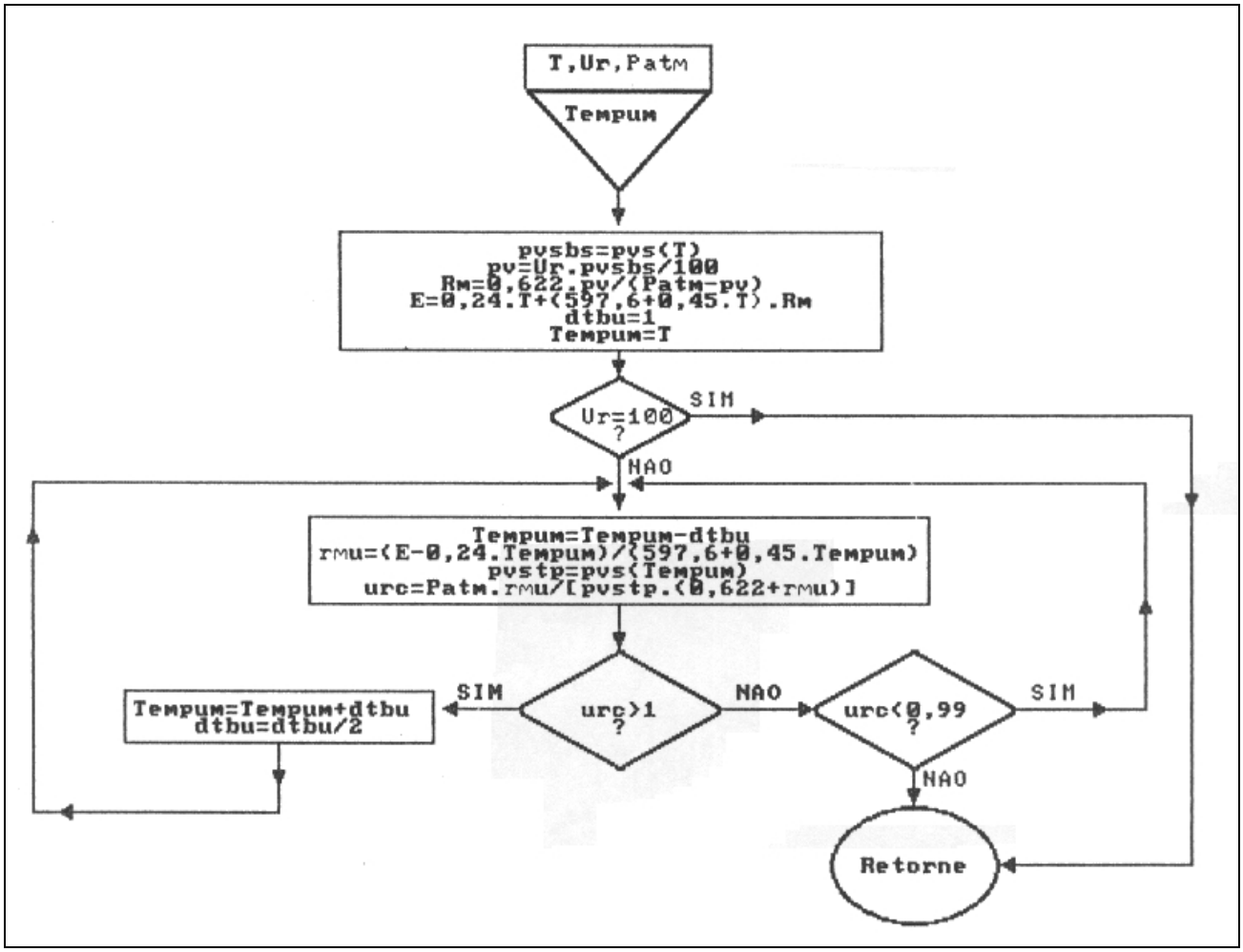


Figura 3 – Diagrama de blocos da subrotina para determinação da temperatura de bulbo úmido

Função pvs

T - temperatura, em ºC;
pvs - pressão de vapor de saturação, em mmHg.

Subrotina Tempum

dtbu - incremento de temperatura para o cálculo da temperatura de bulbo úmida, em ºc;
E - entalpia do ar, em k;cal/kg
Patm - pressão atmosférica, em mmHg;
pv - pressão de vapor atual do ar, em mmHg;
pvsbs - pressão de vapor do ar à temperatura de bulbo seco, em mmHg;
pvstp - variável auxiliar fornece valor da pressão de vapor de saturação, em mmHg;
Rm - razão de mistura do ar, em kg/kg;
Tmu - variável auxiliar que fornece valor de razão de mistura do ar, em kgkg;
T - temperatura de bulo seco, em ^{0}C;]
Tempum - temperatura de bulbo úmido do ar, em ^{0}C;
Ur - umidade relativa do ar, em%;
Urc - variável auxiliar que contém o valor de umidade relativa do ar, em decimal.

Exercício:

Uma determinada região apresenta as seguintes condições:

- capacidade de campo do solo ----------: 31% (em peso)
- ponto de murcha ----------: 22,9 %
- densidade do solo ----------: 0,85 g/cm^3
- latitude do local ----------: 19º 45' S

As condições climáticas em um dado período foram as seguintes:

Dia	**Temperatura média (^{0}C)**	**Umidade relativa média (%)**	**Precipitação (mm)**	**Duração do brilho solar (h)**
1/8	22	80	0,0	5
2/8	22	75	0,0	6
3/8	23	78	0,0	8
4/8	22	90	3,0	3
5/8	22	90	5,0	2
6/8	23	90	7,0	2
7/8	23	70	0,5	8
8/8	23	72	0,0	9
9/8	21	75	0,0	4
10/8	22	92	12,0	3
11/8	23	70	0,0	8
12/8	22	75	0,0	5
13/8	23	73	0,0	6
4/8	24	71	0,0	7
15/8	25	69	0,0	8

Considerando que o solo no dia 31/7 estava com teor de umidade equivalente à capacidade de campo, determine os dias possíveis para trabalho com máquinas agrícolas.